PlussAI NodeJ + Lymph API Documentation

Pluss Contributors

Mar 19, 2026

CONTENTS

1 API Reference **3**
1.1 Coverage Snapshot 3

2 NodeJ Overview **5**

3 Lymph Overview **7**

4 NodeJ Full API Reference **9**
4.1 Function and Method Reference 9

5 Lymph Full API Reference **141**
5.1 Function and Method Reference 141

6 Build Outputs **221**

This set follows a sklearn-style API reference pattern:

- signature + parameter-by-parameter reference
- return semantics + behavior notes
- 2-3 usage snippets where available
- verified output blocks captured from executed demos

CHAPTER

ONE

API REFERENCE

- *NodeJ Overview*
- *Lymph Overview*
- *NodeJ Full API Reference*
- *Lymph Full API Reference*

1.1 Coverage Snapshot

- NodeJ pages: **116** (methods: 102, data-types: 14)
- Lymph pages: **85** (methods: 61, data-types: 24)
- Combined pages: **201**

CHAPTER

TWO

NODEJ OVERVIEW

NodeJ is the explicit classical ML core in **PlussAI**.

Primary public modules:

- types / status / options
- dataset / data
- split
- prep transformers
- model families
- builders
- compose::Flow
- metrics / tune
- online
- io

The full method-by-method surface is documented in *NodeJ Full API Reference*.

CHAPTER

THREE

LYMPH OVERVIEW

Lymph is the convenience layer over NodeJ, with high-level wrappers for:

- workflow-oriented training/inference
- DataFrame-oriented preprocessing and evaluation
- quick one-call builder/trainer helpers
- tuning and cross-validation helpers
- online wrappers

The full method-by-method surface is documented in *Lymph Full API Reference*.

CHAPTER

FOUR

NODEJ FULL API REFERENCE

NodeJ method-centric reference generated from public headers. Each page follows **Signature -> Notes -> Parameters -> Returns -> Options -> Examples -> Output -> Source**. Options/enums are documented under the methods that consume them.

- Method pages: **102**
- Data-type pages: **14**
- Overloads covered: **105**

4.1 Function and Method Reference

4.1.1 NodeJ Full API Reference: Method Index

Total method pages: **102** Total data-type pages: **14**

Data Types

`CrossAssessReport`

Signature

```
struct CrossAssessReport;
```

Notes

Structured report object carrying metrics/diagnostics from training/evaluation routines.

Options

Public fields/options on this type:

- `std::vector<FoldAssessResult> folds{}`
- `std::vector<MetricKind> metrics{}`
- `Series mean_metrics = Series::undefined()`
- `Series std_metrics = Series::undefined()`

Examples

```
CrossAssessReport cfg{};
```

Source

- PlussAI/include/pluss/ai/nodej/metrics.h:16

FeatureBatch

Signature

```
struct FeatureBatch;
```

Notes

Container for model input features and their observation layout. Use `layout` to declare whether rows or columns represent observations.

Options

Public fields/options on this type:

- `Series x = Series::undefined()`
- `ObservationLayout layout = ObservationLayout::Rows`

Examples

Example 1

Source: PlussAI/examples/NodeJ/core/nodej_prep_split_demo.cpp

```
  const Series x_raw = pluss::make_series(std::vector<double>{
                                            1.0,  2.0, std::numeric_limits<double>
↪::quiet_NaN(),
                                            2.0,  2.5, 1.0,
                                            3.0,  std::numeric_limits<double>::quiet_
↪NaN(), 0.0,
                                            4.0,  3.5, 1.0,
                                            5.0,  4.0, 0.0,
                                            6.0,  4.5, 1.0},
                                          {6, 3},
                                          f64());
  const FeatureBatch dense{.x = x_raw, .layout = ObservationLayout::Rows};
  const Series labels = pluss::make_series(std::vector<double>{0.0, 0.0, 0.0, 1.0, 1.0,␣
↪1.0}, {6}, f64());
```

Output

```
NodeJ split + prep demo
holdout train idx count=4 test idx count=2
strat holdout train idx count=4 test idx count=2
kfold folds=3
```

(continues on next page)

(continued from previous page)

```
imputed shape=[6, 3]
standardized shape=[6, 3]
minmax shape=[6, 3]
robust shape=[6, 3]
neighbor-imputed shape=[6, 3]
iterative-imputed shape=[6, 3]
vector-normalized shape=[6, 3]
maxabs shape=[6, 3]
...
```

Example 2

Source: PlussAI/examples/NodeJ/core/nodej_unsupervised_demo.cpp

```
  std::vector<double> xvals{};
  for (int i = 0; i < 30; ++i) {
    xvals.push_back(-3.0 + 0.1 * i);
    xvals.push_back(-2.5 + 0.05 * i);
  }
  for (int i = 0; i < 30; ++i) {
    xvals.push_back(2.0 + 0.08 * i);
    xvals.push_back(2.5 + 0.06 * i);
  }
  const FeatureBatch features{
      .x = pluss::make_series(xvals, {60, 2}, cfg),
      .layout = ObservationLayout::Rows,
  };
```

Output

```
kmeans labels (first 10): 0 0 0 0 0 0 0 0 0 0
pca output shape: [80, 2]
nodej_unsupervised_demo complete
```

Example 3

Source: PlussAI/examples/NodeJ/lymph/lymph_unsupervised_demo.cpp

```
  const auto cfg = pluss::Config{}.with_dtype(pluss::ScalarType::Float64).with_
↪platform(pluss::pCPU);
  const Series x = pluss::make_series(std::vector<double>{
                                        -2.0, -1.0,
                                        -1.8, -0.9,
                                        -1.6, -0.8,
                                         1.2,  1.1,
                                         1.4,  1.2,
                                         1.6,  1.3},
                                      {6, 2},
                                      cfg);
  const FeatureBatch batch{
      .x = x,
```

(continues on next page)

(continued from previous page)

```
    .layout = ObservationLayout::Rows,
};
```

Output

```
lymph kmeans labels: 1 1 1 0 0 0
lymph pca output shape: [40, 2]
lymph_unsupervised_demo complete
```

Source

- PlussAI/include/pluss/ai/nodej/dataset.h:8

FlowBundle

Signature

```
struct FlowBundle;
```

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Options

Public fields/options on this type:

- FlowBundleMetadata metadata{}
- ModelSpec model{}
- std::vector<TransformSpec> transforms{}
- bool trained = false
- bool has_snapshot = false
- std::vector<std::vector<Series>> transform_states{}
- std::vector<Series> model_state{}
- TrainOptions train_options{}
- LabeledBatch train_batch{}

Examples

```
FlowBundle cfg{};
```

Source

- PlussAI/include/pluss/ai/nodej/io.h:19

FlowBundleMetadata

Signature

```
struct FlowBundleMetadata;
```

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Options

Public fields/options on this type:

- `std::uint32_t schema_version = k_flow_bundle_schema`
- `std::string producer = "pluss.ai.nodej"`
- `std::string producer_version = "0.1.0"`
- `std::uint64_t created_unix_nanos = 0`

Examples

```
FlowBundleMetadata cfg{};
// cfg.schema_version = ...;
```

Source

- PlussAI/include/pluss/ai/nodej/io.h:12

FoldAssessResult

Signature

```
struct FoldAssessResult;
```

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Options

Public fields/options on this type:

- `Series metrics = Series::undefined()`
- `Series train_indices = Series::undefined()`
- `Series test_indices = Series::undefined()`

Examples

```
FoldAssessResult cfg{};
// cfg.metrics = ...;
```

Source

- PlussAI/include/pluss/ai/nodej/metrics.h:10

IndexSplit

Signature

```
struct IndexSplit;
```

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Options

Public fields/options on this type:

- Series train_indices = Series::undefined()
- Series test_indices = Series::undefined()

Examples

Example 1

Source: PlussAI/examples/NodeJ/core/nodej_end_to_end_hour_workflow_showcase.cpp

```
  // This section performs deterministic holdout split using core split API.
  SplitOptions holdout_options{};
  holdout_options.holdout_fraction = 0.2;
  holdout_options.shuffle = true;
  holdout_options.random.seed = 42;
  holdout_options.random.deterministic = true;

  StatusOr<IndexSplit> holdout_or = holdout_split(frame.ncases(), holdout_options);
  if (!holdout_or.ok()) {
    std::cerr << "holdout_split failed: " << holdout_or.status().message << "\n";
    return 1;
  }
  const IndexSplit holdout = holdout_or.move_value();

  const std::vector<std::int64_t> holdout_train_idx = holdout.train_indices.
↪to(pluss::pCPU, ScalarType::Int64)
                                                      .reshape({holdout.train_
↪indices.numel()})
                                                      .to_vec<std::int64_t>();
  const std::vector<std::int64_t> holdout_test_idx = holdout.test_indices.to(pluss::pCPU,
```

(continues on next page)

(continued from previous page)

```
↪ ScalarType::Int64)
                                                        .reshape({holdout.test_indices.
↪numel()})
                                                        .to_vec<std::int64_t>();

  const DataFrame train_frame_full = frame.take(holdout_train_idx);
  const DataFrame test_frame_full = frame.take(holdout_test_idx);

  const DataFrame train_x = train_frame_full.features({
      "season", "year", "month", "hour", "is_holiday", "weekday", "is_workingday",
↪"weather_condition",
      "temp", "atemp", "humidity", "windspeed", "temp_sq", "atemp_sq", "humidity_sq",
↪"windspeed_sq",
      "temp_humidity_interaction", "temp_windspeed_interaction", "atemp_humidity_
↪interaction", "atemp_windspeed_interaction",
      "temp_minus_atemp", "hour_centered", "hour_centered_sq", "month_centered", "month_
↪centered_sq", "weather_sq",
  });
  const DataFrame test_x = test_frame_full.features({
      "season", "year", "month", "hour", "is_holiday", "weekday", "is_workingday",
↪"weather_condition",
      "temp", "atemp", "humidity", "windspeed", "temp_sq", "atemp_sq", "humidity_sq",
↪"windspeed_sq",
```

Source

- PlussAI/include/pluss/ai/nodej/split.h:11

LabeledBatch

Signature

```
struct LabeledBatch;
```

Notes

Container for supervised batches: features (`x`) plus aligned labels (`y`) and optional `sample_weights`. Use this as the primary input to training, validation, and tuning workflows.

Options

Public fields/options on this type:

- `FeatureBatch features{}`
- `Series y = Series::undefined()`
- `Series sample_weights = Series::undefined()`

Examples

Example 1

Source: PlussAI/examples/NodeJ/core/nodej_cv_tune_demo.cpp

```
LabeledBatch make_xor_batch() {
  const Series x = pluss::make_series(std::vector<double>{
                                        -1.0, -1.0,
                                        -1.0,  1.0,
                                         1.0, -1.0,
                                         1.0,  1.0,
                                        -0.9, -0.9,
                                        -0.9,  0.9,
                                         0.9, -0.9,
                                         0.9,  0.9,
                                        -1.2, -1.1,
                                        -1.1,  1.2,
                                         1.1, -1.2,
                                         1.2,  1.1,
                                        -0.4, -0.7,
                                        -0.5,  0.8,
                                         0.6, -0.8,
                                         0.5,  0.7},
                                      {16, 2},
```

Output

```
NodeJ CV + tune demo
linear mean metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0.25 0.25 0.
↪208333 0.25 nan 0.0625 0.960532
linear std metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0.176777 0.
↪176777 0.216506 0.25 nan 0.108253 0.192821
kernel mean metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 1 1 1 1 1 1␣
↪0.358895
kernel std metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0 0 0 0 0 0␣
↪0.0358183
sweep trial objectives: 0.25 1
sweep best index: 1
probe best index: 1
regression quantile sweep objectives: 0.0356052 0.736518
regression quantile sweep best index: 0
max |bounded score|: 0
NodeJ CV + tune demo completed.
```

Example 2

Source: PlussAI/examples/NodeJ/core/nodej_flow_bundle_demo.cpp

```
  const auto cfg = pluss::Config{}.with_dtype(pluss::ScalarType::Float64).with_
↪platform(pluss::pCPU);
  const Series x = pluss::make_series({
                                        -2.0, -1.0,
                                        -1.0, -0.5,
```

(continues on next page)

(continued from previous page)

```
                                          1.0,  0.5,
                                          2.0,  1.0,
                                         -1.5, -0.6,
                                          1.5,  0.7},
                                     {6, 2},
                                     cfg);
  const Series y = pluss::make_series({0.0, 0.0, 1.0, 1.0, 0.0, 1.0}, {6}, cfg);
  const LabeledBatch batch{
      .features = FeatureBatch{.x = x, .layout = ObservationLayout::Rows},
      .y = y,
  };
```

Output

```
saved flow bundle to: <repo_root>/PlussAI/examples/NodeJ/trained_models/nodej_flow_
↪bundle_demo.bin
flow bundle roundtrip exact prediction parity: true
nodej_flow_bundle_demo complete
```

Example 3

Source: PlussAI/examples/NodeJ/core/nodej_flow_demo.cpp

```
  {
    const Series x = pluss::make_series(std::vector<double>{
                                          -3.0, -1.0,
                                          -2.0, -1.0,
                                          std::numeric_limits<double>::quiet_NaN(), -0.
↪8,
                                          -1.0, -0.6,
                                          1.0, 0.6,
                                          2.0, 0.8,
                                          3.0, 1.0,
                                          std::numeric_limits<double>::quiet_NaN(), 0.
↪7},
                                       {8, 2},
                                       f64());
    const Series y = pluss::make_series(std::vector<double>{0.0, 0.0, 0.0, 0.0, 1.0, 1.0,
↪ 1.0, 1.0}, {8}, f64());
    const LabeledBatch train{
        .features = FeatureBatch{.x = x, .layout = ObservationLayout::Rows},
        .y = y,
    };
```

Output

```
NodeJ Flow demo
classification flow metrics [accuracy, logloss]: 1 0.000228657
regression flow metrics [mse, r2]: 4.46648e-17 1
NodeJ Flow demo completed.
```

Source

- PlussAI/include/pluss/ai/nodej/dataset.h:23

ModelSpec

Signature

```
struct ModelSpec;
```

Notes

Model construction contract. Set task, family, and family-specific options before calling make_classifier/make_regressor.

Options

Public fields/options on this type:

- TaskKind task = TaskKind::Regression
- ModelFamily family = ModelFamily::Linear
- FamilyOptions options = LinearFamilyOptions{}

Examples

Example 1

Source: PlussAI/examples/NodeJ/core/nodej_cv_tune_demo.cpp

```
ModelSpec linear_logistic_spec() {
  return ModelSpec{
      .task = TaskKind::Classification,
      .family = ModelFamily::Linear,
      .options = LinearFamilyOptions{
          .objective = LinearObjective::Logistic,
          .regularization = RegularizationKind::L2,
          .regularization_strength = 1e-6,
          .fit_intercept = true,
      },
  };
}
```

Output

```
NodeJ CV + tune demo
linear mean metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0.25 0.25 0.
↪208333 0.25 nan 0.0625 0.960532
linear std metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0.176777 0.
↪176777 0.216506 0.25 nan 0.108253 0.192821
kernel mean metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 1 1 1 1 1 1␣
↪0.358895
kernel std metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0 0 0 0 0 0␣
↪0.0358183
```

(continues on next page)

(continued from previous page)

```
sweep trial objectives: 0.25 1
sweep best index: 1
probe best index: 1
regression quantile sweep objectives: 0.0356052 0.736518
regression quantile sweep best index: 0
max |bounded score|: 0
NodeJ CV + tune demo completed.
```

Example 2

Source: PlussAI/examples/NodeJ/core/nodej_dataframe_bridge_demo.cpp

```
  ModelSpec spec{
      .task = TaskKind::Regression,
      .family = ModelFamily::Linear,
      .options = LinearFamilyOptions{
          .objective = LinearObjective::LeastSquares,
          .regularization = RegularizationKind::L2,
          .regularization_strength = 1e-8,
          .fit_intercept = true,
      },
  };
```

Output

```
NodeJ DataFrame bridge demo
adapted batch: samples=6 features=4 weighted=yes
linear metrics [mse, r2]: 9.43684e-08 1
NodeJ DataFrame bridge demo completed.
```

Example 3

Source: PlussAI/examples/NodeJ/core/nodej_flow_bundle_demo.cpp

```
  ModelSpec spec{
      .task = TaskKind::Classification,
      .family = ModelFamily::Linear,
      .options = LinearFamilyOptions{
          .objective = LinearObjective::Logistic,
          .regularization = RegularizationKind::L2,
          .regularization_strength = 1e-6,
          .fit_intercept = true,
      },
  };
```

Output

```
saved flow bundle to: <repo_root>/PlussAI/examples/NodeJ/trained_models/nodej_flow_
↪bundle_demo.bin
flow bundle roundtrip exact prediction parity: true
nodej_flow_bundle_demo complete
```

Source

- PlussAI/include/pluss/ai/nodej/families.h:126

RandomState

Signature

```
struct RandomState;
```

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Options

Public fields/options on this type:

- std::uint64_t seed = 0
- bool deterministic = true

Examples

```
RandomState cfg{};
// cfg.seed = ...;
```

Source

- PlussAI/include/pluss/ai/nodej/options.h:77

SplitPlan

Signature

```
struct SplitPlan;
```

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Options

Public fields/options on this type:

- std::vector<IndexSplit> folds{}

Examples

```
SplitPlan cfg{};
```

Source

- PlussAI/include/pluss/ai/nodej/split.h:16

Status

Signature

```
struct Status;
```

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Options

Public fields/options on this type:

- StatusCode code = StatusCode::Ok
- std::string message{}

Examples

Example 1

Source: PlussAI/examples/NodeJ/core/nodej_dataframe_bridge_demo.cpp

```
      .family = ModelFamily::Linear,
      .options = LinearFamilyOptions{
          .objective = LinearObjective::LeastSquares,
          .regularization = RegularizationKind::L2,
          .regularization_strength = 1e-8,
          .fit_intercept = true,
      },
  };

  auto reg_or = make_regressor(spec);
  if (!require_ok(reg_or, "make_regressor")) return 1;
  auto reg = reg_or.move_value();
  Status st = reg->learn(batch);
  if (!st.ok()) {
    std::cerr << "learn failed: " << st.message << "\n";
    return 1;
  }

  auto metrics_or = reg->assess(batch, AssessOptions{.metrics = {MetricKind::Mse,␣
↪MetricKind::R2}});
  if (!require_ok(metrics_or, "assess")) return 1;
  print_metrics("linear metrics [mse, r2]", metrics_or.value());

  std::cout << "NodeJ DataFrame bridge demo completed.\n";
  return 0;
}
```

Output

```
NodeJ DataFrame bridge demo
adapted batch: samples=6 features=4 weighted=yes
linear metrics [mse, r2]: 9.43684e-08 1
NodeJ DataFrame bridge demo completed.
```

Source

- PlussAI/include/pluss/ai/nodej/status.h:18

TransformSpec

Signature

```
struct TransformSpec;
```

Notes

Transformer construction contract. Choose a transform kind and configure its options before adding it to a flow/workflow.

Options

Public fields/options on this type:

- TransformKind kind = TransformKind::StandardScaler
- TransformOptions options = prep::StandardScalerOptions{}
- std::vector<std::int64_t> feature_route{}

Examples

```
TransformSpec cfg{};
// cfg.kind = ...;
```

Source

- PlussAI/include/pluss/ai/nodej/compose.h:47

TuneReport

Signature

```
struct TuneReport;
```

Notes

Structured report object carrying metrics/diagnostics from training/evaluation routines.

Options

Public fields/options on this type:

- `std::vector<TuneTrialResult> trials{}`
- `std::int64_t best_index = -1`
- `MetricKind objective_metric = MetricKind::Accuracy`
- `TuneObjectivePolicy objective_policy = TuneObjectivePolicy::Auto`

Examples

Example 1

Source: `PlussAI/examples/NodeJ/core/nodej_end_to_end_hour_workflow_showcase.cpp`

```
  StatusOr<TuneReport> broad_sweep_or = sweep(
      broad_specs,
      train_batch,
      cv_split,
      train_options,
      assess_options,
      MetricKind::Rmse,
      TuneObjectivePolicy::Minimize);
  if (!broad_sweep_or.ok()) {
    std::cerr << "sweep(broad) failed: " << broad_sweep_or.status().message << "\n";
    return 1;
  }
  TuneReport broad_sweep = broad_sweep_or.move_value();
```

Source

- `PlussAI/include/pluss/ai/nodej/tune.h:25`

TuneTrialResult

Signature

```
struct TuneTrialResult;
```

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Options

Public fields/options on this type:

- `ModelSpec spec{}`
- `Status status = Status::success()`
- `CrossAssessReport assess{}`
- `double objective = std::numeric_limits<double>::quiet_NaN()`

Examples

```
TuneTrialResult cfg{};
```

Source

- PlussAI/include/pluss/ai/nodej/tune.h:18

Methods

apply_learn_map

Signature

```
[[nodiscard]] StatusOr<FeatureBatch> apply_learn_map(const FeatureBatch& batch, const␣
↪TrainOptions& learn_options, const InferOptions& infer_options);
```

Notes

Fits model/transform state from the provided training data. Call this before inference-oriented methods.

Parameters

- batch : const FeatureBatch& Input feature data used by this call.
- infer_options : const InferOptions& Configuration object controlling algorithm behavior and safety defaults.
- learn_options : const TrainOptions& Configuration object controlling algorithm behavior and safety defaults.

Returns

- Returns StatusOr<FeatureBatch>. Check .ok() first; on success read .value() for the computed result.

Options

Option semantics used by this method:

- FeatureBatch (struct):
  - Series x = Series::undefined() Option field used by this method.
  - ObservationLayout layout = ObservationLayout::Rows Declares whether observations are represented by rows or columns.
- InferOptions (struct):
  - ObservationLayout layout = ObservationLayout::Rows Declares whether observations are represented by rows or columns.
  - MissingPolicy missing = MissingPolicy::Reject Option field used by this method.
  - UnknownCategoryPolicy unknown_categorical = UnknownCategoryPolicy::Error Controls fold/trial/count sizing.
  - double positive_threshold = 0.5 Option field used by this method.
- TrainOptions (struct):

- `ObservationLayout layout = ObservationLayout::Rows` Declares whether observations are represented by rows or columns.
- `MissingPolicy missing = MissingPolicy::Reject` Option field used by this method.
- `UnknownCategoryPolicy unknown_categorical = UnknownCategoryPolicy::Error` Controls fold/trial/count sizing.
- `std::int64_t max_iterations = 1000` Maximum optimization iterations.
- `double tolerance = 1e-6` Convergence tolerance.
- `bool warm_start = false` Option field used by this method.
- `bool track_diagnostics = true` Controls fold/trial/count sizing.
- `RandomState random{}` Random seed for reproducible behavior.
- `Series sample_weights = Series::undefined()` Optional column used as per-row training weight.
- `ClassWeightMode class_weight = ClassWeightMode::None` Option field used by this method.
- `Series class_weights = Series::undefined()` Option field used by this method.
- `ScoreCalibration calibration = ScoreCalibration::None` Option field used by this method.
- `std::int64_t calibration_iterations = 400` Controls fold/trial/count sizing.
- `double calibration_tolerance = 1e-6` Controls fold/trial/count sizing.
- `Series parameter_lower_bounds = Series::undefined()` Option field used by this method.
- `Series parameter_upper_bounds = Series::undefined()` Option field used by this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
auto out = apply_learn_map(/* args */);
```

Source

- `PlussAI/include/pluss/ai/nodej/compose.h:124`

apply_map

Signature

```
[[nodiscard]] StatusOr<FeatureBatch> apply_map(const FeatureBatch& batch, const␣
↪InferOptions& options) const;
```

Notes

Applies preprocessing/transformation behavior configured by the current options.

Parameters

- `batch` : `const FeatureBatch&` Input feature data used by this call.
- `options` : `const InferOptions&` Configuration object controlling algorithm behavior and safety defaults.

Returns

- Returns `StatusOr<FeatureBatch>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

Option semantics used by this method:

- `FeatureBatch` (`struct`):
  - `Series x = Series::undefined()` Option field used by this method.
  - `ObservationLayout layout = ObservationLayout::Rows` Declares whether observations are represented by rows or columns.
- `InferOptions` (`struct`):
  - `ObservationLayout layout = ObservationLayout::Rows` Declares whether observations are represented by rows or columns.
  - `MissingPolicy missing = MissingPolicy::Reject` Option field used by this method.
  - `UnknownCategoryPolicy unknown_categorical = UnknownCategoryPolicy::Error` Controls fold/trial/count sizing.
  - `double positive_threshold = 0.5` Option field used by this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
auto out = apply_map(/* args */);
```

Source

- `PlussAI/include/pluss/ai/nodej/compose.h:123`

check_ready_for_infer

Signature

```
[[nodiscard]] Status check_ready_for_infer(const char* where) const;
```

Notes

Computes predictions from prepared inputs using the currently learned state.

Parameters

- `where` : `const char*` Method argument; see signature type for accepted form.

Returns

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
auto out = check_ready_for_infer(/* args */);
```

Source

- PlussAI/include/pluss/ai/nodej/compose.h:127

ClassifierBase::task_kind

Signature

```
[[nodiscard]] TaskKind ClassifierBase::task_kind(self) const noexcept override;
```

Notes

Instance method on `self`.

Parameters

None.

Returns

- Returns `TaskKind`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
ClassifierBase classifierBase{};
auto out = classifierBase.task_kind(/* args */);
```

Source

- PlussAI/include/pluss/ai/nodej/interfaces.h:30

EstimatorBase::dump_state

Signature

```
[[nodiscard]] virtual StatusOr<std::vector<Series>> EstimatorBase::dump_state(self)␣
↪const;
```

Notes

Instance method on `self`.

Parameters

None.

Returns

- Returns `StatusOr<std::vector<Series>>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1

Source: `PlussAI/examples/NodeJ/core/nodej_end_to_end_hour_workflow_showcase.cpp`

```
    return 1;
  }
  Series train_maxabs_scaled = train_maxabs_scaled_or.move_value();

  StatusOr<Series> test_maxabs_scaled_or = maxabs_scaler.map(test_simple_batch);
  if (!test_maxabs_scaled_or.ok()) {
    std::cerr << "MaxAbsScaler.map(test) failed: " << test_maxabs_scaled_or.status().
↪message << "\n";
    return 1;
  }
  Series test_maxabs_scaled = test_maxabs_scaled_or.move_value();

  // This section saves compact prep-state diagnostics.
  StatusOr<std::vector<Series>> simple_state_or = simple_imputer.dump_state();
  if (!simple_state_or.ok()) {
    std::cerr << "SimpleImputer.dump_state failed: " << simple_state_or.status().message
↪<< "\n";
    return 1;
  }
  std::vector<Series> simple_state = simple_state_or.move_value();

  StatusOr<std::vector<Series>> knn_state_or = knn_imputer.dump_state();
  if (!knn_state_or.ok()) {
    std::cerr << "NeighborImputer.dump_state failed: " << knn_state_or.status().message <
↪< "\n";
    return 1;
  }
  std::vector<Series> knn_state = knn_state_or.move_value();

  StatusOr<std::vector<Series>> iter_state_or = iterative_imputer.dump_state();
  if (!iter_state_or.ok()) {
```

(continues on next page)

(continued from previous page)

```
    std::cerr << "IterativeImputer.dump_state failed: " << iter_state_or.status().
↪message << "\n";
    return 1;
  }
  std::vector<Series> iter_state = iter_state_or.move_value();
```

Source

- PlussAI/include/pluss/ai/nodej/interfaces.h:19

EstimatorBase::load_state

Signature

```
virtual Status EstimatorBase::load_state(self, const std::vector<Series>& state);
```

Notes

Loads a previously persisted artifact and restores its inference-ready state.

Parameters

- `state` : `const std::vector<Series>&` Randomization control for reproducible behavior.

Returns

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
EstimatorBase estimatorBase{};
auto out = estimatorBase.load_state(/* args */);
```

Source

- PlussAI/include/pluss/ai/nodej/interfaces.h:22

EstimatorBase::task_kind

Signature

```
[[nodiscard]] virtual TaskKind EstimatorBase::task_kind(self) const noexcept = 0;
```

Notes

Instance method on `self`.

Parameters

None.

Returns

- Returns `TaskKind`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
EstimatorBase estimatorBase{};
auto out = estimatorBase.task_kind(/* args */);
```

Source

- `PlussAI/include/pluss/ai/nodej/interfaces.h:11`

`EstimatorBase::trained`

Signature

```
[[nodiscard]] virtual bool EstimatorBase::trained(self) const noexcept = 0;
```

Notes

Boolean state check for object readiness.

Parameters

None.

Returns

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
EstimatorBase estimatorBase{};
auto out = estimatorBase.trained(/* args */);
```

Source

- PlussAI/include/pluss/ai/nodej/interfaces.h:12

FeatureBatch::defined

Signature

```
[[nodiscard]] bool FeatureBatch::defined(self) const noexcept;
```

Notes

Boolean state check for object readiness.

Parameters

None.

Returns

- Returns bool.

Options

Option semantics used by this method:

- FeatureBatch (struct):
  - Series x = Series::undefined() Option field used by this method.
  - ObservationLayout layout = ObservationLayout::Rows Declares whether observations are represented by rows or columns.

Examples

Example 1

Source: PlussAI/examples/NodeJ/core/nodej_dataframe_bridge_demo.cpp

```
options.feature_names = {"x0", "x1", "category", "ts", "target", "weight"};
options.label_feature = "target";
options.sample_weight_feature = "weight";
options.note_policy = DataNotePolicy::Hash;
options.thread_policy = DataThreadPolicy::CastTemporalToSeries;

auto batch_or = labeled_batch_from_dataframe(frame, options);
if (!require_ok(batch_or, "labeled_batch_from_dataframe")) return 1;

const LabeledBatch& batch = batch_or.value();
std::cout << "adapted batch: samples=" << batch.features.sample_count()
          << " features=" << batch.features.feature_count()
          << " weighted=" << (batch.sample_weights.defined() ? "yes" : "no") << "\n";

ModelSpec spec{
    .task = TaskKind::Regression,
```

(continues on next page)

(continued from previous page)

```
      .family = ModelFamily::Linear,
      .options = LinearFamilyOptions{
          .objective = LinearObjective::LeastSquares,
          .regularization = RegularizationKind::L2,
          .regularization_strength = 1e-8,
          .fit_intercept = true,
      },
  };

  auto reg_or = make_regressor(spec);
  if (!require_ok(reg_or, "make_regressor")) return 1;
  auto reg = reg_or.move_value();
  Status st = reg->learn(batch);
  if (!st.ok()) {
    std::cerr << "learn failed: " << st.message << "\n";
    return 1;
  }
```

Output

```
NodeJ DataFrame bridge demo
adapted batch: samples=6 features=4 weighted=yes
linear metrics [mse, r2]: 9.43684e-08 1
NodeJ DataFrame bridge demo completed.
```

Example 2

Source: PlussAI/examples/NodeJ/lymph/lymph_online_demo.cpp

```
        .regularization_strength = 1e-6,
        .fit_intercept = true,
    };
    online_options.learning_rate = 0.08;

    TrainOptions warm_opts{};
    warm_opts.max_iterations = 80;

    auto model = pluss::ai::lymph::online_factory::linear_classifier(online_options);
    model.learn_(warm, warm_opts).advance_(stream);

    const auto metrics = model.assess(full);
    const Series metric_values = metrics.accuracy.defined() ? metrics.accuracy : metrics.
↪values;

    std::cout << "lymph online seen_samples=" << model.seen_samples() << '\n';
    std::cout << "lymph online metrics [accuracy]: ";
    print_vec(metric_values);
    return 0;
  } catch (const std::exception& ex) {
    std::cerr << "lymph_online_demo failed: " << ex.what() << '\n';
    return 1;
  }
```

(continues on next page)

(continued from previous page)

```
}
```

Output

```
lymph online seen_samples=1660
lymph online metrics [accuracy]: 0.9875
```

Source

- PlussAI/include/pluss/ai/nodej/dataset.h:12

FeatureBatch::feature_count

Signature

```
[[nodiscard]] std::int64_t FeatureBatch::feature_count(self) const noexcept;
```

Notes

Reports a cardinality/count statistic from the current object state.

Parameters

None.

Returns

- Returns `std::int64_t`.

Options

Option semantics used by this method:

- `FeatureBatch` (`struct`):
  - `Series x = Series::undefined()` Option field used by this method.
  - `ObservationLayout layout = ObservationLayout::Rows` Declares whether observations are represented by rows or columns.

Examples

Example 1

Source: `PlussAI/examples/NodeJ/core/nodej_dataframe_bridge_demo.cpp`

```
DataFrameBatchOptions options{};
options.feature_names = {"x0", "x1", "category", "ts", "target", "weight"};
options.label_feature = "target";
options.sample_weight_feature = "weight";
options.note_policy = DataNotePolicy::Hash;
options.thread_policy = DataThreadPolicy::CastTemporalToSeries;
```

(continues on next page)

(continued from previous page)

```
  auto batch_or = labeled_batch_from_dataframe(frame, options);
  if (!require_ok(batch_or, "labeled_batch_from_dataframe")) return 1;

  const LabeledBatch& batch = batch_or.value();
  std::cout << "adapted batch: samples=" << batch.features.sample_count()
            << " features=" << batch.features.feature_count()
            << " weighted=" << (batch.sample_weights.defined() ? "yes" : "no") << "\n";

  ModelSpec spec{
      .task = TaskKind::Regression,
      .family = ModelFamily::Linear,
      .options = LinearFamilyOptions{
          .objective = LinearObjective::LeastSquares,
          .regularization = RegularizationKind::L2,
          .regularization_strength = 1e-8,
          .fit_intercept = true,
      },
  };

  auto reg_or = make_regressor(spec);
  if (!require_ok(reg_or, "make_regressor")) return 1;
  auto reg = reg_or.move_value();
  Status st = reg->learn(batch);
  if (!st.ok()) {
    std::cerr << "learn failed: " << st.message << "\n";
    return 1;
```

Output

```
NodeJ DataFrame bridge demo
adapted batch: samples=6 features=4 weighted=yes
linear metrics [mse, r2]: 9.43684e-08 1
NodeJ DataFrame bridge demo completed.
```

Example 2

Source: PlussAI/examples/NodeJ/core/nodej_online_demo.cpp

```
using pluss::ai::nodej::online::OnlineLinearOptions;

Config f64() {
  return Config{}.with_dtype(ScalarType::Float64).with_platform(pluss::pCPU);
}

Config i64() {
  return Config{}.with_dtype(ScalarType::Int64).with_platform(pluss::pCPU);
}

LabeledBatch slice_rows(const LabeledBatch& batch, std::int64_t start, std::int64_t␣
↪count) {
  const Series idx = pluss::arange(start, start + count, i64());
  const Series x = batch.features.x.index_select(0, idx).reshape({count, batch.features.
```

(continues on next page)

(continued from previous page)

```
↪feature_count()});
  const Series y = batch.y.index_select(0, idx).reshape({count});
  return LabeledBatch{
      .features = FeatureBatch{.x = x, .layout = ObservationLayout::Rows},
      .y = y,
  };
}

LabeledBatch make_class_batch() {
  std::vector<double> xv{};
  std::vector<double> yv{};
  xv.reserve(2 * 90);
  yv.reserve(90);
  for (int i = 0; i < 45; ++i) {
    xv.push_back(-2.4 + 0.06 * i);
    xv.push_back(-1.5 + 0.03 * i);
    yv.push_back(0.0);
  }
  for (int i = 0; i < 45; ++i) {
    xv.push_back(1.0 + 0.06 * i);
    xv.push_back(1.4 + 0.03 * i);
```

Output

```
online seen_samples=1872
online metrics [accuracy, logloss]: 1 0.27341
```

Example 3

Source: PlussAI/examples/NodeJ/lymph/lymph_online_demo.cpp

```
using pluss::ai::lymph::OnlineLinearOptions;

Config f64() {
  return Config{}.with_dtype(ScalarType::Float64).with_platform(pluss::pCPU);
}

Config i64() {
  return Config{}.with_dtype(ScalarType::Int64).with_platform(pluss::pCPU);
}

LabeledBatch slice_rows(const LabeledBatch& batch, std::int64_t start, std::int64_t␣
↪count) {
  const Series idx = pluss::arange(start, start + count, i64());
  const Series x = batch.features.x.index_select(0, idx).reshape({count, batch.features.
↪feature_count()});
  const Series y = batch.y.index_select(0, idx).reshape({count});
  return LabeledBatch{
      .features = FeatureBatch{.x = x, .layout = ObservationLayout::Rows},
      .y = y,
  };
}
```

(continues on next page)

(continued from previous page)

```
LabeledBatch make_batch() {
  std::vector<double> xv{};
  std::vector<double> yv{};
  for (int i = 0; i < 40; ++i) {
    xv.push_back(-2.0 + 0.05 * i);
    xv.push_back(-1.6 + 0.05 * i);
    yv.push_back(0.0);
  }
  for (int i = 0; i < 40; ++i) {
    xv.push_back(1.2 + 0.05 * i);
    xv.push_back(1.0 + 0.05 * i);
    yv.push_back(1.0);
  }
```

Output

```
lymph online seen_samples=1660
lymph online metrics [accuracy]: 0.9875
```

Source

- `PlussAI/include/pluss/ai/nodej/dataset.h:13`

`FeatureBatch::sample_count`

Signature

```
[[nodiscard]] std::int64_t FeatureBatch::sample_count(self) const noexcept;
```

Notes

Reports a cardinality/count statistic from the current object state.

Parameters

None.

Returns

- Returns `std::int64_t`.

Options

Option semantics used by this method:

- `FeatureBatch` (`struct`):
  - `Series x = Series::undefined()` Option field used by this method.
  - `ObservationLayout layout = ObservationLayout::Rows` Declares whether observations are represented by rows or columns.

Examples

Example 1

Source: PlussAI/examples/NodeJ/core/nodej_dataframe_bridge_demo.cpp

```
  DataFrameBatchOptions options{};
  options.feature_names = {"x0", "x1", "category", "ts", "target", "weight"};
  options.label_feature = "target";
  options.sample_weight_feature = "weight";
  options.note_policy = DataNotePolicy::Hash;
  options.thread_policy = DataThreadPolicy::CastTemporalToSeries;

  auto batch_or = labeled_batch_from_dataframe(frame, options);
  if (!require_ok(batch_or, "labeled_batch_from_dataframe")) return 1;

  const LabeledBatch& batch = batch_or.value();
  std::cout << "adapted batch: samples=" << batch.features.sample_count()
            << " features=" << batch.features.feature_count()
            << " weighted=" << (batch.sample_weights.defined() ? "yes" : "no") << "\n";

  ModelSpec spec{
      .task = TaskKind::Regression,
      .family = ModelFamily::Linear,
      .options = LinearFamilyOptions{
          .objective = LinearObjective::LeastSquares,
          .regularization = RegularizationKind::L2,
          .regularization_strength = 1e-8,
          .fit_intercept = true,
      },
  };

  auto reg_or = make_regressor(spec);
  if (!require_ok(reg_or, "make_regressor")) return 1;
  auto reg = reg_or.move_value();
  Status st = reg->learn(batch);
  if (!st.ok()) {
    std::cerr << "learn failed: " << st.message << "\n";
```

Output

```
NodeJ DataFrame bridge demo
adapted batch: samples=6 features=4 weighted=yes
linear metrics [mse, r2]: 9.43684e-08 1
NodeJ DataFrame bridge demo completed.
```

Example 2

Source: PlussAI/examples/NodeJ/core/nodej_prep_split_demo.cpp

```
                                          4.0,  3.5, 1.0,
                                          5.0,  4.0, 0.0,
                                          6.0,  4.5, 1.0},
                                     {6, 3},
```

(continues on next page)

(continued from previous page)

```
                                        f64());
  const FeatureBatch dense{.x = x_raw, .layout = ObservationLayout::Rows};
  const Series labels = pluss::make_series(std::vector<double>{0.0, 0.0, 0.0, 1.0, 1.0,␣
↪1.0}, {6}, f64());

  SplitOptions split{};
  split.holdout_fraction = 0.33;
  split.random.seed = 7;

  auto holdout_or = pluss::ai::nodej::holdout_split(dense.sample_count(), split);
  if (!require_ok(holdout_or, "holdout_split")) return 1;
  std::cout << "holdout train idx count=" << holdout_or.value().train_indices.numel()
            << " test idx count=" << holdout_or.value().test_indices.numel() << "\n";

  auto strat_or = pluss::ai::nodej::strat_holdout_split(labels, split);
  if (!require_ok(strat_or, "strat_holdout_split")) return 1;
  std::cout << "strat holdout train idx count=" << strat_or.value().train_indices.numel()
            << " test idx count=" << strat_or.value().test_indices.numel() << "\n";

  split.folds = 3;
  auto fold_or = pluss::ai::nodej::fold_plan(dense.sample_count(), split);
  if (!require_ok(fold_or, "fold_plan")) return 1;
  std::cout << "kfold folds=" << fold_or.value().folds.size() << "\n";

  SimpleImputer imputer(SimpleImputerOptions{
      .strategy = SimpleImputerStrategy::Median,
      .constant_value = 0.0,
      .empty_feature_fallback = 0.0,
  });
  auto imp_or = imputer.learn_map(dense);
```

Output

```
NodeJ split + prep demo
holdout train idx count=4 test idx count=2
strat holdout train idx count=4 test idx count=2
kfold folds=3
imputed shape=[6, 3]
standardized shape=[6, 3]
minmax shape=[6, 3]
robust shape=[6, 3]
neighbor-imputed shape=[6, 3]
iterative-imputed shape=[6, 3]
vector-normalized shape=[6, 3]
maxabs shape=[6, 3]
...
```

Source

- PlussAI/include/pluss/ai/nodej/dataset.h:17

FeatureBatch::validate_features

Signature

```
inline Status FeatureBatch::validate_features(self, const FeatureBatch& batch);
```

Notes

Validates shape/layout/typing invariants for this data container before downstream use.

Parameters

- batch : const FeatureBatch& Input feature data used by this call.

Returns

- Returns a status object indicating success/failure and an error message on failure.

Options

Option semantics used by this method:

- FeatureBatch (struct):
  - Series x = Series::undefined() Option field used by this method.
  - ObservationLayout layout = ObservationLayout::Rows Declares whether observations are represented by rows or columns.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
FeatureBatch featureBatch{};
auto out = featureBatch.validate_features(/* args */);
```

Source

- PlussAI/include/pluss/ai/nodej/dataset.h:32

FeatureBatch::validate_labeled

Signature

```
inline Status FeatureBatch::validate_labeled(self, const LabeledBatch& batch);
```

Notes

Validates shape/layout/typing invariants for this data container before downstream use.

Parameters

- `batch` : `const LabeledBatch&` Input feature data used by this call.

Returns

- Returns a status object indicating success/failure and an error message on failure.

Options

Option semantics used by this method:

- `FeatureBatch` (`struct`):
  - `Series x = Series::undefined()` Option field used by this method.
  - `ObservationLayout layout = ObservationLayout::Rows` Declares whether observations are represented by rows or columns.
- `LabeledBatch` (`struct`):
  - `FeatureBatch features{}` Option field used by this method.
  - `Series y = Series::undefined()` Option field used by this method.
  - `Series sample_weights = Series::undefined()` Optional column used as per-row training weight.

Examples

Example 1

Source: `PlussAI/examples/NodeJ/core/nodej_end_to_end_hour_workflow_showcase.cpp`

```
  Status train_batch_check = validate_labeled(train_batch);
  if (!train_batch_check.ok()) {
    std::cerr << "validate_labeled(train_batch) failed: " << train_batch_check.message <
↪< "\n";
    return 1;
  }
  Status test_batch_check = validate_labeled(test_batch);
  if (!test_batch_check.ok()) {
    std::cerr << "validate_labeled(test_batch) failed: " << test_batch_check.message <<
↪"\n";
    return 1;
  }
```

Source

- `PlussAI/include/pluss/ai/nodej/dataset.h:40`

`Flow::append_transform`

Signature

```
Status Flow::append_transform(self, std::unique_ptr<TransformerBase> transformer);
```

Notes

Instance method on `self`.

Parameters

- `transformer` : `std::unique_ptr<TransformerBase>` Method argument; see signature type for accepted form.

Returns

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
Flow flow{};
auto out = flow.append_transform(/* args */);
```

Source

- `PlussAI/include/pluss/ai/nodej/compose.h:70`

`Flow::clear_model`

Signature

```
void Flow::clear_model(self) noexcept;
```

Notes

Instance method on `self`.

Parameters

None.

Returns

- No direct return value; operation applies side effects to object state.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
Flow flow{};
auto out = flow.clear_model(/* args */);
```

Source

- PlussAI/include/pluss/ai/nodej/compose.h:91

Flow::clear_transforms

Signature

```
void Flow::clear_transforms(self) noexcept;
```

Notes

Instance method on `self`.

Parameters

None.

Returns

- No direct return value; operation applies side effects to object state.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
Flow flow{};
auto out = flow.clear_transforms(/* args */);
```

Source

- PlussAI/include/pluss/ai/nodej/compose.h:71

Flow::dump_model_state

Signature

```
[[nodiscard]] StatusOr<std::vector<Series>> Flow::dump_model_state(self) const;
```

Notes

Instance method on `self`.

Parameters

None.

Returns

- Returns `StatusOr<std::vector<Series>>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
Flow flow{};
auto out = flow.dump_model_state(/* args */);
```

Source

- `PlussAI/include/pluss/ai/nodej/compose.h:117`

`Flow::dump_transform_states`

Signature

```
[[nodiscard]] StatusOr<std::vector<std::vector<Series>>> Flow::dump_transform_
↪states(self) const;
```

Notes

Instance method on `self`.

Parameters

None.

Returns

- Returns `StatusOr<std::vector<std::vector<Series>>>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
Flow flow{};
auto out = flow.dump_transform_states(/* args */);
```

Source

- PlussAI/include/pluss/ai/nodej/compose.h:116

Flow::has_model

Signature

```
[[nodiscard]] bool Flow::has_model(self) const noexcept;
```

Notes

Boolean introspection helper that reports whether a capability/state is available.

Parameters

None.

Returns

- Returns bool.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
Flow flow{};
auto out = flow.has_model(/* args */);
```

Source

- PlussAI/include/pluss/ai/nodej/compose.h:62

Flow::has_model_spec

Signature

```
[[nodiscard]] bool Flow::has_model_spec(self) const noexcept;
```

Notes

Boolean introspection helper that reports whether a capability/state is available.

Parameters

None.

Returns

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
Flow flow{};
auto out = flow.has_model_spec(/* args */);
```

Source

- `PlussAI/include/pluss/ai/nodej/compose.h:65`

`Flow::has_opaque_transforms`

Signature

```
[[nodiscard]] bool Flow::has_opaque_transforms(self) const noexcept;
```

Notes

Boolean introspection helper that reports whether a capability/state is available.

Parameters

None.

Returns

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
Flow flow{};
auto out = flow.has_opaque_transforms(/* args */);
```

Source

- PlussAI/include/pluss/ai/nodej/compose.h:66

Flow::load_model_state

Signature

```
Status Flow::load_model_state(self, const std::vector<Series>& state);
```

Notes

Loads a previously persisted artifact and restores its inference-ready state.

Parameters

- state : const std::vector<Series>& Randomization control for reproducible behavior.

Returns

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
Flow flow{};
auto out = flow.load_model_state(/* args */);
```

Source

- PlussAI/include/pluss/ai/nodej/compose.h:119

Flow::load_transform_states

Signature

```
Status Flow::load_transform_states(self, const std::vector<std::vector<Series>>& states);
```

Notes

Loads a previously persisted artifact and restores its inference-ready state.

Parameters

- states : const std::vector<std::vector<Series>>& Randomization control for reproducible behavior.

Returns

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
Flow flow{};
auto out = flow.load_transform_states(/* args */);
```

Source

- `PlussAI/include/pluss/ai/nodej/compose.h:118`

`Flow::mark_trained`

Signature

```
void Flow::mark_trained(self, bool trained) noexcept;
```

Notes

Fits model/transform state from the provided training data. Call this before inference-oriented methods.

Parameters

- `trained` : `bool` Method argument; see signature type for accepted form.

Returns

- No direct return value; operation applies side effects to object state.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
Flow flow{};
auto out = flow.mark_trained(/* args */);
```

Source

- `PlussAI/include/pluss/ai/nodej/compose.h:120`

`Flow::model_spec`

Signature

```
[[nodiscard]] const std::optional<ModelSpec>& Flow::model_spec(self) const noexcept;
```

Notes

Instance method on `self`.

Parameters

None.

Returns

- Returns `const std::optional<ModelSpec>&`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
Flow flow{};
auto out = flow.model_spec(/* args */);
```

Source

- `PlussAI/include/pluss/ai/nodej/compose.h:67`

`Flow::set_classifier`

Signature

```
Status Flow::set_classifier(self, std::unique_ptr<ClassifierBase> model);
```

Notes

Sets or updates internal configuration/state used by later train/infer/evaluate calls.

Parameters

- `model` : `std::unique_ptr<ClassifierBase>` Method argument; see signature type for accepted form.

Returns

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
Flow flow{};
auto out = flow.set_classifier(/* args */);
```

Source

- PlussAI/include/pluss/ai/nodej/compose.h:89

Flow::set_model

Signature

```
Status Flow::set_model(self, const ModelSpec& spec);
```

Notes

Sets or updates internal configuration/state used by later train/infer/evaluate calls.

Parameters

- spec : const ModelSpec& Method argument; see signature type for accepted form.

Returns

- Returns a status object indicating success/failure and an error message on failure.

Options

Option semantics used by this method:

- ModelSpec (struct):
  - TaskKind task = TaskKind::Regression Selects task type (for example classification vs regression).
  - ModelFamily family = ModelFamily::Linear Selects the model family implementation.
  - FamilyOptions options = LinearFamilyOptions{} Family-specific options payload used by the selected model/transform family.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
Flow flow{};
auto out = flow.set_model(/* args */);
```

Source

- PlussAI/include/pluss/ai/nodej/compose.h:88

Flow::set_regressor

Signature

```
Status Flow::set_regressor(self, std::unique_ptr<RegressorBase> model);
```

Notes

Sets or updates internal configuration/state used by later train/infer/evaluate calls.

Parameters

- model : std::unique_ptr<RegressorBase> Method argument; see signature type for accepted form.

Returns

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
Flow flow{};
auto out = flow.set_regressor(/* args */);
```

Source

- PlussAI/include/pluss/ai/nodej/compose.h:90

Flow::stage_count

Signature

```
[[nodiscard]] std::int64_t Flow::stage_count(self) const noexcept;
```

Notes

Reports a cardinality/count statistic from the current object state.

Parameters

None.

Returns

- Returns `std::int64_t`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
Flow flow{};
auto out = flow.stage_count(/* args */);
```

Source

- `PlussAI/include/pluss/ai/nodej/compose.h:64`

`Flow::task_kind`

Signature

```
[[nodiscard]] TaskKind Flow::task_kind(self) const noexcept;
```

Notes

Instance method on `self`.

Parameters

None.

Returns

- Returns `TaskKind`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
Flow flow{};
auto out = flow.task_kind(/* args */);
```

Source

- `PlussAI/include/pluss/ai/nodej/compose.h:63`

Flow::trained

Signature

```
[[nodiscard]] bool Flow::trained(self) const noexcept;
```

Notes

Boolean state check for object readiness.

Parameters

None.

Returns

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
Flow flow{};
auto out = flow.trained(/* args */);
```

Source

- `PlussAI/include/pluss/ai/nodej/compose.h:61`

Flow::transform_specs

Signature

```
[[nodiscard]] const std::vector<TransformSpec>& Flow::transform_specs(self) const␣
↪noexcept;
```

Notes

Instance method on `self`.

Parameters

None.

Returns

- Returns `const std::vector<TransformSpec>&`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```cpp
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
Flow flow{};
auto out = flow.transform_specs(/* args */);
```

Source

- PlussAI/include/pluss/ai/nodej/compose.h:68

FrequencyEncoder::dump_state

Signature

```cpp
[[nodiscard]] StatusOr<std::vector<Series>> FrequencyEncoder::dump_state(self) const␣
↪override;
```

Notes

Instance method on `self`.

Parameters

None.

Returns

- Returns `StatusOr<std::vector<Series>>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1

Source: `PlussAI/examples/NodeJ/core/nodej_end_to_end_hour_workflow_showcase.cpp`

```cpp
    return 1;
  }
  Series train_maxabs_scaled = train_maxabs_scaled_or.move_value();

  StatusOr<Series> test_maxabs_scaled_or = maxabs_scaler.map(test_simple_batch);
  if (!test_maxabs_scaled_or.ok()) {
    std::cerr << "MaxAbsScaler.map(test) failed: " << test_maxabs_scaled_or.status().
↪message << "\n";
```

(continues on next page)

(continued from previous page)

```
    return 1;
  }
  Series test_maxabs_scaled = test_maxabs_scaled_or.move_value();

  // This section saves compact prep-state diagnostics.
  StatusOr<std::vector<Series>> simple_state_or = simple_imputer.dump_state();
  if (!simple_state_or.ok()) {
    std::cerr << "SimpleImputer.dump_state failed: " << simple_state_or.status().message
↪<< "\n";
    return 1;
  }
  std::vector<Series> simple_state = simple_state_or.move_value();

  StatusOr<std::vector<Series>> knn_state_or = knn_imputer.dump_state();
  if (!knn_state_or.ok()) {
    std::cerr << "NeighborImputer.dump_state failed: " << knn_state_or.status().message <
↪< "\n";
    return 1;
  }
  std::vector<Series> knn_state = knn_state_or.move_value();

  StatusOr<std::vector<Series>> iter_state_or = iterative_imputer.dump_state();
  if (!iter_state_or.ok()) {
    std::cerr << "IterativeImputer.dump_state failed: " << iter_state_or.status().
↪message << "\n";
    return 1;
  }
  std::vector<Series> iter_state = iter_state_or.move_value();
```

Source

- PlussAI/include/pluss/ai/nodej/prep.h:427

FrequencyEncoder::load_state

Signature

```
Status FrequencyEncoder::load_state(self, const std::vector<Series>& state) override;
```

Notes

Loads a previously persisted artifact and restores its inference-ready state.

Parameters

- `state` : `const std::vector<Series>&` Randomization control for reproducible behavior.

Returns

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
FrequencyEncoder frequencyEncoder{};
auto out = frequencyEncoder.load_state(/* args */);
```

Source

- `PlussAI/include/pluss/ai/nodej/prep.h:428`

`FrequencyEncoder::trained`

Signature

```
[[nodiscard]] bool FrequencyEncoder::trained(self) const noexcept override;
```

Notes

Boolean state check for object readiness.

Parameters

None.

Returns

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
FrequencyEncoder frequencyEncoder{};
auto out = frequencyEncoder.trained(/* args */);
```

Source

- `PlussAI/include/pluss/ai/nodej/prep.h:421`

IterativeImputer::dump_state

Signature

```
[[nodiscard]] StatusOr<std::vector<Series>> IterativeImputer::dump_state(self) const␣
↪override;
```

Notes

Instance method on `self`.

Parameters

None.

Returns

- Returns `StatusOr<std::vector<Series>>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1

Source: `PlussAI/examples/NodeJ/core/nodej_end_to_end_hour_workflow_showcase.cpp`

```
    return 1;
  }
  Series train_maxabs_scaled = train_maxabs_scaled_or.move_value();

  StatusOr<Series> test_maxabs_scaled_or = maxabs_scaler.map(test_simple_batch);
  if (!test_maxabs_scaled_or.ok()) {
    std::cerr << "MaxAbsScaler.map(test) failed: " << test_maxabs_scaled_or.status().
↪message << "\n";
    return 1;
  }
  Series test_maxabs_scaled = test_maxabs_scaled_or.move_value();

  // This section saves compact prep-state diagnostics.
  StatusOr<std::vector<Series>> simple_state_or = simple_imputer.dump_state();
  if (!simple_state_or.ok()) {
    std::cerr << "SimpleImputer.dump_state failed: " << simple_state_or.status().message
↪<< "\n";
    return 1;
  }
  std::vector<Series> simple_state = simple_state_or.move_value();

  StatusOr<std::vector<Series>> knn_state_or = knn_imputer.dump_state();
  if (!knn_state_or.ok()) {
```

(continues on next page)

(continued from previous page)

```
    std::cerr << "NeighborImputer.dump_state failed: " << knn_state_or.status().message <
↪< "\n";
    return 1;
  }
  std::vector<Series> knn_state = knn_state_or.move_value();

  StatusOr<std::vector<Series>> iter_state_or = iterative_imputer.dump_state();
  if (!iter_state_or.ok()) {
    std::cerr << "IterativeImputer.dump_state failed: " << iter_state_or.status().
↪message << "\n";
    return 1;
  }
  std::vector<Series> iter_state = iter_state_or.move_value();
```

Source

- PlussAI/include/pluss/ai/nodej/prep.h:253

IterativeImputer::load_state

Signature

```
Status IterativeImputer::load_state(self, const std::vector<Series>& state) override;
```

Notes

Loads a previously persisted artifact and restores its inference-ready state.

Parameters

- `state` : `const std::vector<Series>&` Randomization control for reproducible behavior.

Returns

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
IterativeImputer iterativeImputer{};
auto out = iterativeImputer.load_state(/* args */);
```

Source

- PlussAI/include/pluss/ai/nodej/prep.h:254

IterativeImputer::trained

Signature

```
[[nodiscard]] bool IterativeImputer::trained(self) const noexcept override;
```

Notes

Boolean state check for object readiness.

Parameters

None.

Returns

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
IterativeImputer iterativeImputer{};
auto out = iterativeImputer.trained(/* args */);
```

Source

- `PlussAI/include/pluss/ai/nodej/prep.h:247`

labeled_batch_from_dataframe

Signature

```
StatusOr<LabeledBatch> labeled_batch_from_dataframe(const pluss::table::DataFrame& frame,
↪ const DataFrameBatchOptions& options);
```

Notes

Builds a supervised `LabeledBatch` from a DataFrame. Use `options` to pick feature columns, label column, weight column, and categorical handling policy.

Parameters

- `frame` : `const pluss::table::DataFrame&` Input feature data used by this call.
- `options` : `const DataFrameBatchOptions&` Configuration object controlling algorithm behavior and safety defaults.

Returns

- Returns `StatusOr<LabeledBatch>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

Option semantics used by this method:

- `DataFrameBatchOptions` (`struct`):
  - `std::vector<std::string> feature_names{}` Explicit list of feature columns to read from the input frame.
  - `std::optional<std::string> label_feature{std::nullopt}` Column used as the supervised target (`y`).
  - `std::optional<std::string> sample_weight_feature{std::nullopt}` Optional column used as per-row training weight.
  - `ObservationLayout layout{ObservationLayout::Rows}` Declares whether observations are represented by rows or columns.
  - `DataThreadPolicy thread_policy{DataThreadPolicy::CastTemporalToSeries}` Controls how numeric/temporal frame columns are adapted into `Series`.
  - `DataNotePolicy note_policy{DataNotePolicy::Reject}` Controls how note/categorical columns are handled during adaptation.
  - `std::uint64_t note_hash_seed{14695981039346656037ULL}` Random seed for reproducible behavior.

Examples

Example 1

Source: `PlussAI/examples/NodeJ/core/nodej_dataframe_bridge_demo.cpp`

```
int main() {
  std::cout << "NodeJ DataFrame bridge demo\n";

  const DataFrame frame = make_frame();

  DataFrameBatchOptions options{};
  options.feature_names = {"x0", "x1", "category", "ts", "target", "weight"};
  options.label_feature = "target";
  options.sample_weight_feature = "weight";
  options.note_policy = DataNotePolicy::Hash;
  options.thread_policy = DataThreadPolicy::CastTemporalToSeries;

  auto batch_or = labeled_batch_from_dataframe(frame, options);
  if (!require_ok(batch_or, "labeled_batch_from_dataframe")) return 1;

  const LabeledBatch& batch = batch_or.value();
  std::cout << "adapted batch: samples=" << batch.features.sample_count()
            << " features=" << batch.features.feature_count()
            << " weighted=" << (batch.sample_weights.defined() ? "yes" : "no") << "\n";

  ModelSpec spec{
```

(continues on next page)

(continued from previous page)

```
    .task = TaskKind::Regression,
    .family = ModelFamily::Linear,
    .options = LinearFamilyOptions{
        .objective = LinearObjective::LeastSquares,
        .regularization = RegularizationKind::L2,
        .regularization_strength = 1e-8,
        .fit_intercept = true,
    },
};

auto reg_or = make_regressor(spec);
if (!require_ok(reg_or, "make_regressor")) return 1;
```

Output

```
NodeJ DataFrame bridge demo
adapted batch: samples=6 features=4 weighted=yes
linear metrics [mse, r2]: 9.43684e-08 1
NodeJ DataFrame bridge demo completed.
```

Source

- `PlussAI/include/pluss/ai/nodej/data.h:40`

`LabeledBatch::has_labels`

Signature

```
[[nodiscard]] bool LabeledBatch::has_labels(self) const noexcept;
```

Notes

Boolean introspection helper that reports whether a capability/state is available.

Parameters

None.

Returns

- Returns `bool`.

Options

Option semantics used by this method:

- `LabeledBatch` (`struct`):
  - `FeatureBatch features{}` Option field used by this method.
  - `Series y = Series::undefined()` Option field used by this method.
  - `Series sample_weights = Series::undefined()` Optional column used as per-row training weight.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
LabeledBatch labeledBatch{};
auto out = labeledBatch.has_labels(/* args */);
```

Source

- PlussAI/include/pluss/ai/nodej/dataset.h:28

LabeledBatch::has_weights

Signature

```
[[nodiscard]] bool LabeledBatch::has_weights(self) const noexcept;
```

Notes

Boolean introspection helper that reports whether a capability/state is available.

Parameters

None.

Returns

- Returns bool.

Options

Option semantics used by this method:

- LabeledBatch (struct):
  - FeatureBatch features{} Option field used by this method.
  - Series y = Series::undefined() Option field used by this method.
  - Series sample_weights = Series::undefined() Optional column used as per-row training weight.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
LabeledBatch labeledBatch{};
auto out = labeledBatch.has_weights(/* args */);
```

Source

- PlussAI/include/pluss/ai/nodej/dataset.h:29

load

Signature

```
StatusOr<FlowBundle> load(const std::string& path);
```

Notes

Loads a previously persisted artifact and restores its inference-ready state.

Parameters

- `path` : `const std::string&` Path to persisted model/data artifact.

Returns

- Returns `StatusOr<FlowBundle>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1

Source: `PlussAI/examples/NodeJ/core/bike_sharing_end_to_end.cpp`

```
    linear_workflow
        .simple_imputer(lymph::prep::SimpleImputerOptions{.strategy =␣
↪lymph::prep::SimpleImputerStrategy::Median})
        .standard_scaler();
    linear_workflow.learn_(train_x, train_y, train_options);
    const auto linear_metrics = linear_workflow.assess(test_x, test_y, assess_options);
    std::cout << "linear test metrics [rmse, r2, mae]: " << linear_metrics.rmse << " " <
↪< linear_metrics.r2 << " "
              << linear_metrics.mae << "\n";
    std::cout << "linear params: intercept=" << linear_workflow.intercept() << " coeff="
↪<< linear_workflow.coeff() << "\n";

    const Series linear_pred_series = linear_workflow.infer(test_x);
    const fs::path linear_model_path = model_dir / "bike_sharing_linear.bin";
    linear_workflow.save(linear_model_path.string(), train_x, train_y, train_options);
    const auto linear_restored = lymph::Workflow::load(linear_model_path.string());
    const Series linear_pred_restored_series = linear_restored.infer(test_x);
    const Series y_true = test_y.to(pluss::pCPU, ScalarType::Float64).reshape({test_y.
↪numel()});

    const Series linear_pred = linear_pred_series.to(pluss::pCPU, ScalarType::Float64).
↪reshape({linear_pred_series.numel()});
    const double linear_max_abs_diff = lymph::max_abs_diff(linear_pred_series, linear_
↪pred_restored_series);
    std::cout << "linear restored prediction max_abs_diff: " << linear_max_abs_diff << "\
```

(continues on next page)

(continued from previous page)

```
↪n";

    DataFrame linear_frame({
        {"y_true", Thread(y_true)},
        {"y_pred", Thread(linear_pred)},
        {"residual", Thread(y_true.sub(linear_pred))},
    });
    linear_frame.save_csv((out_dir / "bike_sharing_test_predictions_linear.csv").
↪string());
    linear_frame.plot()
        .scatter("y_true", "y_pred")
        .headline("Observed vs predicted bike demand (linear)")
        .save_pdf((out_dir / "bike_pred_vs_true_linear.pdf").string());
    linear_frame.plot()
        .hist("residual", 80)
        .headline("Residual distribution (linear)")
```

Example 2

Source: PlussAI/examples/NodeJ/core/nodej_workflow_tuning_demo.cpp

```
                << " cv_mae=" << trial.cv.mean.mae << "\n";
    }
    std::cout << "best candidate: " << tuned.report.best_name << "\n";

    auto best = std::move(tuned.best_workflow);
    best.learn_(train_x, train_y, train_options);
    const auto test_metrics = best.assess(test_x, test_y, assess_options);
    std::cout << "test metrics [rmse, r2, mae]: " << test_metrics.rmse << " " << test_
↪metrics.r2 << " " << test_metrics.mae << "\n";

    const Series pred = best.infer(test_x);
    const fs::path best_model_path = model_dir / "hour_best_workflow.bin";
    best.save(best_model_path.string(), train_x, train_y, train_options);
    auto restored = nja::Workflow::load(best_model_path.string());
    const Series pred_restored = restored.infer(test_x);
    const double restored_max_abs_diff = nja::max_abs_diff(pred, pred_restored);
    std::cout << "restored prediction max_abs_diff: " << restored_max_abs_diff << "\n";

    const Series y_true = test_y.to(pluss::pCPU, ScalarType::Float64).reshape({test_y.
↪numel()});
    const Series y_pred = pred.to(pluss::pCPU, ScalarType::Float64).reshape({pred.
↪numel()});
    DataFrame pred_frame({
        {"y_true", Thread(y_true)},
        {"y_pred", Thread(y_pred)},
        {"residual", Thread(y_true.sub(y_pred))},
    });
    pred_frame.save_csv((out_dir / "hour_workflow_tuning_predictions.csv").string());
    pred_frame.plot()
        .scatter("y_true", "y_pred")
        .headline("Observed vs predicted bike demand (best workflow)")
```

(continues on next page)

(continued from previous page)

```
        .save_pdf((out_dir / "hour_workflow_tuning_pred_vs_true.pdf").string());

    std::cout << "saved best model: " << best_model_path << "\n";
    std::cout << "saved outputs under: " << out_dir << "\n";
    return 0;
```

Source

- PlussAI/include/pluss/ai/nodej/io.h:37

load_and_restore

Signature

```
StatusOr<Flow> load_and_restore(const std::string& path);
```

Notes

Loads a previously persisted artifact and restores its inference-ready state.

Parameters

- `path` : `const std::string&` Path to persisted model/data artifact.

Returns

- Returns `StatusOr<Flow>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1

Source: PlussAI/examples/NodeJ/core/nodej_flow_bundle_demo.cpp

```
  auto restored_or = nio::load_and_restore(bundle_path);
  if (!restored_or.ok()) {
    std::cerr << "load_and_restore failed: " << restored_or.status().message << "\n";
    return 1;
  }
  auto pred_after_or = restored_or.value().infer(batch.features);
  if (!pred_after_or.ok()) {
    std::cerr << "restored flow infer failed: " << pred_after_or.status().message << "\n
↪";
    return 1;
  }
```

Output

```
saved flow bundle to: <repo_root>/PlussAI/examples/NodeJ/trained_models/nodej_flow_
↪bundle_demo.bin
flow bundle roundtrip exact prediction parity: true
nodej_flow_bundle_demo complete
```

Source

- PlussAI/include/pluss/ai/nodej/io.h:39

make_classifier

Signature

```
StatusOr<std::unique_ptr<ClassifierBase>> make_classifier(const ModelSpec& spec);
```

Notes

Factory entry point that validates spec/options and returns a ready-to-train component.

Parameters

- spec : const ModelSpec& Method argument; see signature type for accepted form.

Returns

- Returns StatusOr<std::unique_ptr<ClassifierBase>>. Check .ok() first; on success read .value() for the computed result.

Options

Option semantics used by this method:

- ModelSpec (struct):
  - TaskKind task = TaskKind::Regression Selects task type (for example classification vs regression).
  - ModelFamily family = ModelFamily::Linear Selects the model family implementation.
  - FamilyOptions options = LinearFamilyOptions{} Family-specific options payload used by the selected model/transform family.

Examples

Example 1

Source: PlussAI/examples/NodeJ/core/nodej_cv_tune_demo.cpp

```
                          reg_batch,
                          split,
                          TrainOptions{},
                          reg_assess,
                          MetricKind::QuantileLoss,
                          TuneObjectivePolicy::Auto);
  if (!require_ok(reg_sweep_or, "sweep(regression quantile)")) return 1;
```

(continues on next page)

(continued from previous page)

```
  std::cout << "regression quantile sweep objectives:";
  for (const auto& trial : reg_sweep_or.value().trials) std::cout << " " << trial.
↪objective;
  std::cout << "\n";
  std::cout << "regression quantile sweep best index: " << reg_sweep_or.value().best_
↪index << "\n";

  auto bounded_or = make_classifier(linear_logistic_spec());
  if (!require_ok(bounded_or, "make_classifier(linear bounded)")) return 1;
  auto bounded = bounded_or.move_value();

  TrainOptions bounded_train = train;
  bounded_train.parameter_lower_bounds = pluss::make_series(std::vector<double>{0.0}, {1}
↪, f64());
  bounded_train.parameter_upper_bounds = pluss::make_series(std::vector<double>{0.0}, {1}
↪, f64());
  if (!bounded->learn(batch, bounded_train).ok()) {
    std::cerr << "bounded linear learn failed\n";
    return 1;
  }

  auto bounded_scores_or = bounded->infer_scores(batch.features);
  if (!require_ok(bounded_scores_or, "infer_scores(bounded)")) return 1;
  std::cout << "max |bounded score|: " << bounded_scores_or.value().abs().max().item
↪<double>() << "\n";

  std::cout << "NodeJ CV + tune demo completed.\n";
  return 0;
}
```

Output

```
NodeJ CV + tune demo
linear mean metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0.25 0.25 0.
↪208333 0.25 nan 0.0625 0.960532
linear std metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0.176777 0.
↪176777 0.216506 0.25 nan 0.108253 0.192821
kernel mean metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 1 1 1 1 1 1␣
↪0.358895
kernel std metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0 0 0 0 0 0␣
↪0.0358183
sweep trial objectives: 0.25 1
sweep best index: 1
probe best index: 1
regression quantile sweep objectives: 0.0356052 0.736518
regression quantile sweep best index: 0
max |bounded score|: 0
NodeJ CV + tune demo completed.
```

Example 2

Source: PlussAI/examples/NodeJ/core/nodej_model_families_demo.cpp

```
  std::cerr << label << " failed: " << st.message << "\n";
  return false;
}

void print_metric(const std::string& label, const Series& m) {
  const auto v = m.to(pluss::pCPU, ScalarType::Float64).to_vec<double>();
  std::cout << label << ":";
  for (double x : v) std::cout << " " << x;
  std::cout << "\n";
}

void run_classifier_family(const std::string& name, const ModelSpec& spec, const␣
↪LabeledBatch& train) {
  auto model_or = make_classifier(spec);
  if (!require_ok(model_or, name + " make_classifier")) return;
  std::unique_ptr<ClassifierBase> model = model_or.move_value();
  if (!require_status(model->learn(train), name + " learn")) return;

  auto pred_or = model->infer(train.features);
  if (!require_ok(pred_or, name + " infer")) return;
  auto metric_or = model->assess(train, AssessOptions{.metrics = {MetricKind::Accuracy,␣
↪MetricKind::LogLoss}});
  if (!require_ok(metric_or, name + " assess")) return;
  print_metric("  " + name + " (acc, logloss)", metric_or.value());
}

void run_regressor_family(const std::string& name, const ModelSpec& spec, const␣
↪LabeledBatch& train) {
  auto model_or = make_regressor(spec);
  if (!require_ok(model_or, name + " make_regressor")) return;
  std::unique_ptr<RegressorBase> model = model_or.move_value();
  if (!require_status(model->learn(train), name + " learn")) return;

  auto pred_or = model->infer(train.features);
  if (!require_ok(pred_or, name + " infer")) return;
  auto metric_or = model->assess(train, AssessOptions{.metrics = {MetricKind::Mse,␣
↪MetricKind::R2}});
```

Output

```
NodeJ model families demo
Classification families:
  linear (acc, logloss): 1 0.000580684
  linear_hinge (acc, logloss): 1 0.0886185
  tree (acc, logloss): 1 9.99978e-13
  ensemble (acc, logloss): 1 1.00003e-12
  neighbor (acc, logloss): 1 9.99978e-13
  kernel (acc, logloss): 1 0.318438
  bayes (acc, logloss): 1 9.99978e-13
Regression families:
```

(continues on next page)

(continued from previous page)

```
  linear (mse, r2): 3.96212e-15 1
  tree (mse, r2): 0 1
...
```

Example 3

Source: PlussAI/examples/NodeJ/core/nodej_multiclass_demo.cpp

```cpp
  auto cls_or = pluss::ai::nodej::make_classifier(spec);
  if (!cls_or.ok()) {
    std::cerr << "make_classifier failed: " << cls_or.status().message << '\n';
    return 1;
  }
  auto cls = cls_or.move_value();
  const auto learn_status = cls->learn(batch);
  if (!learn_status.ok()) {
    std::cerr << "learn failed: " << learn_status.message << '\n';
    return 1;
  }
```

Output

```
scores shape: [12, 3]
metrics [accuracy, f1, logloss]: 1 1 2.26237e-06
flow metrics [accuracy, f1, logloss]: 1 1 1.37143e-05
```

Source

- PlussAI/include/pluss/ai/nodej/builders.h:10

make_flow

Signature

```cpp
[[nodiscard]] StatusOr<Flow> make_flow(const ModelSpec& spec);
```

Notes

Constructs a `Flow` from model spec/options; use it to chain transforms and model training.

Parameters

- `spec` : `const ModelSpec&` Method argument; see signature type for accepted form.

Returns

- Returns `StatusOr<Flow>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

Option semantics used by this method:

- `ModelSpec` (`struct`):
  - `TaskKind task = TaskKind::Regression` Selects task type (for example classification vs regression).
  - `ModelFamily family = ModelFamily::Linear` Selects the model family implementation.
  - `FamilyOptions options = LinearFamilyOptions{}` Family-specific options payload used by the selected model/transform family.

Examples

Example 1

Source: `PlussAI/examples/NodeJ/core/nodej_flow_bundle_demo.cpp`

```
  auto flow_or = make_flow(spec);
  if (!flow_or.ok()) {
    std::cerr << "make_flow failed: " << flow_or.status().message << "\n";
    return 1;
  }
  Flow flow = flow_or.move_value();
  if (!flow.add_standard_scaler().ok()) {
    std::cerr << "add_standard_scaler failed\n";
    return 1;
  }
  TrainOptions train{};
  train.max_iterations = 2500;
  train.tolerance = 1e-8;
  train.random.seed = 77;
  train.random.deterministic = true;
  if (!flow.learn(batch, train).ok()) {
    std::cerr << "flow learn failed\n";
    return 1;
  }
```

Output

```
saved flow bundle to: <repo_root>/PlussAI/examples/NodeJ/trained_models/nodej_flow_
↪bundle_demo.bin
flow bundle roundtrip exact prediction parity: true
nodej_flow_bundle_demo complete
```

Example 2

Source: `PlussAI/examples/NodeJ/core/nodej_flow_demo.cpp`

```
    ModelSpec cls_spec{
        .task = TaskKind::Classification,
        .family = ModelFamily::Linear,
        .options = LinearFamilyOptions{
            .objective = LinearObjective::Logistic,
            .regularization = RegularizationKind::L2,
```

(continues on next page)

(continued from previous page)

```
            .regularization_strength = 1e-6,
            .fit_intercept = true,
        },
    };

    auto flow_or = make_flow(cls_spec);
    if (!require_ok(flow_or, "make_flow(classification)")) return 1;
    Flow cls = flow_or.move_value();
    if (!cls.add_simple_imputer(prep::SimpleImputerOptions{.strategy =␣
↪prep::SimpleImputerStrategy::Mean}).ok()) return 1;
    if (!cls.add_standard_scaler().ok()) return 1;

    TrainOptions opts{};
    opts.max_iterations = 3000;
    opts.tolerance = 1e-8;
    if (!cls.learn(train, opts).ok()) {
      std::cerr << "classification flow learn failed\n";
      return 1;
    }

    auto metric_or = cls.assess(train, AssessOptions{.metrics = {MetricKind::Accuracy,␣
↪MetricKind::LogLoss}});
    if (!require_ok(metric_or, "classification flow assess")) return 1;
    print_series("classification flow metrics [accuracy, logloss]", metric_or.value());
  }

  {
    const Series x = pluss::make_series(std::vector<double>{
```

Output

```
NodeJ Flow demo
classification flow metrics [accuracy, logloss]: 1 0.000228657
regression flow metrics [mse, r2]: 4.46648e-17 1
NodeJ Flow demo completed.
```

Example 3

Source: PlussAI/examples/NodeJ/core/nodej_multiclass_demo.cpp

```
  auto flow_or = pluss::ai::nodej::make_flow(spec);
  if (!flow_or.ok()) {
    std::cerr << "make_flow failed: " << flow_or.status().message << '\n';
    return 1;
  }
  auto flow = flow_or.move_value();
  const auto scaler_status = flow.add_standard_scaler();
  if (!scaler_status.ok()) {
    std::cerr << "add_standard_scaler failed: " << scaler_status.message << '\n';
    return 1;
  }
  const auto flow_learn = flow.learn(batch);
```

(continues on next page)

(continued from previous page)

```
  if (!flow_learn.ok()) {
    std::cerr << "flow learn failed: " << flow_learn.message << '\n';
    return 1;
  }
```

Output

```
scores shape: [12, 3]
metrics [accuracy, f1, logloss]: 1 1 2.26237e-06
flow metrics [accuracy, f1, logloss]: 1 1 1.37143e-05
```

Source

- `PlussAI/include/pluss/ai/nodej/compose.h`:138

`make_regressor`

Signature

```
StatusOr<std::unique_ptr<RegressorBase>> make_regressor(const ModelSpec& spec);
```

Notes

Factory entry point that validates spec/options and returns a ready-to-train component.

Parameters

- `spec` : `const ModelSpec&` Method argument; see signature type for accepted form.

Returns

- Returns `StatusOr<std::unique_ptr<RegressorBase>>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

Option semantics used by this method:

- `ModelSpec` (`struct`):
  - `TaskKind task = TaskKind::Regression` Selects task type (for example classification vs regression).
  - `ModelFamily family = ModelFamily::Linear` Selects the model family implementation.
  - `FamilyOptions options = LinearFamilyOptions{}` Family-specific options payload used by the selected model/transform family.

Examples

Example 1

Source: `PlussAI/examples/NodeJ/core/nodej_dataframe_bridge_demo.cpp`

```
  ModelSpec spec{
      .task = TaskKind::Regression,
      .family = ModelFamily::Linear,
      .options = LinearFamilyOptions{
          .objective = LinearObjective::LeastSquares,
          .regularization = RegularizationKind::L2,
          .regularization_strength = 1e-8,
          .fit_intercept = true,
      },
  };

  auto reg_or = make_regressor(spec);
  if (!require_ok(reg_or, "make_regressor")) return 1;
  auto reg = reg_or.move_value();
  Status st = reg->learn(batch);
  if (!st.ok()) {
    std::cerr << "learn failed: " << st.message << "\n";
    return 1;
  }

  auto metrics_or = reg->assess(batch, AssessOptions{.metrics = {MetricKind::Mse,␣
↪MetricKind::R2}});
  if (!require_ok(metrics_or, "assess")) return 1;
  print_metrics("linear metrics [mse, r2]", metrics_or.value());

  std::cout << "NodeJ DataFrame bridge demo completed.\n";
  return 0;
}
```

Output

```
NodeJ DataFrame bridge demo
adapted batch: samples=6 features=4 weighted=yes
linear metrics [mse, r2]: 9.43684e-08 1
NodeJ DataFrame bridge demo completed.
```

Example 2

Source: PlussAI/examples/NodeJ/core/nodej_model_families_demo.cpp

```
  if (!require_ok(model_or, name + " make_classifier")) return;
  std::unique_ptr<ClassifierBase> model = model_or.move_value();
  if (!require_status(model->learn(train), name + " learn")) return;

  auto pred_or = model->infer(train.features);
  if (!require_ok(pred_or, name + " infer")) return;
  auto metric_or = model->assess(train, AssessOptions{.metrics = {MetricKind::Accuracy,␣
↪MetricKind::LogLoss}});
  if (!require_ok(metric_or, name + " assess")) return;
  print_metric("  " + name + " (acc, logloss)", metric_or.value());
}

void run_regressor_family(const std::string& name, const ModelSpec& spec, const␣
```

(continues on next page)

(continued from previous page)

```
↪LabeledBatch& train) {
  auto model_or = make_regressor(spec);
  if (!require_ok(model_or, name + " make_regressor")) return;
  std::unique_ptr<RegressorBase> model = model_or.move_value();
  if (!require_status(model->learn(train), name + " learn")) return;

  auto pred_or = model->infer(train.features);
  if (!require_ok(pred_or, name + " infer")) return;
  auto metric_or = model->assess(train, AssessOptions{.metrics = {MetricKind::Mse,␣
↪MetricKind::R2}});
  if (!require_ok(metric_or, name + " assess")) return;
  print_metric("  " + name + " (mse, r2)", metric_or.value());
}

} // namespace

int main() {
  std::cout << "NodeJ model families demo\n";

  const Series x_cls = pluss::make_series(std::vector<double>{
                                          -2.0, -1.0,
                                          -1.5, -1.0,
                                          -2.0, -2.0,
```

Output

```
NodeJ model families demo
Classification families:
  linear (acc, logloss): 1 0.000580684
  linear_hinge (acc, logloss): 1 0.0886185
  tree (acc, logloss): 1 9.99978e-13
  ensemble (acc, logloss): 1 1.00003e-12
  neighbor (acc, logloss): 1 9.99978e-13
  kernel (acc, logloss): 1 0.318438
  bayes (acc, logloss): 1 9.99978e-13
Regression families:
  linear (mse, r2): 3.96212e-15 1
  tree (mse, r2): 0 1
...
```

Source

- PlussAI/include/pluss/ai/nodej/builders.h:11

make_transformer

Signature

```
StatusOr<std::unique_ptr<TransformerBase>> make_transformer(const ModelSpec& spec);
```

Notes

Factory entry point that validates spec/options and returns a ready-to-train component.

Parameters

- `spec` : `const ModelSpec&` Method argument; see signature type for accepted form.

Returns

- Returns `StatusOr<std::unique_ptr<TransformerBase>>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

Option semantics used by this method:

- `ModelSpec` (`struct`):
  - `TaskKind task = TaskKind::Regression` Selects task type (for example classification vs regression).
  - `ModelFamily family = ModelFamily::Linear` Selects the model family implementation.
  - `FamilyOptions options = LinearFamilyOptions{}` Family-specific options payload used by the selected model/transform family.

Examples

Example 1

Source: `PlussAI/examples/NodeJ/core/nodej_unsupervised_demo.cpp`

```
  ModelSpec kmeans_spec{
      .task = TaskKind::Transform,
      .family = ModelFamily::Cluster,
      .options = ClusterFamilyOptions{
          .method = ClusterMethod::KMeans,
          .clusters = 2,
          .max_iterations = 200,
          .restarts = 5,
          .tolerance = 1e-5,
      },
  };
  auto kmeans_or = make_transformer(kmeans_spec);
  if (!kmeans_or.ok()) {
    std::cerr << "make_transformer(kmeans) failed: " << kmeans_or.status().message << "\n
↪";
    return 1;
  }
  auto kmeans = kmeans_or.move_value();
  if (!kmeans->learn(features).ok()) {
    std::cerr << "kmeans learn failed\n";
    return 1;
  }
  auto labels_or = kmeans->map(features);
  if (!labels_or.ok()) {
```

(continues on next page)

(continued from previous page)

```
    std::cerr << "kmeans map failed: " << labels_or.status().message << "\n";
    return 1;
  }
  const auto labels = labels_or.value().reshape({60}).to(pluss::pCPU,␣
↪pluss::ScalarType::Float64).to_vec<double>();
  std::cout << "kmeans labels (first 10): ";
  for (int i = 0; i < 10; ++i) std::cout << labels[static_cast<std::size_t>(i)] << " ";
  std::cout << "\n";
```

Output

```
kmeans labels (first 10): 0 0 0 0 0 0 0 0 0 0
pca output shape: [80, 2]
nodej_unsupervised_demo complete
```

Source

- `PlussAI/include/pluss/ai/nodej/builders.h:12`

`make_workflow`

Signature

```
[[nodiscard]] inline StatusOr<Workflow> make_workflow(const ModelSpec& spec);
```

Notes

Constructs a high-level workflow object with model and transform stages.

Parameters

- `spec` : `const ModelSpec&` Method argument; see signature type for accepted form.

Returns

- Returns `StatusOr<Workflow>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

Option semantics used by this method:

- `ModelSpec` (`struct`):
  - `TaskKind task = TaskKind::Regression` Selects task type (for example classification vs regression).
  - `ModelFamily family = ModelFamily::Linear` Selects the model family implementation.
  - `FamilyOptions options = LinearFamilyOptions{}` Family-specific options payload used by the selected model/transform family.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
auto out = make_workflow(/* args */);
```

Source

- PlussAI/include/pluss/ai/nodej/compose.h:140

MaxAbsScaler::dump_state

Signature

```
[[nodiscard]] StatusOr<std::vector<Series>> MaxAbsScaler::dump_state(self) const␣
↪override;
```

Notes

Instance method on `self`.

Parameters

None.

Returns

- Returns `StatusOr<std::vector<Series>>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1

Source: `PlussAI/examples/NodeJ/core/nodej_end_to_end_hour_workflow_showcase.cpp`

```
    return 1;
  }
  Series train_maxabs_scaled = train_maxabs_scaled_or.move_value();

  StatusOr<Series> test_maxabs_scaled_or = maxabs_scaler.map(test_simple_batch);
  if (!test_maxabs_scaled_or.ok()) {
    std::cerr << "MaxAbsScaler.map(test) failed: " << test_maxabs_scaled_or.status().
↪message << "\n";
    return 1;
  }
  Series test_maxabs_scaled = test_maxabs_scaled_or.move_value();

  // This section saves compact prep-state diagnostics.
```

(continues on next page)

(continued from previous page)

```
  StatusOr<std::vector<Series>> simple_state_or = simple_imputer.dump_state();
  if (!simple_state_or.ok()) {
    std::cerr << "SimpleImputer.dump_state failed: " << simple_state_or.status().message
↪<< "\n";
    return 1;
  }
  std::vector<Series> simple_state = simple_state_or.move_value();

  StatusOr<std::vector<Series>> knn_state_or = knn_imputer.dump_state();
  if (!knn_state_or.ok()) {
    std::cerr << "NeighborImputer.dump_state failed: " << knn_state_or.status().message <
↪< "\n";
    return 1;
  }
  std::vector<Series> knn_state = knn_state_or.move_value();

  StatusOr<std::vector<Series>> iter_state_or = iterative_imputer.dump_state();
  if (!iter_state_or.ok()) {
    std::cerr << "IterativeImputer.dump_state failed: " << iter_state_or.status().
↪message << "\n";
    return 1;
  }
  std::vector<Series> iter_state = iter_state_or.move_value();
```

Source

- PlussAI/include/pluss/ai/nodej/prep.h:296

MaxAbsScaler::load_state

Signature

```
Status MaxAbsScaler::load_state(self, const std::vector<Series>& state) override;
```

Notes

Loads a previously persisted artifact and restores its inference-ready state.

Parameters

- `state` : `const std::vector<Series>&` Randomization control for reproducible behavior.

Returns

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
MaxAbsScaler maxAbsScaler{};
auto out = maxAbsScaler.load_state(/* args */);
```

Source

- PlussAI/include/pluss/ai/nodej/prep.h:297

MaxAbsScaler::trained

Signature

```
[[nodiscard]] bool MaxAbsScaler::trained(self) const noexcept override;
```

Notes

Boolean state check for object readiness.

Parameters

None.

Returns

- Returns bool.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
MaxAbsScaler maxAbsScaler{};
auto out = maxAbsScaler.trained(/* args */);
```

Source

- PlussAI/include/pluss/ai/nodej/prep.h:290

MinMaxScaler::dump_state

Signature

```
[[nodiscard]] StatusOr<std::vector<Series>> MinMaxScaler::dump_state(self) const␣
↪override;
```

Notes

Instance method on `self`.

Parameters

None.

Returns

- Returns `StatusOr<std::vector<Series>>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1

Source: `PlussAI/examples/NodeJ/core/nodej_end_to_end_hour_workflow_showcase.cpp`

```
    return 1;
  }
  Series train_maxabs_scaled = train_maxabs_scaled_or.move_value();

  StatusOr<Series> test_maxabs_scaled_or = maxabs_scaler.map(test_simple_batch);
  if (!test_maxabs_scaled_or.ok()) {
    std::cerr << "MaxAbsScaler.map(test) failed: " << test_maxabs_scaled_or.status().
↪message << "\n";
    return 1;
  }
  Series test_maxabs_scaled = test_maxabs_scaled_or.move_value();

  // This section saves compact prep-state diagnostics.
  StatusOr<std::vector<Series>> simple_state_or = simple_imputer.dump_state();
  if (!simple_state_or.ok()) {
    std::cerr << "SimpleImputer.dump_state failed: " << simple_state_or.status().message
↪<< "\n";
    return 1;
  }
  std::vector<Series> simple_state = simple_state_or.move_value();

  StatusOr<std::vector<Series>> knn_state_or = knn_imputer.dump_state();
  if (!knn_state_or.ok()) {
    std::cerr << "NeighborImputer.dump_state failed: " << knn_state_or.status().message <
↪< "\n";
    return 1;
  }
  std::vector<Series> knn_state = knn_state_or.move_value();

  StatusOr<std::vector<Series>> iter_state_or = iterative_imputer.dump_state();
  if (!iter_state_or.ok()) {
```

(continues on next page)

(continued from previous page)

```
    std::cerr << "IterativeImputer.dump_state failed: " << iter_state_or.status().
↪message << "\n";
    return 1;
  }
  std::vector<Series> iter_state = iter_state_or.move_value();
```

Source

- PlussAI/include/pluss/ai/nodej/prep.h:165

MinMaxScaler::load_state

Signature

```
Status MinMaxScaler::load_state(self, const std::vector<Series>& state) override;
```

Notes

Loads a previously persisted artifact and restores its inference-ready state.

Parameters

- `state` : `const std::vector<Series>&` Randomization control for reproducible behavior.

Returns

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
MinMaxScaler minMaxScaler{};
auto out = minMaxScaler.load_state(/* args */);
```

Source

- PlussAI/include/pluss/ai/nodej/prep.h:166

MinMaxScaler::trained

Signature

```
[[nodiscard]] bool MinMaxScaler::trained(self) const noexcept override;
```

Notes

Boolean state check for object readiness.

Parameters

None.

Returns

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
MinMaxScaler minMaxScaler{};
auto out = minMaxScaler.trained(/* args */);
```

Source

- `PlussAI/include/pluss/ai/nodej/prep.h:159`

`NeighborImputer::dump_state`

Signature

```
[[nodiscard]] StatusOr<std::vector<Series>> NeighborImputer::dump_state(self) const␣
↪override;
```

Notes

Instance method on `self`.

Parameters

None.

Returns

- Returns `StatusOr<std::vector<Series>>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1

Source: PlussAI/examples/NodeJ/core/nodej_end_to_end_hour_workflow_showcase.cpp

```
    return 1;
  }
  Series train_maxabs_scaled = train_maxabs_scaled_or.move_value();

  StatusOr<Series> test_maxabs_scaled_or = maxabs_scaler.map(test_simple_batch);
  if (!test_maxabs_scaled_or.ok()) {
    std::cerr << "MaxAbsScaler.map(test) failed: " << test_maxabs_scaled_or.status().
↪message << "\n";
    return 1;
  }
  Series test_maxabs_scaled = test_maxabs_scaled_or.move_value();

  // This section saves compact prep-state diagnostics.
  StatusOr<std::vector<Series>> simple_state_or = simple_imputer.dump_state();
  if (!simple_state_or.ok()) {
    std::cerr << "SimpleImputer.dump_state failed: " << simple_state_or.status().message
↪<< "\n";
    return 1;
  }
  std::vector<Series> simple_state = simple_state_or.move_value();

  StatusOr<std::vector<Series>> knn_state_or = knn_imputer.dump_state();
  if (!knn_state_or.ok()) {
    std::cerr << "NeighborImputer.dump_state failed: " << knn_state_or.status().message <
↪< "\n";
    return 1;
  }
  std::vector<Series> knn_state = knn_state_or.move_value();

  StatusOr<std::vector<Series>> iter_state_or = iterative_imputer.dump_state();
  if (!iter_state_or.ok()) {
    std::cerr << "IterativeImputer.dump_state failed: " << iter_state_or.status().
↪message << "\n";
    return 1;
  }
  std::vector<Series> iter_state = iter_state_or.move_value();
```

Source

- PlussAI/include/pluss/ai/nodej/prep.h:230

NeighborImputer::load_state

Signature

```
Status NeighborImputer::load_state(self, const std::vector<Series>& state) override;
```

Notes

Loads a previously persisted artifact and restores its inference-ready state.

Parameters

- `state` : `const std::vector<Series>&` Randomization control for reproducible behavior.

Returns

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
NeighborImputer neighborImputer{};
auto out = neighborImputer.load_state(/* args */);
```

Source

- `PlussAI/include/pluss/ai/nodej/prep.h`:231

`NeighborImputer::trained`

Signature

```
[[nodiscard]] bool NeighborImputer::trained(self) const noexcept override;
```

Notes

Boolean state check for object readiness.

Parameters

None.

Returns

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
NeighborImputer neighborImputer{};
auto out = neighborImputer.trained(/* args */);
```

Source

- PlussAI/include/pluss/ai/nodej/prep.h:224

OneHotEncoder::dump_state

Signature

```
[[nodiscard]] StatusOr<std::vector<Series>> OneHotEncoder::dump_state(self) const␣
↪override;
```

Notes

Instance method on `self`.

Parameters

None.

Returns

- Returns `StatusOr<std::vector<Series>>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1

Source: PlussAI/examples/NodeJ/core/nodej_end_to_end_hour_workflow_showcase.cpp

```
    return 1;
  }
  Series train_maxabs_scaled = train_maxabs_scaled_or.move_value();

  StatusOr<Series> test_maxabs_scaled_or = maxabs_scaler.map(test_simple_batch);
  if (!test_maxabs_scaled_or.ok()) {
    std::cerr << "MaxAbsScaler.map(test) failed: " << test_maxabs_scaled_or.status().
↪message << "\n";
    return 1;
  }
  Series test_maxabs_scaled = test_maxabs_scaled_or.move_value();

  // This section saves compact prep-state diagnostics.
  StatusOr<std::vector<Series>> simple_state_or = simple_imputer.dump_state();
  if (!simple_state_or.ok()) {
    std::cerr << "SimpleImputer.dump_state failed: " << simple_state_or.status().message
↪<< "\n";
    return 1;
  }
```

(continues on next page)

(continued from previous page)

```
  std::vector<Series> simple_state = simple_state_or.move_value();

  StatusOr<std::vector<Series>> knn_state_or = knn_imputer.dump_state();
  if (!knn_state_or.ok()) {
    std::cerr << "NeighborImputer.dump_state failed: " << knn_state_or.status().message <
↪< "\n";
    return 1;
  }
  std::vector<Series> knn_state = knn_state_or.move_value();

  StatusOr<std::vector<Series>> iter_state_or = iterative_imputer.dump_state();
  if (!iter_state_or.ok()) {
    std::cerr << "IterativeImputer.dump_state failed: " << iter_state_or.status().
↪message << "\n";
    return 1;
  }
  std::vector<Series> iter_state = iter_state_or.move_value();
```

Source

- PlussAI/include/pluss/ai/nodej/prep.h:403

OneHotEncoder::load_state

Signature

```
Status OneHotEncoder::load_state(self, const std::vector<Series>& state) override;
```

Notes

Loads a previously persisted artifact and restores its inference-ready state.

Parameters

- `state` : `const std::vector<Series>&` Randomization control for reproducible behavior.

Returns

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
OneHotEncoder oneHotEncoder{};
auto out = oneHotEncoder.load_state(/* args */);
```

Source

- PlussAI/include/pluss/ai/nodej/prep.h:404

OneHotEncoder::trained

Signature

```
[[nodiscard]] bool OneHotEncoder::trained(self) const noexcept override;
```

Notes

Boolean state check for object readiness.

Parameters

None.

Returns

- Returns bool.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
OneHotEncoder oneHotEncoder{};
auto out = oneHotEncoder.trained(/* args */);
```

Source

- PlussAI/include/pluss/ai/nodej/prep.h:397

OnlineLinearClassifier::seen_samples

Signature

```
[[nodiscard]] std::int64_t OnlineLinearClassifier::seen_samples(self) const noexcept;
```

Notes

Reports a cardinality/count statistic from the current object state.

Parameters

None.

Returns

- Returns `std::int64_t`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1

Source: `PlussAI/examples/NodeJ/core/nodej_online_demo.cpp`

```
    std::cerr << "advance failed\n";
    return 1;
  }

  pluss::ai::nodej::AssessOptions assess{};
  assess.metrics = {MetricKind::Accuracy, MetricKind::LogLoss};
  auto metric_or = model.assess(full, assess);
  if (!metric_or.ok()) {
    std::cerr << "assess failed: " << metric_or.status().message << '\n';
    return 1;
  }

  std::cout << "online seen_samples=" << model.seen_samples() << '\n';
  std::cout << "online metrics [accuracy, logloss]: ";
  print_vec(metric_or.value());
  return 0;
}
```

Output

```
online seen_samples=1872
online metrics [accuracy, logloss]: 1 0.27341
```

Example 2

Source: `PlussAI/examples/NodeJ/lymph/lymph_online_demo.cpp`

```
    };
    online_options.learning_rate = 0.08;

    TrainOptions warm_opts{};
    warm_opts.max_iterations = 80;

    auto model = pluss::ai::lymph::online_factory::linear_classifier(online_options);
    model.learn_(warm, warm_opts).advance_(stream);

    const auto metrics = model.assess(full);
    const Series metric_values = metrics.accuracy.defined() ? metrics.accuracy : metrics.
↪values;
```

(continues on next page)

(continued from previous page)

```
    std::cout << "lymph online seen_samples=" << model.seen_samples() << '\n';
    std::cout << "lymph online metrics [accuracy]: ";
    print_vec(metric_values);
    return 0;
  } catch (const std::exception& ex) {
    std::cerr << "lymph_online_demo failed: " << ex.what() << '\n';
    return 1;
  }
}
```

Output

```
lymph online seen_samples=1660
lymph online metrics [accuracy]: 0.9875
```

Source

- PlussAI/include/pluss/ai/nodej/online.h:24

OnlineLinearClassifier::trained

Signature

```
[[nodiscard]] bool OnlineLinearClassifier::trained(self) const noexcept;
```

Notes

Boolean state check for object readiness.

Parameters

None.

Returns

- Returns bool.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
OnlineLinearClassifier onlineLinearClassifier{};
auto out = onlineLinearClassifier.trained(/* args */);
```

Source

- PlussAI/include/pluss/ai/nodej/online.h:23

OnlineLinearRegressor::seen_samples

Signature

```
[[nodiscard]] std::int64_t OnlineLinearRegressor::seen_samples(self) const noexcept;
```

Notes

Reports a cardinality/count statistic from the current object state.

Parameters

None.

Returns

- Returns `std::int64_t`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1

Source: PlussAI/examples/NodeJ/core/nodej_online_demo.cpp

```
    std::cerr << "advance failed\n";
    return 1;
  }

  pluss::ai::nodej::AssessOptions assess{};
  assess.metrics = {MetricKind::Accuracy, MetricKind::LogLoss};
  auto metric_or = model.assess(full, assess);
  if (!metric_or.ok()) {
    std::cerr << "assess failed: " << metric_or.status().message << '\n';
    return 1;
  }

  std::cout << "online seen_samples=" << model.seen_samples() << '\n';
  std::cout << "online metrics [accuracy, logloss]: ";
  print_vec(metric_or.value());
  return 0;
}
```

Output

```
online seen_samples=1872
online metrics [accuracy, logloss]: 1 0.27341
```

Example 2

Source: PlussAI/examples/NodeJ/lymph/lymph_online_demo.cpp

```
    };
    online_options.learning_rate = 0.08;

    TrainOptions warm_opts{};
    warm_opts.max_iterations = 80;

    auto model = pluss::ai::lymph::online_factory::linear_classifier(online_options);
    model.learn_(warm, warm_opts).advance_(stream);

    const auto metrics = model.assess(full);
    const Series metric_values = metrics.accuracy.defined() ? metrics.accuracy : metrics.
↪values;

    std::cout << "lymph online seen_samples=" << model.seen_samples() << '\n';
    std::cout << "lymph online metrics [accuracy]: ";
    print_vec(metric_values);
    return 0;
  } catch (const std::exception& ex) {
    std::cerr << "lymph_online_demo failed: " << ex.what() << '\n';
    return 1;
  }
}
```

Output

```
lymph online seen_samples=1660
lymph online metrics [accuracy]: 0.9875
```

Source

- PlussAI/include/pluss/ai/nodej/online.h:50

OnlineLinearRegressor::trained

Signature

```
[[nodiscard]] bool OnlineLinearRegressor::trained(self) const noexcept;
```

Notes

Boolean state check for object readiness.

Parameters

None.

Returns

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
OnlineLinearRegressor onlineLinearRegressor{};
auto out = onlineLinearRegressor.trained(/* args */);
```

Source

- `PlussAI/include/pluss/ai/nodej/online.h:49`

`PolynomialLift::dump_state`

Signature

```
[[nodiscard]] StatusOr<std::vector<Series>> PolynomialLift::dump_state(self) const␣
↪override;
```

Notes

Instance method on `self`.

Parameters

None.

Returns

- Returns `StatusOr<std::vector<Series>>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1

Source: `PlussAI/examples/NodeJ/core/nodej_end_to_end_hour_workflow_showcase.cpp`

```
  return 1;
}
Series train_maxabs_scaled = train_maxabs_scaled_or.move_value();
```

(continues on next page)

(continued from previous page)

```
  StatusOr<Series> test_maxabs_scaled_or = maxabs_scaler.map(test_simple_batch);
  if (!test_maxabs_scaled_or.ok()) {
    std::cerr << "MaxAbsScaler.map(test) failed: " << test_maxabs_scaled_or.status().
↪message << "\n";
    return 1;
  }
  Series test_maxabs_scaled = test_maxabs_scaled_or.move_value();

  // This section saves compact prep-state diagnostics.
  StatusOr<std::vector<Series>> simple_state_or = simple_imputer.dump_state();
  if (!simple_state_or.ok()) {
    std::cerr << "SimpleImputer.dump_state failed: " << simple_state_or.status().message
↪<< "\n";
    return 1;
  }
  std::vector<Series> simple_state = simple_state_or.move_value();

  StatusOr<std::vector<Series>> knn_state_or = knn_imputer.dump_state();
  if (!knn_state_or.ok()) {
    std::cerr << "NeighborImputer.dump_state failed: " << knn_state_or.status().message <
↪< "\n";
    return 1;
  }
  std::vector<Series> knn_state = knn_state_or.move_value();

  StatusOr<std::vector<Series>> iter_state_or = iterative_imputer.dump_state();
  if (!iter_state_or.ok()) {
    std::cerr << "IterativeImputer.dump_state failed: " << iter_state_or.status().
↪message << "\n";
    return 1;
  }
  std::vector<Series> iter_state = iter_state_or.move_value();
```

Source

- PlussAI/include/pluss/ai/nodej/prep.h:381

PolynomialLift::load_state

Signature

```
Status PolynomialLift::load_state(self, const std::vector<Series>& state) override;
```

Notes

Loads a previously persisted artifact and restores its inference-ready state.

Parameters

- `state` : `const std::vector<Series>&` Randomization control for reproducible behavior.

Returns

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
PolynomialLift polynomialLift{};
auto out = polynomialLift.load_state(/* args */);
```

Source

- `PlussAI/include/pluss/ai/nodej/prep.h`:382

`PolynomialLift::trained`

Signature

```
[[nodiscard]] bool PolynomialLift::trained(self) const noexcept override;
```

Notes

Boolean state check for object readiness.

Parameters

None.

Returns

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
PolynomialLift polynomialLift{};
auto out = polynomialLift.trained(/* args */);
```

Source

- PlussAI/include/pluss/ai/nodej/prep.h:375

PowerMap::dump_state

Signature

```
[[nodiscard]] StatusOr<std::vector<Series>> PowerMap::dump_state(self) const override;
```

Notes

Instance method on `self`.

Parameters

None.

Returns

- Returns `StatusOr<std::vector<Series>>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1

Source: `PlussAI/examples/NodeJ/core/nodej_end_to_end_hour_workflow_showcase.cpp`

```
    return 1;
  }
  Series train_maxabs_scaled = train_maxabs_scaled_or.move_value();

  StatusOr<Series> test_maxabs_scaled_or = maxabs_scaler.map(test_simple_batch);
  if (!test_maxabs_scaled_or.ok()) {
    std::cerr << "MaxAbsScaler.map(test) failed: " << test_maxabs_scaled_or.status().
↪message << "\n";
    return 1;
  }
  Series test_maxabs_scaled = test_maxabs_scaled_or.move_value();

  // This section saves compact prep-state diagnostics.
  StatusOr<std::vector<Series>> simple_state_or = simple_imputer.dump_state();
  if (!simple_state_or.ok()) {
    std::cerr << "SimpleImputer.dump_state failed: " << simple_state_or.status().message
↪<< "\n";
    return 1;
  }
  std::vector<Series> simple_state = simple_state_or.move_value();
```

(continues on next page)

(continued from previous page)

```
  StatusOr<std::vector<Series>> knn_state_or = knn_imputer.dump_state();
  if (!knn_state_or.ok()) {
    std::cerr << "NeighborImputer.dump_state failed: " << knn_state_or.status().message <
↪< "\n";
    return 1;
  }
  std::vector<Series> knn_state = knn_state_or.move_value();

  StatusOr<std::vector<Series>> iter_state_or = iterative_imputer.dump_state();
  if (!iter_state_or.ok()) {
    std::cerr << "IterativeImputer.dump_state failed: " << iter_state_or.status().
↪message << "\n";
    return 1;
  }
  std::vector<Series> iter_state = iter_state_or.move_value();
```

Source

- `PlussAI/include/pluss/ai/nodej/prep.h:338`

`PowerMap::load_state`

Signature

```
Status PowerMap::load_state(self, const std::vector<Series>& state) override;
```

Notes

Loads a previously persisted artifact and restores its inference-ready state.

Parameters

- `state` : `const std::vector<Series>&` Randomization control for reproducible behavior.

Returns

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
PowerMap powerMap{};
auto out = powerMap.load_state(/* args */);
```

Source

- PlussAI/include/pluss/ai/nodej/prep.h:339

PowerMap::trained

Signature

```
[[nodiscard]] bool PowerMap::trained(self) const noexcept override;
```

Notes

Boolean state check for object readiness.

Parameters

None.

Returns

- Returns bool.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
PowerMap powerMap{};
auto out = powerMap.trained(/* args */);
```

Source

- PlussAI/include/pluss/ai/nodej/prep.h:332

QuantileMap::dump_state

Signature

```
[[nodiscard]] StatusOr<std::vector<Series>> QuantileMap::dump_state(self) const override;
```

Notes

Instance method on self.

Parameters

None.

Returns

- Returns `StatusOr<std::vector<Series>>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1

Source: `PlussAI/examples/NodeJ/core/nodej_end_to_end_hour_workflow_showcase.cpp`

```
    return 1;
  }
  Series train_maxabs_scaled = train_maxabs_scaled_or.move_value();

  StatusOr<Series> test_maxabs_scaled_or = maxabs_scaler.map(test_simple_batch);
  if (!test_maxabs_scaled_or.ok()) {
    std::cerr << "MaxAbsScaler.map(test) failed: " << test_maxabs_scaled_or.status().
↪message << "\n";
    return 1;
  }
  Series test_maxabs_scaled = test_maxabs_scaled_or.move_value();

  // This section saves compact prep-state diagnostics.
  StatusOr<std::vector<Series>> simple_state_or = simple_imputer.dump_state();
  if (!simple_state_or.ok()) {
    std::cerr << "SimpleImputer.dump_state failed: " << simple_state_or.status().message
↪<< "\n";
    return 1;
  }
  std::vector<Series> simple_state = simple_state_or.move_value();

  StatusOr<std::vector<Series>> knn_state_or = knn_imputer.dump_state();
  if (!knn_state_or.ok()) {
    std::cerr << "NeighborImputer.dump_state failed: " << knn_state_or.status().message <
↪< "\n";
    return 1;
  }
  std::vector<Series> knn_state = knn_state_or.move_value();

  StatusOr<std::vector<Series>> iter_state_or = iterative_imputer.dump_state();
  if (!iter_state_or.ok()) {
    std::cerr << "IterativeImputer.dump_state failed: " << iter_state_or.status().
↪message << "\n";
    return 1;
  }
  std::vector<Series> iter_state = iter_state_or.move_value();
```

Source

- PlussAI/include/pluss/ai/nodej/prep.h:317

QuantileMap::load_state

Signature

```
Status QuantileMap::load_state(self, const std::vector<Series>& state) override;
```

Notes

Loads a previously persisted artifact and restores its inference-ready state.

Parameters

- state : const std::vector<Series>& Randomization control for reproducible behavior.

Returns

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
QuantileMap quantileMap{};
auto out = quantileMap.load_state(/* args */);
```

Source

- PlussAI/include/pluss/ai/nodej/prep.h:318

QuantileMap::trained

Signature

```
[[nodiscard]] bool QuantileMap::trained(self) const noexcept override;
```

Notes

Boolean state check for object readiness.

Parameters

None.

Returns

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
QuantileMap quantileMap{};
auto out = quantileMap.trained(/* args */);
```

Source

- `PlussAI/include/pluss/ai/nodej/prep.h`:311

`RegressorBase::task_kind`

Signature

```
[[nodiscard]] TaskKind RegressorBase::task_kind(self) const noexcept override;
```

Notes

Instance method on `self`.

Parameters

None.

Returns

- Returns `TaskKind`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
RegressorBase regressorBase{};
auto out = regressorBase.task_kind(/* args */);
```

Source

- `PlussAI/include/pluss/ai/nodej/interfaces.h`:37

restore

Signature

```
StatusOr<Flow> restore(const FlowBundle& bundle);
```

Notes

Namespace-level API function.

Parameters

- `bundle` : `const FlowBundle&` Method argument; see signature type for accepted form.

Returns

- Returns `StatusOr<Flow>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

Option semantics used by this method:

- `FlowBundle` (`struct`):
  - `FlowBundleMetadata metadata{}` Learning-rate step size.
  - `ModelSpec model{}` Option field used by this method.
  - `std::vector<TransformSpec> transforms{}` Option field used by this method.
  - `bool trained = false` Option field used by this method.
  - `bool has_snapshot = false` Option field used by this method.
  - `std::vector<std::vector<Series>> transform_states{}` Option field used by this method.
  - `std::vector<Series> model_state{}` Option field used by this method.
  - `TrainOptions train_options{}` Controls fold/trial/count sizing.
  - `LabeledBatch train_batch{}` Controls fold/trial/count sizing.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
auto out = restore(/* args */);
```

Source

- `PlussAI/include/pluss/ai/nodej/io.h:38`

RobustScaler::dump_state

Signature

```
[[nodiscard]] StatusOr<std::vector<Series>> RobustScaler::dump_state(self) const␣
↪override;
```

Notes

Instance method on `self`.

Parameters

None.

Returns

- Returns `StatusOr<std::vector<Series>>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1

Source: `PlussAI/examples/NodeJ/core/nodej_end_to_end_hour_workflow_showcase.cpp`

```
    return 1;
  }
  Series train_maxabs_scaled = train_maxabs_scaled_or.move_value();

  StatusOr<Series> test_maxabs_scaled_or = maxabs_scaler.map(test_simple_batch);
  if (!test_maxabs_scaled_or.ok()) {
    std::cerr << "MaxAbsScaler.map(test) failed: " << test_maxabs_scaled_or.status().
↪message << "\n";
    return 1;
  }
  Series test_maxabs_scaled = test_maxabs_scaled_or.move_value();

  // This section saves compact prep-state diagnostics.
  StatusOr<std::vector<Series>> simple_state_or = simple_imputer.dump_state();
  if (!simple_state_or.ok()) {
    std::cerr << "SimpleImputer.dump_state failed: " << simple_state_or.status().message
↪<< "\n";
    return 1;
  }
  std::vector<Series> simple_state = simple_state_or.move_value();

  StatusOr<std::vector<Series>> knn_state_or = knn_imputer.dump_state();
  if (!knn_state_or.ok()) {
    std::cerr << "NeighborImputer.dump_state failed: " << knn_state_or.status().message <
↪< "\n";
    return 1;
  }
  std::vector<Series> knn_state = knn_state_or.move_value();

  StatusOr<std::vector<Series>> iter_state_or = iterative_imputer.dump_state();
  if (!iter_state_or.ok()) {
```

(continues on next page)

(continued from previous page)

```
    std::cerr << "IterativeImputer.dump_state failed: " << iter_state_or.status().
↪message << "\n";
    return 1;
  }
  std::vector<Series> iter_state = iter_state_or.move_value();
```

Source

- PlussAI/include/pluss/ai/nodej/prep.h:187

RobustScaler::load_state

Signature

```
Status RobustScaler::load_state(self, const std::vector<Series>& state) override;
```

Notes

Loads a previously persisted artifact and restores its inference-ready state.

Parameters

- state : const std::vector<Series>& Randomization control for reproducible behavior.

Returns

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
RobustScaler robustScaler{};
auto out = robustScaler.load_state(/* args */);
```

Source

- PlussAI/include/pluss/ai/nodej/prep.h:188

RobustScaler::trained

Signature

```
[[nodiscard]] bool RobustScaler::trained(self) const noexcept override;
```

Notes

Boolean state check for object readiness.

Parameters

None.

Returns

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
RobustScaler robustScaler{};
auto out = robustScaler.trained(/* args */);
```

Source

- `PlussAI/include/pluss/ai/nodej/prep.h:181`

`SimpleImputer::dump_state`

Signature

```
[[nodiscard]] StatusOr<std::vector<Series>> SimpleImputer::dump_state(self) const␣
↪override;
```

Notes

Instance method on `self`.

Parameters

None.

Returns

- Returns `StatusOr<std::vector<Series>>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1

Source: PlussAI/examples/NodeJ/core/nodej_end_to_end_hour_workflow_showcase.cpp

```
    return 1;
  }
  Series train_maxabs_scaled = train_maxabs_scaled_or.move_value();

  StatusOr<Series> test_maxabs_scaled_or = maxabs_scaler.map(test_simple_batch);
  if (!test_maxabs_scaled_or.ok()) {
    std::cerr << "MaxAbsScaler.map(test) failed: " << test_maxabs_scaled_or.status().
↪message << "\n";
    return 1;
  }
  Series test_maxabs_scaled = test_maxabs_scaled_or.move_value();

  // This section saves compact prep-state diagnostics.
  StatusOr<std::vector<Series>> simple_state_or = simple_imputer.dump_state();
  if (!simple_state_or.ok()) {
    std::cerr << "SimpleImputer.dump_state failed: " << simple_state_or.status().message
↪<< "\n";
    return 1;
  }
  std::vector<Series> simple_state = simple_state_or.move_value();

  StatusOr<std::vector<Series>> knn_state_or = knn_imputer.dump_state();
  if (!knn_state_or.ok()) {
    std::cerr << "NeighborImputer.dump_state failed: " << knn_state_or.status().message <
↪< "\n";
    return 1;
  }
  std::vector<Series> knn_state = knn_state_or.move_value();

  StatusOr<std::vector<Series>> iter_state_or = iterative_imputer.dump_state();
  if (!iter_state_or.ok()) {
    std::cerr << "IterativeImputer.dump_state failed: " << iter_state_or.status().
↪message << "\n";
    return 1;
  }
  std::vector<Series> iter_state = iter_state_or.move_value();
```

Source

- PlussAI/include/pluss/ai/nodej/prep.h:209

SimpleImputer::load_state

Signature

```
Status SimpleImputer::load_state(self, const std::vector<Series>& state) override;
```

Notes

Loads a previously persisted artifact and restores its inference-ready state.

Parameters

- `state` : `const std::vector<Series>&` Randomization control for reproducible behavior.

Returns

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
SimpleImputer simpleImputer{};
auto out = simpleImputer.load_state(/* args */);
```

Source

- `PlussAI/include/pluss/ai/nodej/prep.h:210`

`SimpleImputer::trained`

Signature

```
[[nodiscard]] bool SimpleImputer::trained(self) const noexcept override;
```

Notes

Boolean state check for object readiness.

Parameters

None.

Returns

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
SimpleImputer simpleImputer{};
auto out = simpleImputer.trained(/* args */);
```

Source

- PlussAI/include/pluss/ai/nodej/prep.h:203

slice_feature_batch

Signature

```
StatusOr<FeatureBatch> slice_feature_batch(const FeatureBatch& in, const Series&␣
↪indices);
```

Notes

Returns a row-subset of a feature-only batch using the provided index selection.

Parameters

- in : const FeatureBatch& Method argument; see signature type for accepted form.
- indices : const Series& Pluss numeric vector/tensor input.

Returns

- Returns StatusOr<FeatureBatch>. Check .ok() first; on success read .value() for the computed result.

Options

Option semantics used by this method:

- FeatureBatch (struct):
  - Series x = Series::undefined() Option field used by this method.
  - ObservationLayout layout = ObservationLayout::Rows Declares whether observations are represented by rows or columns.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
auto out = slice_feature_batch(/* args */);
```

Source

- PlussAI/include/pluss/ai/nodej/split.h:27

slice_labeled_batch

Signature

```
StatusOr<LabeledBatch> slice_labeled_batch(const LabeledBatch& in, const Series&␣
↪indices);
```

Notes

Returns a row-subset of a labeled batch using the provided index selection.

Parameters

- `in` : `const LabeledBatch&` Method argument; see signature type for accepted form.
- `indices` : `const Series&` Pluss numeric vector/tensor input.

Returns

- Returns `StatusOr<LabeledBatch>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

Option semantics used by this method:

- `LabeledBatch` (`struct`):
  - `FeatureBatch features{}` Option field used by this method.
  - `Series y = Series::undefined()` Option field used by this method.
  - `Series sample_weights = Series::undefined()` Optional column used as per-row training weight.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
auto out = slice_labeled_batch(/* args */);
```

Source

- `PlussAI/include/pluss/ai/nodej/split.h:28`

`StandardScaler::dump_state`

Signature

```
[[nodiscard]] StatusOr<std::vector<Series>> StandardScaler::dump_state(self) const␣
↪override;
```

Notes

Instance method on `self`.

Parameters

None.

Returns

- Returns `StatusOr<std::vector<Series>>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1

Source: PlussAI/examples/NodeJ/core/nodej_end_to_end_hour_workflow_showcase.cpp

```
    return 1;
  }
  Series train_maxabs_scaled = train_maxabs_scaled_or.move_value();

  StatusOr<Series> test_maxabs_scaled_or = maxabs_scaler.map(test_simple_batch);
  if (!test_maxabs_scaled_or.ok()) {
    std::cerr << "MaxAbsScaler.map(test) failed: " << test_maxabs_scaled_or.status().
↪message << "\n";
    return 1;
  }
  Series test_maxabs_scaled = test_maxabs_scaled_or.move_value();

  // This section saves compact prep-state diagnostics.
  StatusOr<std::vector<Series>> simple_state_or = simple_imputer.dump_state();
  if (!simple_state_or.ok()) {
    std::cerr << "SimpleImputer.dump_state failed: " << simple_state_or.status().message
↪<< "\n";
    return 1;
  }
  std::vector<Series> simple_state = simple_state_or.move_value();

  StatusOr<std::vector<Series>> knn_state_or = knn_imputer.dump_state();
  if (!knn_state_or.ok()) {
    std::cerr << "NeighborImputer.dump_state failed: " << knn_state_or.status().message <
↪< "\n";
    return 1;
  }
  std::vector<Series> knn_state = knn_state_or.move_value();

  StatusOr<std::vector<Series>> iter_state_or = iterative_imputer.dump_state();
  if (!iter_state_or.ok()) {
    std::cerr << "IterativeImputer.dump_state failed: " << iter_state_or.status().
↪message << "\n";
    return 1;
  }
  std::vector<Series> iter_state = iter_state_or.move_value();
```

Source

- PlussAI/include/pluss/ai/nodej/prep.h:143

StandardScaler::load_state

Signature

```
Status StandardScaler::load_state(self, const std::vector<Series>& state) override;
```

Notes

Loads a previously persisted artifact and restores its inference-ready state.

Parameters

- state : const std::vector<Series>& Randomization control for reproducible behavior.

Returns

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
StandardScaler standardScaler{};
auto out = standardScaler.load_state(/* args */);
```

Source

- PlussAI/include/pluss/ai/nodej/prep.h:144

StandardScaler::trained

Signature

```
[[nodiscard]] bool StandardScaler::trained(self) const noexcept override;
```

Notes

Boolean state check for object readiness.

Parameters

None.

Returns

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
StandardScaler standardScaler{};
auto out = standardScaler.trained(/* args */);
```

Source

- `PlussAI/include/pluss/ai/nodej/prep.h:137`

`Status::ok`

Signature

```
[[nodiscard]] bool Status::ok(self) const noexcept;
```

Notes

Boolean state check for object readiness.

Parameters

None.

Returns

- Returns `bool`.

Options

Option semantics used by this method:

- `Status` (`struct`):
  - `StatusCode code = StatusCode::Ok` Option field used by this method.
  - `std::string message{}` Option field used by this method.

Examples

Example 1

Source: `PlussAI/examples/NodeJ/core/nodej_cv_tune_demo.cpp`

```
using pluss::Config;
using pluss::ScalarType;
using pluss::Series;
using namespace pluss::ai::nodej;

Config f64() {
  return Config{}.with_dtype(ScalarType::Float64).with_platform(pluss::pCPU);
}

template <class T>
bool require_ok(const StatusOr<T>& out, const std::string& label) {
  if (out.ok()) return true;
  std::cerr << label << " failed: " << out.status().message << "\n";
  return false;
}

void print_series(const std::string& label, const Series& x) {
  const auto v = x.to(pluss::pCPU, ScalarType::Float64).reshape({x.numel()}).to_vec
↪<double>();
  std::cout << label << ":";
  for (double value : v) std::cout << " " << value;
  std::cout << "\n";
}

LabeledBatch make_xor_batch() {
  const Series x = pluss::make_series(std::vector<double>{
                                        -1.0, -1.0,
                                        -1.0,  1.0,
                                         1.0, -1.0,
                                         1.0,  1.0,
                                        -0.9, -0.9,
                                        -0.9,  0.9,
                                         0.9, -0.9,
```

Output

```
NodeJ CV + tune demo
linear mean metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0.25 0.25 0.
↪208333 0.25 nan 0.0625 0.960532
linear std metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0.176777 0.
↪176777 0.216506 0.25 nan 0.108253 0.192821
kernel mean metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 1 1 1 1 1 1␣
↪0.358895
kernel std metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0 0 0 0 0 0␣
↪0.0358183
sweep trial objectives: 0.25 1
sweep best index: 1
probe best index: 1
regression quantile sweep objectives: 0.0356052 0.736518
regression quantile sweep best index: 0
max |bounded score|: 0
NodeJ CV + tune demo completed.
```

Example 2

Source: PlussAI/examples/NodeJ/core/nodej_dataframe_bridge_demo.cpp

```
using pluss::ScalarType;
using pluss::Series;
using pluss::table::DataFrame;
using pluss::table::Thread;
using namespace pluss::ai::nodej;

Config f64() {
  return Config{}.with_dtype(ScalarType::Float64).with_platform(pluss::pCPU);
}

template <class T>
bool require_ok(const StatusOr<T>& out, const std::string& label) {
  if (out.ok()) return true;
  std::cerr << label << " failed: " << out.status().message << "\n";
  return false;
}

void print_metrics(const std::string& label, const Series& metrics) {
  const auto v = metrics.to(pluss::pCPU, ScalarType::Float64).reshape({metrics.numel()}).
↪to_vec<double>();
  std::cout << label << ":";
  for (double m : v) std::cout << " " << m;
  std::cout << "\n";
}

DataFrame make_frame() {
  const Series x0 = pluss::make_series(std::vector<double>{0.0, 1.0, 2.0, 3.0, 4.0, 5.0},
↪ {6}, f64());
  const Series x1 = pluss::make_series(std::vector<double>{1.0, 0.0, 2.0, 1.0, 3.0, 2.0},
↪ {6}, f64());
  const auto category = pluss::make_note({"red", "blue", "red", "green", "blue", "red"});
  const Series epoch_nanos = pluss::make_series(
      std::vector<std::int64_t>{
          1'700'000'000'000'000'000LL, 1'700'000'001'000'000'000LL, 1'700'000'002'000'000
↪'000LL,
          1'700'000'003'000'000'000LL, 1'700'000'004'000'000'000LL, 1'700'000'005'000'000
↪'000LL},
      {6},
```

Output

```
NodeJ DataFrame bridge demo
adapted batch: samples=6 features=4 weighted=yes
linear metrics [mse, r2]: 9.43684e-08 1
NodeJ DataFrame bridge demo completed.
```

Example 3

Source: PlussAI/examples/NodeJ/core/nodej_flow_bundle_demo.cpp

```
  auto flow_or = make_flow(spec);
  if (!flow_or.ok()) {
    std::cerr << "make_flow failed: " << flow_or.status().message << "\n";
    return 1;
  }
  Flow flow = flow_or.move_value();
  if (!flow.add_standard_scaler().ok()) {
    std::cerr << "add_standard_scaler failed\n";
    return 1;
  }
  TrainOptions train{};
  train.max_iterations = 2500;
  train.tolerance = 1e-8;
  train.random.seed = 77;
  train.random.deterministic = true;
  if (!flow.learn(batch, train).ok()) {
    std::cerr << "flow learn failed\n";
    return 1;
  }
```

Output

```
saved flow bundle to: <repo_root>/PlussAI/examples/NodeJ/trained_models/nodej_flow_
↪bundle_demo.bin
flow bundle roundtrip exact prediction parity: true
nodej_flow_bundle_demo complete
```

Source

- PlussAI/include/pluss/ai/nodej/status.h:22

Status::success

Signature

```
static Status Status::success(self);
```

Notes

Instance method on `self`.

Parameters

None.

Returns

- Returns a status object indicating success/failure and an error message on failure.

Options

Option semantics used by this method:

- `Status` (`struct`):
  - `StatusCode code = StatusCode::Ok` Option field used by this method.
  - `std::string message{}` Option field used by this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
auto out = Status::success(/* args */);
```

Source

- `PlussAI/include/pluss/ai/nodej/status.h:24`

`StatusOr::move_value`

Signature

```
[[nodiscard]] T&& StatusOr::move_value(self) noexcept;
```

Notes

Instance method on `self`.

Parameters

None.

Returns

- Returns `T&&`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1

Source: `PlussAI/examples/NodeJ/core/nodej_cv_tune_demo.cpp`

```
                            TrainOptions{},
                            reg_assess,
                            MetricKind::QuantileLoss,
```

(continues on next page)

(continued from previous page)

```
                            TuneObjectivePolicy::Auto);
  if (!require_ok(reg_sweep_or, "sweep(regression quantile)")) return 1;
  std::cout << "regression quantile sweep objectives:";
  for (const auto& trial : reg_sweep_or.value().trials) std::cout << " " << trial.
↪objective;
  std::cout << "\n";
  std::cout << "regression quantile sweep best index: " << reg_sweep_or.value().best_
↪index << "\n";

  auto bounded_or = make_classifier(linear_logistic_spec());
  if (!require_ok(bounded_or, "make_classifier(linear bounded)")) return 1;
  auto bounded = bounded_or.move_value();

  TrainOptions bounded_train = train;
  bounded_train.parameter_lower_bounds = pluss::make_series(std::vector<double>{0.0}, {1}
↪, f64());
  bounded_train.parameter_upper_bounds = pluss::make_series(std::vector<double>{0.0}, {1}
↪, f64());
  if (!bounded->learn(batch, bounded_train).ok()) {
    std::cerr << "bounded linear learn failed\n";
    return 1;
  }

  auto bounded_scores_or = bounded->infer_scores(batch.features);
  if (!require_ok(bounded_scores_or, "infer_scores(bounded)")) return 1;
  std::cout << "max |bounded score|: " << bounded_scores_or.value().abs().max().item
↪<double>() << "\n";

  std::cout << "NodeJ CV + tune demo completed.\n";
  return 0;
}
```

Output

```
NodeJ CV + tune demo
linear mean metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0.25 0.25 0.
↪208333 0.25 nan 0.0625 0.960532
linear std metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0.176777 0.
↪176777 0.216506 0.25 nan 0.108253 0.192821
kernel mean metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 1 1 1 1 1 1␣
↪0.358895
kernel std metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0 0 0 0 0 0␣
↪0.0358183
sweep trial objectives: 0.25 1
sweep best index: 1
probe best index: 1
regression quantile sweep objectives: 0.0356052 0.736518
regression quantile sweep best index: 0
max |bounded score|: 0
NodeJ CV + tune demo completed.
```

Example 2

Source: PlussAI/examples/NodeJ/core/nodej_dataframe_bridge_demo.cpp

```
      .task = TaskKind::Regression,
      .family = ModelFamily::Linear,
      .options = LinearFamilyOptions{
          .objective = LinearObjective::LeastSquares,
          .regularization = RegularizationKind::L2,
          .regularization_strength = 1e-8,
          .fit_intercept = true,
      },
  };

  auto reg_or = make_regressor(spec);
  if (!require_ok(reg_or, "make_regressor")) return 1;
  auto reg = reg_or.move_value();
  Status st = reg->learn(batch);
  if (!st.ok()) {
    std::cerr << "learn failed: " << st.message << "\n";
    return 1;
  }

  auto metrics_or = reg->assess(batch, AssessOptions{.metrics = {MetricKind::Mse,␣
↪MetricKind::R2}});
  if (!require_ok(metrics_or, "assess")) return 1;
  print_metrics("linear metrics [mse, r2]", metrics_or.value());

  std::cout << "NodeJ DataFrame bridge demo completed.\n";
  return 0;
}
```

Output

```
NodeJ DataFrame bridge demo
adapted batch: samples=6 features=4 weighted=yes
linear metrics [mse, r2]: 9.43684e-08 1
NodeJ DataFrame bridge demo completed.
```

Example 3

Source: PlussAI/examples/NodeJ/core/nodej_flow_bundle_demo.cpp

```
  auto flow_or = make_flow(spec);
  if (!flow_or.ok()) {
    std::cerr << "make_flow failed: " << flow_or.status().message << "\n";
    return 1;
  }
  Flow flow = flow_or.move_value();
  if (!flow.add_standard_scaler().ok()) {
    std::cerr << "add_standard_scaler failed\n";
    return 1;
  }
  TrainOptions train{};
```

(continues on next page)

(continued from previous page)

```
  train.max_iterations = 2500;
  train.tolerance = 1e-8;
  train.random.seed = 77;
  train.random.deterministic = true;
  if (!flow.learn(batch, train).ok()) {
    std::cerr << "flow learn failed\n";
    return 1;
  }
```

Output

```
saved flow bundle to: <repo_root>/PlussAI/examples/NodeJ/trained_models/nodej_flow_
↪bundle_demo.bin
flow bundle roundtrip exact prediction parity: true
nodej_flow_bundle_demo complete
```

Source

- `PlussAI/include/pluss/ai/nodej/status.h:58`

`StatusOr::ok`

Signature

```
[[nodiscard]] bool StatusOr::ok(self) const noexcept;
```

Notes

Boolean state check for object readiness.

Parameters

None.

Returns

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1

Source: `PlussAI/examples/NodeJ/core/nodej_cv_tune_demo.cpp`

```
using pluss::Config;
using pluss::ScalarType;
using pluss::Series;
```

(continues on next page)

(continued from previous page)

```cpp
using namespace pluss::ai::nodej;

Config f64() {
  return Config{}.with_dtype(ScalarType::Float64).with_platform(pluss::pCPU);
}

template <class T>
bool require_ok(const StatusOr<T>& out, const std::string& label) {
  if (out.ok()) return true;
  std::cerr << label << " failed: " << out.status().message << "\n";
  return false;
}

void print_series(const std::string& label, const Series& x) {
  const auto v = x.to(pluss::pCPU, ScalarType::Float64).reshape({x.numel()}).to_vec
↪<double>();
  std::cout << label << ":";
  for (double value : v) std::cout << " " << value;
  std::cout << "\n";
}

LabeledBatch make_xor_batch() {
  const Series x = pluss::make_series(std::vector<double>{
                                          -1.0, -1.0,
                                          -1.0,  1.0,
                                           1.0, -1.0,
                                           1.0,  1.0,
                                          -0.9, -0.9,
                                          -0.9,  0.9,
                                           0.9, -0.9,
```

Output

```
NodeJ CV + tune demo
linear mean metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0.25 0.25 0.
↪208333 0.25 nan 0.0625 0.960532
linear std metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0.176777 0.
↪176777 0.216506 0.25 nan 0.108253 0.192821
kernel mean metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 1 1 1 1 1 1␣
↪0.358895
kernel std metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0 0 0 0 0 0␣
↪0.0358183
sweep trial objectives: 0.25 1
sweep best index: 1
probe best index: 1
regression quantile sweep objectives: 0.0356052 0.736518
regression quantile sweep best index: 0
max |bounded score|: 0
NodeJ CV + tune demo completed.
```

Example 2

Source: PlussAI/examples/NodeJ/core/nodej_dataframe_bridge_demo.cpp

```
using pluss::ScalarType;
using pluss::Series;
using pluss::table::DataFrame;
using pluss::table::Thread;
using namespace pluss::ai::nodej;

Config f64() {
  return Config{}.with_dtype(ScalarType::Float64).with_platform(pluss::pCPU);
}

template <class T>
bool require_ok(const StatusOr<T>& out, const std::string& label) {
  if (out.ok()) return true;
  std::cerr << label << " failed: " << out.status().message << "\n";
  return false;
}

void print_metrics(const std::string& label, const Series& metrics) {
  const auto v = metrics.to(pluss::pCPU, ScalarType::Float64).reshape({metrics.numel()}).
↪to_vec<double>();
  std::cout << label << ":";
  for (double m : v) std::cout << " " << m;
  std::cout << "\n";
}

DataFrame make_frame() {
  const Series x0 = pluss::make_series(std::vector<double>{0.0, 1.0, 2.0, 3.0, 4.0, 5.0},
↪ {6}, f64());
  const Series x1 = pluss::make_series(std::vector<double>{1.0, 0.0, 2.0, 1.0, 3.0, 2.0},
↪ {6}, f64());
  const auto category = pluss::make_note({"red", "blue", "red", "green", "blue", "red"});
  const Series epoch_nanos = pluss::make_series(
      std::vector<std::int64_t>{
          1'700'000'000'000'000'000LL, 1'700'000'001'000'000'000LL, 1'700'000'002'000'000
↪'000LL,
          1'700'000'003'000'000'000LL, 1'700'000'004'000'000'000LL, 1'700'000'005'000'000
↪'000LL},
      {6},
```

Output

```
NodeJ DataFrame bridge demo
adapted batch: samples=6 features=4 weighted=yes
linear metrics [mse, r2]: 9.43684e-08 1
NodeJ DataFrame bridge demo completed.
```

Example 3

Source: PlussAI/examples/NodeJ/core/nodej_flow_bundle_demo.cpp

```
  auto flow_or = make_flow(spec);
  if (!flow_or.ok()) {
    std::cerr << "make_flow failed: " << flow_or.status().message << "\n";
    return 1;
  }
  Flow flow = flow_or.move_value();
  if (!flow.add_standard_scaler().ok()) {
    std::cerr << "add_standard_scaler failed\n";
    return 1;
  }
  TrainOptions train{};
  train.max_iterations = 2500;
  train.tolerance = 1e-8;
  train.random.seed = 77;
  train.random.deterministic = true;
  if (!flow.learn(batch, train).ok()) {
    std::cerr << "flow learn failed\n";
    return 1;
  }
```

Output

```
saved flow bundle to: <repo_root>/PlussAI/examples/NodeJ/trained_models/nodej_flow_
↪bundle_demo.bin
flow bundle roundtrip exact prediction parity: true
nodej_flow_bundle_demo complete
```

Source

- PlussAI/include/pluss/ai/nodej/status.h:54

StatusOr::status

Signature

```
[[nodiscard]] const Status& StatusOr::status(self) const noexcept;
```

Notes

Instance method on `self`.

Parameters

None.

Returns

- Returns `const Status&`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1

Source: `PlussAI/examples/NodeJ/core/nodej_cv_tune_demo.cpp`

```
using pluss::Config;
using pluss::ScalarType;
using pluss::Series;
using namespace pluss::ai::nodej;

Config f64() {
  return Config{}.with_dtype(ScalarType::Float64).with_platform(pluss::pCPU);
}

template <class T>
bool require_ok(const StatusOr<T>& out, const std::string& label) {
  if (out.ok()) return true;
  std::cerr << label << " failed: " << out.status().message << "\n";
  return false;
}

void print_series(const std::string& label, const Series& x) {
  const auto v = x.to(pluss::pCPU, ScalarType::Float64).reshape({x.numel()}).to_vec
↪<double>();
  std::cout << label << ":";
  for (double value : v) std::cout << " " << value;
  std::cout << "\n";
}

LabeledBatch make_xor_batch() {
  const Series x = pluss::make_series(std::vector<double>{
                                        -1.0, -1.0,
                                        -1.0,  1.0,
                                         1.0, -1.0,
                                         1.0,  1.0,
                                        -0.9, -0.9,
                                        -0.9,  0.9,
                                         0.9, -0.9,
                                         0.9,  0.9,
```

Output

```
NodeJ CV + tune demo
linear mean metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0.25 0.25 0.
↪208333 0.25 nan 0.0625 0.960532
```

(continues on next page)

(continued from previous page)

```
linear std metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0.176777 0.
↪176777 0.216506 0.25 nan 0.108253 0.192821
kernel mean metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 1 1 1 1 1 1␣
↪0.358895
kernel std metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0 0 0 0 0 0␣
↪0.0358183
sweep trial objectives: 0.25 1
sweep best index: 1
probe best index: 1
regression quantile sweep objectives: 0.0356052 0.736518
regression quantile sweep best index: 0
max |bounded score|: 0
NodeJ CV + tune demo completed.
```

Example 2

Source: PlussAI/examples/NodeJ/core/nodej_dataframe_bridge_demo.cpp

```cpp
using pluss::Series;
using pluss::table::DataFrame;
using pluss::table::Thread;
using namespace pluss::ai::nodej;

Config f64() {
  return Config{}.with_dtype(ScalarType::Float64).with_platform(pluss::pCPU);
}

template <class T>
bool require_ok(const StatusOr<T>& out, const std::string& label) {
  if (out.ok()) return true;
  std::cerr << label << " failed: " << out.status().message << "\n";
  return false;
}

void print_metrics(const std::string& label, const Series& metrics) {
  const auto v = metrics.to(pluss::pCPU, ScalarType::Float64).reshape({metrics.numel()}).
↪to_vec<double>();
  std::cout << label << ":";
  for (double m : v) std::cout << " " << m;
  std::cout << "\n";
}

DataFrame make_frame() {
  const Series x0 = pluss::make_series(std::vector<double>{0.0, 1.0, 2.0, 3.0, 4.0, 5.0},
↪ {6}, f64());
  const Series x1 = pluss::make_series(std::vector<double>{1.0, 0.0, 2.0, 1.0, 3.0, 2.0},
↪ {6}, f64());
  const auto category = pluss::make_note({"red", "blue", "red", "green", "blue", "red"});
  const Series epoch_nanos = pluss::make_series(
      std::vector<std::int64_t>{
          1'700'000'000'000'000'000LL, 1'700'000'001'000'000'000LL, 1'700'000'002'000'000
↪'000LL,
```

(continues on next page)

(continued from previous page)

```
          1'700'000'003'000'000'000LL, 1'700'000'004'000'000'000LL, 1'700'000'005'000'000
↪'000LL},
      {6},
      Config{}.with_dtype(ScalarType::Int64).with_platform(pluss::pCPU));
```

Output

```
NodeJ DataFrame bridge demo
adapted batch: samples=6 features=4 weighted=yes
linear metrics [mse, r2]: 9.43684e-08 1
NodeJ DataFrame bridge demo completed.
```

Example 3

Source: PlussAI/examples/NodeJ/core/nodej_flow_bundle_demo.cpp

```
  auto flow_or = make_flow(spec);
  if (!flow_or.ok()) {
    std::cerr << "make_flow failed: " << flow_or.status().message << "\n";
    return 1;
  }
  Flow flow = flow_or.move_value();
  if (!flow.add_standard_scaler().ok()) {
    std::cerr << "add_standard_scaler failed\n";
    return 1;
  }
  TrainOptions train{};
  train.max_iterations = 2500;
  train.tolerance = 1e-8;
  train.random.seed = 77;
  train.random.deterministic = true;
  if (!flow.learn(batch, train).ok()) {
    std::cerr << "flow learn failed\n";
    return 1;
  }
```

Output

```
saved flow bundle to: <repo_root>/PlussAI/examples/NodeJ/trained_models/nodej_flow_
↪bundle_demo.bin
flow bundle roundtrip exact prediction parity: true
nodej_flow_bundle_demo complete
```

Source

- PlussAI/include/pluss/ai/nodej/status.h:55

`StatusOr::StatusOr`

Signature

```
explicit StatusOr(Status status) : status_(std::move(status)), value_;
```

```
explicit StatusOr(T value) : status_(Status::success()), value_(std::move(value)), has_
↪value_(true);
```

```
StatusOr(Status status, T value) : status_(std::move(status)), value_(std::move(value)),␣
↪has_value_(status_.ok());
```

Notes

Instance method on `self`.

Parameters

- `arg1 : Status status) : status_(std::move(status) | T value) : status_(Status::success()) | std::move(status))` Method argument; see signature type for accepted form.
- `arg2` : `value_(std::move(value))` Method argument; see signature type for accepted form.
- `arg3` : `has_value_(status_.ok()` Method argument; see signature type for accepted form.
- `true` : `has_value_(` Method argument; see signature type for accepted form.

Returns

- Returns `StatusOr(Status status, T value) :`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1

Source: `PlussAI/examples/NodeJ/core/nodej_cv_tune_demo.cpp`

```
namespace {

using pluss::Config;
using pluss::ScalarType;
using pluss::Series;
using namespace pluss::ai::nodej;

Config f64() {
  return Config{}.with_dtype(ScalarType::Float64).with_platform(pluss::pCPU);
}

template <class T>
bool require_ok(const StatusOr<T>& out, const std::string& label) {
  if (out.ok()) return true;
```

(continues on next page)

(continued from previous page)

```
  std::cerr << label << " failed: " << out.status().message << "\n";
  return false;
}

void print_series(const std::string& label, const Series& x) {
  const auto v = x.to(pluss::pCPU, ScalarType::Float64).reshape({x.numel()}).to_vec
↪<double>();
  std::cout << label << ":";
  for (double value : v) std::cout << " " << value;
  std::cout << "\n";
}

LabeledBatch make_xor_batch() {
  const Series x = pluss::make_series(std::vector<double>{
                                        -1.0, -1.0,
                                        -1.0,  1.0,
                                         1.0, -1.0,
                                         1.0,  1.0,
                                        -0.9, -0.9,
                                        -0.9,  0.9,
```

Output

```
NodeJ CV + tune demo
linear mean metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0.25 0.25 0.
↪208333 0.25 nan 0.0625 0.960532
linear std metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0.176777 0.
↪176777 0.216506 0.25 nan 0.108253 0.192821
kernel mean metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 1 1 1 1 1 1␣
↪0.358895
kernel std metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0 0 0 0 0 0␣
↪0.0358183
sweep trial objectives: 0.25 1
sweep best index: 1
probe best index: 1
regression quantile sweep objectives: 0.0356052 0.736518
regression quantile sweep best index: 0
max |bounded score|: 0
NodeJ CV + tune demo completed.
```

Example 2

Source: `PlussAI/examples/NodeJ/core/nodej_dataframe_bridge_demo.cpp`

```
using pluss::Config;
using pluss::ScalarType;
using pluss::Series;
using pluss::table::DataFrame;
using pluss::table::Thread;
using namespace pluss::ai::nodej;

Config f64() {
```

(continues on next page)

(continued from previous page)

```
  return Config{}.with_dtype(ScalarType::Float64).with_platform(pluss::pCPU);
}

template <class T>
bool require_ok(const StatusOr<T>& out, const std::string& label) {
  if (out.ok()) return true;
  std::cerr << label << " failed: " << out.status().message << "\n";
  return false;
}

void print_metrics(const std::string& label, const Series& metrics) {
  const auto v = metrics.to(pluss::pCPU, ScalarType::Float64).reshape({metrics.numel()}).
↪to_vec<double>();
  std::cout << label << ":";
  for (double m : v) std::cout << " " << m;
  std::cout << "\n";
}

DataFrame make_frame() {
  const Series x0 = pluss::make_series(std::vector<double>{0.0, 1.0, 2.0, 3.0, 4.0, 5.0},
↪ {6}, f64());
  const Series x1 = pluss::make_series(std::vector<double>{1.0, 0.0, 2.0, 1.0, 3.0, 2.0},
↪ {6}, f64());
  const auto category = pluss::make_note({"red", "blue", "red", "green", "blue", "red"});
  const Series epoch_nanos = pluss::make_series(
      std::vector<std::int64_t>{
          1'700'000'000'000'000'000LL, 1'700'000'001'000'000'000LL, 1'700'000'002'000'000
↪'000LL,
          1'700'000'003'000'000'000LL, 1'700'000'004'000'000'000LL, 1'700'000'005'000'000
↪'000LL},
```

Output

```
NodeJ DataFrame bridge demo
adapted batch: samples=6 features=4 weighted=yes
linear metrics [mse, r2]: 9.43684e-08 1
NodeJ DataFrame bridge demo completed.
```

Example 3

Source: PlussAI/examples/NodeJ/core/nodej_flow_demo.cpp

```
namespace {

using pluss::Config;
using pluss::ScalarType;
using pluss::Series;
using namespace pluss::ai::nodej;

Config f64() {
  return Config{}.with_dtype(ScalarType::Float64).with_platform(pluss::pCPU);
}
```

(continues on next page)

(continued from previous page)

```
template <class T>
bool require_ok(const StatusOr<T>& out, const std::string& label) {
  if (out.ok()) return true;
  std::cerr << label << " failed: " << out.status().message << "\n";
  return false;
}

void print_series(const std::string& label, const Series& x) {
  const auto v = x.to(pluss::pCPU, ScalarType::Float64).reshape({x.numel()}).to_vec
↪<double>();
  std::cout << label << ":";
  for (double value : v) std::cout << " " << value;
  std::cout << "\n";
}

} // namespace

int main() {
  std::cout << "NodeJ Flow demo\n";

  {
    const Series x = pluss::make_series(std::vector<double>{
                                          -3.0, -1.0,
```

Output

```
NodeJ Flow demo
classification flow metrics [accuracy, logloss]: 1 0.000228657
regression flow metrics [mse, r2]: 4.46648e-17 1
NodeJ Flow demo completed.
```

Source

- PlussAI/include/pluss/ai/nodej/status.h:52
- PlussAI/include/pluss/ai/nodej/status.h:51
- PlussAI/include/pluss/ai/nodej/status.h:48

StatusOr::value

Signature

```
[[nodiscard]] const T& StatusOr::value(self) const noexcept;
```

```
[[nodiscard]] T& StatusOr::value(self) noexcept;
```

Notes

Instance method on `self`.

Parameters

None.

Returns

- Returns `T&`.
- Returns `const T&`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1

Source: `PlussAI/examples/NodeJ/core/nodej_cv_tune_demo.cpp`

```
  AssessOptions assess{};
  assess.metrics = {
      MetricKind::Accuracy,
      MetricKind::BalancedAccuracy,
      MetricKind::Precision,
      MetricKind::Recall,
      MetricKind::F1,
      MetricKind::RocAuc,
      MetricKind::LogLoss};

  auto linear_cv_or = cross_assess(linear_logistic_spec(), batch, split, train, assess);
  if (!require_ok(linear_cv_or, "cross_assess(linear)")) return 1;
  print_series("linear mean metrics [acc, bal_acc, precision, recall, f1, roc_auc,␣
↪logloss]", linear_cv_or.value().mean_metrics);
  print_series("linear std metrics [acc, bal_acc, precision, recall, f1, roc_auc,␣
↪logloss]", linear_cv_or.value().std_metrics);

  auto kernel_cv_or = cross_assess(rbf_kernel_spec(), batch, split, train, assess);
  if (!require_ok(kernel_cv_or, "cross_assess(kernel)")) return 1;
  print_series("kernel mean metrics [acc, bal_acc, precision, recall, f1, roc_auc,␣
↪logloss]", kernel_cv_or.value().mean_metrics);
  print_series("kernel std metrics [acc, bal_acc, precision, recall, f1, roc_auc,␣
↪logloss]", kernel_cv_or.value().std_metrics);

  const std::vector<ModelSpec> candidates{linear_logistic_spec(), rbf_kernel_spec()};
  AssessOptions tune_assess{};
  tune_assess.metrics = {MetricKind::Accuracy};
```

(continues on next page)

(continued from previous page)

```
  auto sweep_or = sweep(candidates,
                        batch,
                        split,
                        train,
                        tune_assess,
                        MetricKind::Accuracy,
                        TuneObjectivePolicy::Auto);
  if (!require_ok(sweep_or, "sweep")) return 1;
```

Output

```
NodeJ CV + tune demo
linear mean metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0.25 0.25 0.
↪208333 0.25 nan 0.0625 0.960532
linear std metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0.176777 0.
↪176777 0.216506 0.25 nan 0.108253 0.192821
kernel mean metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 1 1 1 1 1 1␣
↪0.358895
kernel std metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0 0 0 0 0 0␣
↪0.0358183
sweep trial objectives: 0.25 1
sweep best index: 1
probe best index: 1
regression quantile sweep objectives: 0.0356052 0.736518
regression quantile sweep best index: 0
max |bounded score|: 0
NodeJ CV + tune demo completed.
```

Example 2

Source: PlussAI/examples/NodeJ/core/nodej_dataframe_bridge_demo.cpp

```
  const DataFrame frame = make_frame();

  DataFrameBatchOptions options{};
  options.feature_names = {"x0", "x1", "category", "ts", "target", "weight"};
  options.label_feature = "target";
  options.sample_weight_feature = "weight";
  options.note_policy = DataNotePolicy::Hash;
  options.thread_policy = DataThreadPolicy::CastTemporalToSeries;

  auto batch_or = labeled_batch_from_dataframe(frame, options);
  if (!require_ok(batch_or, "labeled_batch_from_dataframe")) return 1;

  const LabeledBatch& batch = batch_or.value();
  std::cout << "adapted batch: samples=" << batch.features.sample_count()
            << " features=" << batch.features.feature_count()
            << " weighted=" << (batch.sample_weights.defined() ? "yes" : "no") << "\n";

  ModelSpec spec{
      .task = TaskKind::Regression,
```

(continues on next page)

(continued from previous page)

```
      .family = ModelFamily::Linear,
      .options = LinearFamilyOptions{
          .objective = LinearObjective::LeastSquares,
          .regularization = RegularizationKind::L2,
          .regularization_strength = 1e-8,
          .fit_intercept = true,
      },
  };

  auto reg_or = make_regressor(spec);
  if (!require_ok(reg_or, "make_regressor")) return 1;
  auto reg = reg_or.move_value();
  Status st = reg->learn(batch);
  if (!st.ok()) {
```

Output

```
NodeJ DataFrame bridge demo
adapted batch: samples=6 features=4 weighted=yes
linear metrics [mse, r2]: 9.43684e-08 1
NodeJ DataFrame bridge demo completed.
```

Example 3

Source: PlussAI/examples/NodeJ/core/nodej_flow_bundle_demo.cpp

```
  auto restored_or = nio::load_and_restore(bundle_path);
  if (!restored_or.ok()) {
    std::cerr << "load_and_restore failed: " << restored_or.status().message << "\n";
    return 1;
  }
  auto pred_after_or = restored_or.value().infer(batch.features);
  if (!pred_after_or.ok()) {
    std::cerr << "restored flow infer failed: " << pred_after_or.status().message << "\n
↪";
    return 1;
  }
```

Output

```
saved flow bundle to: <repo_root>/PlussAI/examples/NodeJ/trained_models/nodej_flow_
↪bundle_demo.bin
flow bundle roundtrip exact prediction parity: true
nodej_flow_bundle_demo complete
```

Source

- PlussAI/include/pluss/ai/nodej/status.h:56
- PlussAI/include/pluss/ai/nodej/status.h:57

ThresholdBinarizer::dump_state

Signature

```
[[nodiscard]] StatusOr<std::vector<Series>> ThresholdBinarizer::dump_state(self) const␣
↪override;
```

Notes

Instance method on `self`.

Parameters

None.

Returns

- Returns `StatusOr<std::vector<Series>>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1

Source: `PlussAI/examples/NodeJ/core/nodej_end_to_end_hour_workflow_showcase.cpp`

```
    return 1;
  }
  Series train_maxabs_scaled = train_maxabs_scaled_or.move_value();

  StatusOr<Series> test_maxabs_scaled_or = maxabs_scaler.map(test_simple_batch);
  if (!test_maxabs_scaled_or.ok()) {
    std::cerr << "MaxAbsScaler.map(test) failed: " << test_maxabs_scaled_or.status().
↪message << "\n";
    return 1;
  }
  Series test_maxabs_scaled = test_maxabs_scaled_or.move_value();

  // This section saves compact prep-state diagnostics.
  StatusOr<std::vector<Series>> simple_state_or = simple_imputer.dump_state();
  if (!simple_state_or.ok()) {
    std::cerr << "SimpleImputer.dump_state failed: " << simple_state_or.status().message
↪<< "\n";
    return 1;
  }
  std::vector<Series> simple_state = simple_state_or.move_value();

  StatusOr<std::vector<Series>> knn_state_or = knn_imputer.dump_state();
  if (!knn_state_or.ok()) {
```

(continues on next page)

(continued from previous page)

```
    std::cerr << "NeighborImputer.dump_state failed: " << knn_state_or.status().message <
↪< "\n";
    return 1;
  }
  std::vector<Series> knn_state = knn_state_or.move_value();

  StatusOr<std::vector<Series>> iter_state_or = iterative_imputer.dump_state();
  if (!iter_state_or.ok()) {
    std::cerr << "IterativeImputer.dump_state failed: " << iter_state_or.status().
↪message << "\n";
    return 1;
  }
  std::vector<Series> iter_state = iter_state_or.move_value();
```

Source

- PlussAI/include/pluss/ai/nodej/prep.h:361

ThresholdBinarizer::load_state

Signature

```
Status ThresholdBinarizer::load_state(self, const std::vector<Series>& state) override;
```

Notes

Loads a previously persisted artifact and restores its inference-ready state.

Parameters

- `state` : `const std::vector<Series>&` Randomization control for reproducible behavior.

Returns

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
ThresholdBinarizer thresholdBinarizer{};
auto out = thresholdBinarizer.load_state(/* args */);
```

Source

- PlussAI/include/pluss/ai/nodej/prep.h:362

ThresholdBinarizer::trained

Signature

```
[[nodiscard]] bool ThresholdBinarizer::trained(self) const noexcept override;
```

Notes

Boolean state check for object readiness.

Parameters

None.

Returns

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
ThresholdBinarizer thresholdBinarizer{};
auto out = thresholdBinarizer.trained(/* args */);
```

Source

- `PlussAI/include/pluss/ai/nodej/prep.h:355`

TransformerBase::dump_state

Signature

```
[[nodiscard]] virtual StatusOr<std::vector<Series>> TransformerBase::dump_state(self)␣
↪const;
```

Notes

Instance method on `self`.

Parameters

None.

Returns

- Returns `StatusOr<std::vector<Series>>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1

Source: PlussAI/examples/NodeJ/core/nodej_end_to_end_hour_workflow_showcase.cpp

```
    return 1;
  }
  Series train_maxabs_scaled = train_maxabs_scaled_or.move_value();

  StatusOr<Series> test_maxabs_scaled_or = maxabs_scaler.map(test_simple_batch);
  if (!test_maxabs_scaled_or.ok()) {
    std::cerr << "MaxAbsScaler.map(test) failed: " << test_maxabs_scaled_or.status().
↪message << "\n";
    return 1;
  }
  Series test_maxabs_scaled = test_maxabs_scaled_or.move_value();

  // This section saves compact prep-state diagnostics.
  StatusOr<std::vector<Series>> simple_state_or = simple_imputer.dump_state();
  if (!simple_state_or.ok()) {
    std::cerr << "SimpleImputer.dump_state failed: " << simple_state_or.status().message
↪<< "\n";
    return 1;
  }
  std::vector<Series> simple_state = simple_state_or.move_value();

  StatusOr<std::vector<Series>> knn_state_or = knn_imputer.dump_state();
  if (!knn_state_or.ok()) {
    std::cerr << "NeighborImputer.dump_state failed: " << knn_state_or.status().message <
↪< "\n";
    return 1;
  }
  std::vector<Series> knn_state = knn_state_or.move_value();

  StatusOr<std::vector<Series>> iter_state_or = iterative_imputer.dump_state();
  if (!iter_state_or.ok()) {
    std::cerr << "IterativeImputer.dump_state failed: " << iter_state_or.status().
↪message << "\n";
    return 1;
  }
  std::vector<Series> iter_state = iter_state_or.move_value();
```

Source

- PlussAI/include/pluss/ai/nodej/interfaces.h:51

TransformerBase::load_state

Signature

```
virtual Status TransformerBase::load_state(self, const std::vector<Series>& state);
```

Notes

Loads a previously persisted artifact and restores its inference-ready state.

Parameters

- `state` : `const std::vector<Series>&` Randomization control for reproducible behavior.

Returns

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
TransformerBase transformerBase{};
auto out = transformerBase.load_state(/* args */);
```

Source

- PlussAI/include/pluss/ai/nodej/interfaces.h:54

TransformerBase::trained

Signature

```
[[nodiscard]] virtual bool TransformerBase::trained(self) const noexcept = 0;
```

Notes

Boolean state check for object readiness.

Parameters

None.

Returns

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
TransformerBase transformerBase{};
auto out = transformerBase.trained(/* args */);
```

Source

- `PlussAI/include/pluss/ai/nodej/interfaces.h:44`

`VectorNormalizer::dump_state`

Signature

```
[[nodiscard]] StatusOr<std::vector<Series>> VectorNormalizer::dump_state(self) const␣
↪override;
```

Notes

Instance method on `self`.

Parameters

None.

Returns

- Returns `StatusOr<std::vector<Series>>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1

Source: `PlussAI/examples/NodeJ/core/nodej_end_to_end_hour_workflow_showcase.cpp`

```
  return 1;
}
Series train_maxabs_scaled = train_maxabs_scaled_or.move_value();
```

(continues on next page)

(continued from previous page)

```
  StatusOr<Series> test_maxabs_scaled_or = maxabs_scaler.map(test_simple_batch);
  if (!test_maxabs_scaled_or.ok()) {
    std::cerr << "MaxAbsScaler.map(test) failed: " << test_maxabs_scaled_or.status().
↪message << "\n";
    return 1;
  }
  Series test_maxabs_scaled = test_maxabs_scaled_or.move_value();

  // This section saves compact prep-state diagnostics.
  StatusOr<std::vector<Series>> simple_state_or = simple_imputer.dump_state();
  if (!simple_state_or.ok()) {
    std::cerr << "SimpleImputer.dump_state failed: " << simple_state_or.status().message
↪<< "\n";
    return 1;
  }
  std::vector<Series> simple_state = simple_state_or.move_value();

  StatusOr<std::vector<Series>> knn_state_or = knn_imputer.dump_state();
  if (!knn_state_or.ok()) {
    std::cerr << "NeighborImputer.dump_state failed: " << knn_state_or.status().message <
↪< "\n";
    return 1;
  }
  std::vector<Series> knn_state = knn_state_or.move_value();

  StatusOr<std::vector<Series>> iter_state_or = iterative_imputer.dump_state();
  if (!iter_state_or.ok()) {
    std::cerr << "IterativeImputer.dump_state failed: " << iter_state_or.status().
↪message << "\n";
    return 1;
  }
  std::vector<Series> iter_state = iter_state_or.move_value();
```

Source

- PlussAI/include/pluss/ai/nodej/prep.h:276

VectorNormalizer::load_state

Signature

```
Status VectorNormalizer::load_state(self, const std::vector<Series>& state) override;
```

Notes

Loads a previously persisted artifact and restores its inference-ready state.

Parameters

- `state` : `const std::vector<Series>&` Randomization control for reproducible behavior.

Returns

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
VectorNormalizer vectorNormalizer{};
auto out = vectorNormalizer.load_state(/* args */);
```

Source

- `PlussAI/include/pluss/ai/nodej/prep.h:277`

`VectorNormalizer::trained`

Signature

```
[[nodiscard]] bool VectorNormalizer::trained(self) const noexcept override;
```

Notes

Boolean state check for object readiness.

Parameters

None.

Returns

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
VectorNormalizer vectorNormalizer{};
auto out = vectorNormalizer.trained(/* args */);
```

Source

- PlussAI/include/pluss/ai/nodej/prep.h:270

CHAPTER

FIVE

LYMPH FULL API REFERENCE

Lymph method-centric reference generated from public headers. Each page follows **Signature -> Notes -> Parameters -> Returns -> Options -> Examples -> Output -> Source**. Options/enums are documented under the methods that consume them.

- Method pages: **61**
- Data-type pages: **24**
- Overloads covered: **69**

5.1 Function and Method Reference

5.1.1 Lymph Full API Reference: Method Index

Total method pages: **61** Total data-type pages: **24**

Data Types

`BatchHoldoutSplit`

Signature

```
struct BatchHoldoutSplit;
```

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Options

Public fields/options on this type:

- `LabeledBatch train_set{}`
- `LabeledBatch test_set{}`
- `IndexSplit indices{}`

Examples

```
BatchHoldoutSplit cfg{};
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:321

bayes_factory

Signature

```
struct bayes_factory;
```

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Examples

```
bayes_factory cfg{};
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:2214

Candidate

Signature

```
struct Candidate;
```

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Options

Public fields/options on this type:

- std::string name{}
- Workflow workflow{}
- Candidate& operator=(Candidate&&) noexcept = default
- Candidate& operator=(const Candidate&) = delete

Examples

```
Candidate cfg{};
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:2581

cluster_factory

Signature

```
struct cluster_factory;
```

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Examples

```
cluster_factory cfg{};
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:2242

CrossValidationReport

Signature

```
struct CrossValidationReport;
```

Notes

Structured report object carrying metrics/diagnostics from training/evaluation routines.

Options

Public fields/options on this type:

- CrossAssessReport report{}
- MetricReport mean{}
- MetricReport stdev{}

Examples

```
CrossValidationReport cfg{};
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:340

decompose_factory

Signature

```
struct decompose_factory;
```

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Examples

```
decompose_factory cfg{};
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:2257

ensemble_factory

Signature

```
struct ensemble_factory;
```

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Examples

```
ensemble_factory cfg{};
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:2074

FrameFlowArtifact

Signature

```
struct FrameFlowArtifact;
```

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Options

Public fields/options on this type:

- nodej::Flow flow{}
- nodej::Series metrics = nodej::Series::undefined()
- nodej::LabeledBatch train_batch{}

Examples

```
FrameFlowArtifact cfg{};
```

Source

- PlussAI/include/pluss/ai/lymph/quick.h:43

FrameHoldoutSplit

Signature

```
struct FrameHoldoutSplit;
```

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Options

Public fields/options on this type:

- pluss::table::DataFrame train_set{}
- pluss::table::DataFrame test_set{}
- IndexSplit indices{}

Examples

```
FrameHoldoutSplit cfg{};
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:315

kernel_factory

Signature

```
struct kernel_factory;
```

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Examples

```
kernel_factory cfg{};
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:2148

linear_factory

Signature

```
struct linear_factory;
```

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Examples

```
linear_factory cfg{};
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:2021

LinearParameters

Signature

```
struct LinearParameters;
```

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Options

Public fields/options on this type:

- Series coeff = Series::undefined()
- Series intercept = Series::undefined()
- bool fit_intercept = false

Examples

```
LinearParameters cfg{};
// cfg.coeff = ...;
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:300

MetricReport

Signature

```
struct MetricReport;
```

Notes

Structured report object carrying metrics/diagnostics from training/evaluation routines.

Options

Public fields/options on this type:

- `std::vector<MetricKind> requested{}`
- `Series values = Series::undefined()`
- `Series accuracy = Series::undefined()`
- `Series balanced_accuracy = Series::undefined()`
- `Series precision = Series::undefined()`
- `Series recall = Series::undefined()`
- `Series f1 = Series::undefined()`
- `Series roc_auc = Series::undefined()`
- `Series logloss = Series::undefined()`
- `Series mse = Series::undefined()`
- `Series rmse = Series::undefined()`
- `Series mae = Series::undefined()`
- `Series r2 = Series::undefined()`
- `Series quantile_loss = Series::undefined()`

Examples

```
MetricReport cfg{};
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:259

neighbor_factory

Signature

```
struct neighbor_factory;
```

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Examples

```
neighbor_factory cfg{};
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:2122

online_factory

Signature

```
struct online_factory;
```

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Examples

```
online_factory cfg{};
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:2569

SeriesCompareReport

Signature

```
struct SeriesCompareReport;
```

Notes

Structured report object carrying metrics/diagnostics from training/evaluation routines.

Options

Public fields/options on this type:

- Series abs_error = Series::undefined()
- Series max_abs_diff = Series::undefined()
- Series mae = Series::undefined()

Examples

```
SeriesCompareReport cfg{};
// cfg.abs_error = ...;
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:276

SimpleImputerSummary

Signature

```
struct SimpleImputerSummary;
```

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Options

Public fields/options on this type:

- std::vector<std::string> feature_names{}
- Series statistics = Series::undefined()

Examples

```
SimpleImputerSummary cfg{};
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:61

SpecTuneResult

Signature

```
struct SpecTuneResult;
```

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Options

Public fields/options on this type:

- ::pluss::ai::nodej::TuneReport report{}
- ModelSpec best_spec{}

Examples

```
SpecTuneResult cfg{};
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:2612

StandardScalerSummary

Signature

```
struct StandardScalerSummary;
```

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Options

Public fields/options on this type:

- std::vector<std::string> feature_names{}
- Series mean = Series::undefined()
- Series scale = Series::undefined()

Examples

```
StandardScalerSummary cfg{};
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:66

TransformerSummary

Signature

```
struct TransformerSummary;
```

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Options

Public fields/options on this type:

- std::vector<std::string> feature_names{}
- std::vector<Series> state{}

Examples

```
TransformerSummary cfg{};
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:72

tree_factory

Signature

```
struct tree_factory;
```

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Examples

```
tree_factory cfg{};
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:2050

Trial

Signature

```
struct Trial;
```

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Options

Public fields/options on this type:

- std::string name{}
- double objective = std::numeric_limits<double>::quiet_NaN()
- CrossValidationReport cv{}

Examples

```
Trial cfg{};
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:2593

TuneReport

Signature

```
struct TuneReport;
```

Notes

Structured report object carrying metrics/diagnostics from training/evaluation routines.

Options

Public fields/options on this type:

- std::vector<Trial> trials{}
- std::int64_t best_index = -1
- std::string best_name{}
- MetricKind objective_metric = MetricKind::Accuracy
- TuneObjectivePolicy objective_policy = TuneObjectivePolicy::Auto

Examples

Example 1

Source: PlussAI/examples/NodeJ/core/nodej_end_to_end_hour_workflow_showcase.cpp

```
  StatusOr<TuneReport> broad_sweep_or = sweep(
      broad_specs,
      train_batch,
      cv_split,
      train_options,
      assess_options,
      MetricKind::Rmse,
      TuneObjectivePolicy::Minimize);
  if (!broad_sweep_or.ok()) {
    std::cerr << "sweep(broad) failed: " << broad_sweep_or.status().message << "\n";
    return 1;
  }
  TuneReport broad_sweep = broad_sweep_or.move_value();
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:2599

TuneResult

Signature

```
struct TuneResult;
```

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Options

Public fields/options on this type:

- `TuneReport report{}`
- `Workflow best_workflow{}`

Examples

```
TuneResult cfg{};
```

Source

- `PlussAI/include/pluss/ai/lymph/api.h:2607`

Methods

as_stratify_series

Signature

```
[[nodiscard]] inline Series as_stratify_series(const pluss::table::Thread& thread);
```

```
[[nodiscard]] inline Series as_stratify_series(const pluss::table::Thread& thread);
```

Notes

Namespace-level API function.

Parameters

- `thread` : `const pluss::table::Thread&` Pluss numeric vector/tensor input.

Returns

- Returns `Series`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```cpp
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
auto out = as_stratify_series(/* args */);
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:327
- PlussAI/include/pluss/ai/lymph/api.h:487

Candidate::Candidate

Signature

```cpp
Candidate(std::string in_name, Workflow in_workflow) : name(std::move(in_name)),␣
↪workflow(std::move(in_workflow));
```

Notes

Instance method on `self`.

Parameters

- `arg1` : `std::move(in_name))` Method argument; see signature type for accepted form.
- `arg2` : `workflow(std::move(in_workflow)` Method argument; see signature type for accepted form.

Returns

- Returns `Candidate(std::string in_name, Workflow in_workflow) :`.

Options

Option semantics used by this method:

- `Candidate` (`struct`):
  - `std::string name{}` Option field used by this method.
  - `Workflow workflow{}` Option field used by this method.
  - `Candidate& operator=(Candidate&&) noexcept = default` Option field used by this method.
  - `Candidate& operator=(const Candidate&) = delete` Option field used by this method.

Examples

Example 1

Source: `PlussAI/examples/NodeJ/core/nodej_workflow_tuning_demo.cpp`

```cpp
    std::vector<nja::selection::Candidate> candidates{};
    {
      auto wf = nja::Workflow(nja::linear_factory::linear_regression(LinearFamilyOptions{
          .objective = LinearObjective::LeastSquares,
```

(continues on next page)

(continued from previous page)

```
          .regularization = RegularizationKind::L2,
          .regularization_strength = 1e-6,
          .fit_intercept = true,
      }));
      wf.simple_imputer(nja::prep::SimpleImputerOptions{.strategy =␣
↪nja::prep::SimpleImputerStrategy::Median})
          .standard_scaler();
      candidates.emplace_back("linear_ridge", std::move(wf));
    }
    {
      auto wf = nja::Workflow(nja::tree_factory::decision_tree_
↪regressor(TreeFamilyOptions{
          .criterion = TreeCriterion::Mse,
          .max_depth = 18,
          .min_samples_leaf = 8,
          .min_samples_split = 24,
      }));
```

Example 2

Source: PlussAI/examples/NodeJ/lymph/lymph_end_to_end_hour_workflow_showcase.cpp

```
    lymph::CrossValidationOptions cv_options{};
    cv_options.plan = CVPlanKind::KFold;
    cv_options.folds = 5;
    cv_options.stratified = false;
    cv_options.reliability_defaults = true;
    cv_options.shuffle = true;
    cv_options.random_state = 73;
    cv_options.deterministic = true;

    //========== modelling: broad candidate sweep ========
    // This section builds a broad set of workflows spanning linear, tree,
    // ensemble, neighbor, kernel, and bayes regressors.
    std::vector<lymph::selection::Candidate> broad_candidates{};

    {
      auto wf = lymph::Workflow(lymph::linear_factory::linear_
↪regression(LinearFamilyOptions{
          .objective = LinearObjective::LeastSquares,
          .regularization = RegularizationKind::None,
          .regularization_strength = 0.0,
          .fit_intercept = true,
      }));
      wf.simple_imputer(lymph::prep::SimpleImputerOptions{.strategy =␣
↪lymph::prep::SimpleImputerStrategy::Median})
          .standard_scaler();
      broad_candidates.emplace_back("linear_ols_standard", std::move(wf));
    }
    {
      auto wf = lymph::Workflow(lymph::linear_factory::linear_
↪regression(LinearFamilyOptions{
```

(continues on next page)

(continued from previous page)

```
        .objective = LinearObjective::LeastSquares,
        .regularization = RegularizationKind::L2,
        .regularization_strength = 1e-8,
        .fit_intercept = true,
    }));
    wf.simple_imputer(lymph::prep::SimpleImputerOptions{.strategy =␣
↪lymph::prep::SimpleImputerStrategy::Median})
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:2585

compare_series

Signature

```
[[nodiscard]] inline SeriesCompareReport compare_series(const Series& lhs, const Series&␣
↪rhs);
```

Notes

Namespace-level API function.

Parameters

- lhs : const Series& Pluss numeric vector/tensor input.
- rhs : const Series& Pluss numeric vector/tensor input.

Returns

- Returns SeriesCompareReport.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
auto out = compare_series(/* args */);
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:282

make_cross_validation_split_options

Signature

```
[[nodiscard]] inline SplitOptions make_cross_validation_split_options(const␣
↪CrossValidationOptions& options);
```

Notes

Builds train/test index partitions according to split-plan options.

Parameters

- `options` : `const CrossValidationOptions&` Configuration object controlling algorithm behavior and safety defaults.

Returns

- Returns `SplitOptions`.

Options

Option semantics used by this method:

- `CrossValidationOptions` (`struct`):
  - `CVPlanKind plan = CVPlanKind::KFold` Controls fold/trial/count sizing.
  - `double holdout_fraction = 0.2` Option field used by this method.
  - `std::int64_t folds = 5` Controls fold/trial/count sizing.
  - `bool stratified = false` Option field used by this method.
  - `bool reliability_defaults = true` Option field used by this method.
  - `bool shuffle = true` Option field used by this method.
  - `std::uint64_t random_state = 0` Random seed for reproducible behavior.
  - `bool deterministic = true` Option field used by this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
auto out = make_cross_validation_split_options(/* args */);
```

Source

- `PlussAI/include/pluss/ai/lymph/api.h:355`

make_holdout_indices

Signature

```
[[nodiscard]] inline IndexSplit make_holdout_indices(const pluss::table::DataFrame&␣
↪frame, const HoldoutSplitOptions& options);
```

Notes

Namespace-level API function.

Parameters

- `frame` : `const pluss::table::DataFrame&` Input feature data used by this call.
- `options` : `const HoldoutSplitOptions&` Configuration object controlling algorithm behavior and safety defaults.

Returns

- Returns `IndexSplit`.

Options

Option semantics used by this method:

- `HoldoutSplitOptions` (`struct`):
  - `double test_size = 0.2` Option field used by this method.
  - `Series stratify = Series::undefined()` Option field used by this method.
  - `std::string stratify_feature{}` Option field used by this method.
  - `std::uint64_t random_state = 0` Random seed for reproducible behavior.
  - `bool deterministic = true` Option field used by this method.
  - `bool shuffle = true` Option field used by this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
auto out = make_holdout_indices(/* args */);
```

Source

- `PlussAI/include/pluss/ai/lymph/api.h:492`

make_holdout_split_options

Signature

```
[[nodiscard]] inline SplitOptions make_holdout_split_options(const HoldoutSplitOptions&
↪options);
```

Notes

Builds train/test index partitions according to split-plan options.

Parameters

- `options` : `const HoldoutSplitOptions&` Configuration object controlling algorithm behavior and safety defaults.

Returns

- Returns `SplitOptions`.

Options

Option semantics used by this method:

- `HoldoutSplitOptions` (`struct`):
  - `double test_size = 0.2` Option field used by this method.
  - `Series stratify = Series::undefined()` Option field used by this method.
  - `std::string stratify_feature{}` Option field used by this method.
  - `std::uint64_t random_state = 0` Random seed for reproducible behavior.
  - `bool deterministic = true` Option field used by this method.
  - `bool shuffle = true` Option field used by this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
auto out = make_holdout_split_options(/* args */);
```

Source

- `PlussAI/include/pluss/ai/lymph/api.h:346`

`max_abs_diff`

Signature

```
[[nodiscard]] inline double max_abs_diff(const Series& lhs, const Series& rhs);
```

Notes

Namespace-level API function.

Parameters

- `lhs` : `const Series&` Pluss numeric vector/tensor input.
- `rhs` : `const Series&` Pluss numeric vector/tensor input.

Returns

- Returns double.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1

Source: PlussAI/examples/NodeJ/core/bike_sharing_end_to_end.cpp

```
    std::cout << "linear test metrics [rmse, r2, mae]: " << linear_metrics.rmse << " " <
↪< linear_metrics.r2 << " "
              << linear_metrics.mae << "\n";
    std::cout << "linear params: intercept=" << linear_workflow.intercept() << " coeff="
↪<< linear_workflow.coeff() << "\n";

    const Series linear_pred_series = linear_workflow.infer(test_x);
    const fs::path linear_model_path = model_dir / "bike_sharing_linear.bin";
    linear_workflow.save(linear_model_path.string(), train_x, train_y, train_options);
    const auto linear_restored = lymph::Workflow::load(linear_model_path.string());
    const Series linear_pred_restored_series = linear_restored.infer(test_x);
    const Series y_true = test_y.to(pluss::pCPU, ScalarType::Float64).reshape({test_y.
↪numel()});

    const Series linear_pred = linear_pred_series.to(pluss::pCPU, ScalarType::Float64).
↪reshape({linear_pred_series.numel()});
    const double linear_max_abs_diff = lymph::max_abs_diff(linear_pred_series, linear_
↪pred_restored_series);
    std::cout << "linear restored prediction max_abs_diff: " << linear_max_abs_diff << "\
↪n";

    DataFrame linear_frame({
        {"y_true", Thread(y_true)},
        {"y_pred", Thread(linear_pred)},
        {"residual", Thread(y_true.sub(linear_pred))},
    });
    linear_frame.save_csv((out_dir / "bike_sharing_test_predictions_linear.csv").
↪string());
    linear_frame.plot()
        .scatter("y_true", "y_pred")
        .headline("Observed vs predicted bike demand (linear)")
        .save_pdf((out_dir / "bike_pred_vs_true_linear.pdf").string());
    linear_frame.plot()
        .hist("residual", 80)
        .headline("Residual distribution (linear)")
        .save_pdf((out_dir / "bike_residual_hist_linear.pdf").string());
    std::cout << "saved model (linear): " << linear_model_path << "\n";

    auto tree_workflow = lymph::Workflow(
  lymph::tree_factory::decision_tree_regressor(TreeFamilyOptions{
```

Example 2

Source: PlussAI/examples/NodeJ/core/nodej_workflow_tuning_demo.cpp

```
    std::cout << "best candidate: " << tuned.report.best_name << "\n";

    auto best = std::move(tuned.best_workflow);
    best.learn_(train_x, train_y, train_options);
    const auto test_metrics = best.assess(test_x, test_y, assess_options);
    std::cout << "test metrics [rmse, r2, mae]: " << test_metrics.rmse << " " << test_
↪metrics.r2 << " " << test_metrics.mae << "\n";

    const Series pred = best.infer(test_x);
    const fs::path best_model_path = model_dir / "hour_best_workflow.bin";
    best.save(best_model_path.string(), train_x, train_y, train_options);
    auto restored = nja::Workflow::load(best_model_path.string());
    const Series pred_restored = restored.infer(test_x);
    const double restored_max_abs_diff = nja::max_abs_diff(pred, pred_restored);
    std::cout << "restored prediction max_abs_diff: " << restored_max_abs_diff << "\n";

    const Series y_true = test_y.to(pluss::pCPU, ScalarType::Float64).reshape({test_y.
↪numel()});
    const Series y_pred = pred.to(pluss::pCPU, ScalarType::Float64).reshape({pred.
↪numel()});
    DataFrame pred_frame({
        {"y_true", Thread(y_true)},
        {"y_pred", Thread(y_pred)},
        {"residual", Thread(y_true.sub(y_pred))},
    });
    pred_frame.save_csv((out_dir / "hour_workflow_tuning_predictions.csv").string());
    pred_frame.plot()
        .scatter("y_true", "y_pred")
        .headline("Observed vs predicted bike demand (best workflow)")
        .save_pdf((out_dir / "hour_workflow_tuning_pred_vs_true.pdf").string());

    std::cout << "saved best model: " << best_model_path << "\n";
    std::cout << "saved outputs under: " << out_dir << "\n";
    return 0;
  } catch (const std::exception& ex) {
    std::cerr << "nodej_workflow_tuning_demo failed: " << ex.what() << "\n";
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:296

OnlineLinearClassifier::core

Signature

```
[[nodiscard]] ::pluss::ai::nodej::online::OnlineLinearClassifier&␣
↪OnlineLinearClassifier::core(self) noexcept;
```

```
[[nodiscard]] const ::pluss::ai::nodej::online::OnlineLinearClassifier&␣
↪OnlineLinearClassifier::core(self) const noexcept;
```

Notes

Instance method on `self`.

Parameters

None.

Returns

- Returns `::pluss::ai::nodej::online::OnlineLinearClassifier&`.
- Returns `const ::pluss::ai::nodej::online::OnlineLinearClassifier&`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1

Source: `PlussAI/examples/NodeJ/core/nodej_end_to_end_hour_workflow_showcase.cpp`

```
  //========== schema normalization ========
  // This section gives consistent names and explicit core column dtypes.
  DataFrame frame = raw.rename_features({
      {"dteday", "date"},
      {"holiday", "is_holiday"},
      {"workingday", "is_workingday"},
      {"weathersit", "weather_condition"},
      {"hum", "humidity"},
      {"mnth", "month"},
      {"cnt", "total_count"},
      {"hr", "hour"},
      {"yr", "year"},
  });
```

Source

- `PlussAI/include/pluss/ai/lymph/api.h:2382`
- `PlussAI/include/pluss/ai/lymph/api.h:2381`

`OnlineLinearClassifier::OnlineLinearClassifier`

Signature

```
explicit OnlineLinearClassifier(::pluss::ai::nodej::online::OnlineLinearClassifier core)␣
↪: core_(std::move(core));
```

Notes

Instance method on `self`.

Parameters

- `arg1 : ::pluss::ai::nodej::online::OnlineLinearClassifier core) : core_(std::move(core)` Method argument; see signature type for accepted form.

Returns

- Returns method result for the selected overload.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
OnlineLinearClassifier onlineLinearClassifier{};
auto out = onlineLinearClassifier.OnlineLinearClassifier(/* args */);
```

Source

- `PlussAI/include/pluss/ai/lymph/api.h:2279`

`OnlineLinearClassifier::seen_samples`

Signature

```
[[nodiscard]] std::int64_t OnlineLinearClassifier::seen_samples(self) const noexcept;
```

Notes

Reports a cardinality/count statistic from the current object state.

Parameters

None.

Returns

- Returns `std::int64_t`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1

Source: PlussAI/examples/NodeJ/core/nodej_online_demo.cpp

```
    std::cerr << "advance failed\n";
    return 1;
  }

  pluss::ai::nodej::AssessOptions assess{};
  assess.metrics = {MetricKind::Accuracy, MetricKind::LogLoss};
  auto metric_or = model.assess(full, assess);
  if (!metric_or.ok()) {
    std::cerr << "assess failed: " << metric_or.status().message << '\n';
    return 1;
  }

  std::cout << "online seen_samples=" << model.seen_samples() << '\n';
  std::cout << "online metrics [accuracy, logloss]: ";
  print_vec(metric_or.value());
  return 0;
}
```

Output

```
online seen_samples=1872
online metrics [accuracy, logloss]: 1 0.27341
```

Example 2

Source: PlussAI/examples/NodeJ/lymph/lymph_online_demo.cpp

```
    };
    online_options.learning_rate = 0.08;

    TrainOptions warm_opts{};
    warm_opts.max_iterations = 80;

    auto model = pluss::ai::lymph::online_factory::linear_classifier(online_options);
    model.learn_(warm, warm_opts).advance_(stream);

    const auto metrics = model.assess(full);
    const Series metric_values = metrics.accuracy.defined() ? metrics.accuracy : metrics.
↪values;

    std::cout << "lymph online seen_samples=" << model.seen_samples() << '\n';
    std::cout << "lymph online metrics [accuracy]: ";
    print_vec(metric_values);
    return 0;
  } catch (const std::exception& ex) {
    std::cerr << "lymph_online_demo failed: " << ex.what() << '\n';
    return 1;
  }
```

(continues on next page)

(continued from previous page)

```
}
```

Output

```
lymph online seen_samples=1660
lymph online metrics [accuracy]: 0.9875
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:2283

OnlineLinearClassifier::trained

Signature

```
[[nodiscard]] bool OnlineLinearClassifier::trained(self) const noexcept;
```

Notes

Boolean state check for object readiness.

Parameters

None.

Returns

- Returns bool.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
OnlineLinearClassifier onlineLinearClassifier{};
auto out = onlineLinearClassifier.trained(/* args */);
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:2282

OnlineLinearRegressor::core

Signature

```
[[nodiscard]] ::pluss::ai::nodej::online::OnlineLinearRegressor&␣
↪OnlineLinearRegressor::core(self) noexcept;
```

```
[[nodiscard]] const ::pluss::ai::nodej::online::OnlineLinearRegressor&␣
↪OnlineLinearRegressor::core(self) const noexcept;
```

Notes

Instance method on `self`.

Parameters

None.

Returns

- Returns `::pluss::ai::nodej::online::OnlineLinearRegressor&`.
- Returns `const ::pluss::ai::nodej::online::OnlineLinearRegressor&`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1

Source: `PlussAI/examples/NodeJ/core/nodej_end_to_end_hour_workflow_showcase.cpp`

```
//========== schema normalization ========
// This section gives consistent names and explicit core column dtypes.
DataFrame frame = raw.rename_features({
    {"dteday", "date"},
    {"holiday", "is_holiday"},
    {"workingday", "is_workingday"},
    {"weathersit", "weather_condition"},
    {"hum", "humidity"},
    {"mnth", "month"},
    {"cnt", "total_count"},
    {"hr", "hour"},
    {"yr", "year"},
});
```

Source

- `PlussAI/include/pluss/ai/lymph/api.h:2524`
- `PlussAI/include/pluss/ai/lymph/api.h:2523`

`OnlineLinearRegressor::OnlineLinearRegressor`

Signature

```
explicit OnlineLinearRegressor(::pluss::ai::nodej::online::OnlineLinearRegressor core) :␣
↪core_(std::move(core));
```

Notes

Instance method on `self`.

Parameters

- `arg1 : ::pluss::ai::nodej::online::OnlineLinearRegressor core) : core_(std::move(core)` Method argument; see signature type for accepted form.

Returns

- Returns method result for the selected overload.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
OnlineLinearRegressor onlineLinearRegressor{};
auto out = onlineLinearRegressor.OnlineLinearRegressor(/* args */);
```

Source

- `PlussAI/include/pluss/ai/lymph/api.h:2432`

`OnlineLinearRegressor::seen_samples`

Signature

```
[[nodiscard]] std::int64_t OnlineLinearRegressor::seen_samples(self) const noexcept;
```

Notes

Reports a cardinality/count statistic from the current object state.

Parameters

None.

Returns

- Returns `std::int64_t`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1

Source: PlussAI/examples/NodeJ/core/nodej_online_demo.cpp

```
    std::cerr << "advance failed\n";
    return 1;
  }

  pluss::ai::nodej::AssessOptions assess{};
  assess.metrics = {MetricKind::Accuracy, MetricKind::LogLoss};
  auto metric_or = model.assess(full, assess);
  if (!metric_or.ok()) {
    std::cerr << "assess failed: " << metric_or.status().message << '\n';
    return 1;
  }

  std::cout << "online seen_samples=" << model.seen_samples() << '\n';
  std::cout << "online metrics [accuracy, logloss]: ";
  print_vec(metric_or.value());
  return 0;
}
```

Output

```
online seen_samples=1872
online metrics [accuracy, logloss]: 1 0.27341
```

Example 2

Source: PlussAI/examples/NodeJ/lymph/lymph_online_demo.cpp

```
    };
    online_options.learning_rate = 0.08;

    TrainOptions warm_opts{};
    warm_opts.max_iterations = 80;

    auto model = pluss::ai::lymph::online_factory::linear_classifier(online_options);
    model.learn_(warm, warm_opts).advance_(stream);

    const auto metrics = model.assess(full);
    const Series metric_values = metrics.accuracy.defined() ? metrics.accuracy : metrics.
↪values;

    std::cout << "lymph online seen_samples=" << model.seen_samples() << '\n';
    std::cout << "lymph online metrics [accuracy]: ";
    print_vec(metric_values);
    return 0;
  } catch (const std::exception& ex) {
    std::cerr << "lymph_online_demo failed: " << ex.what() << '\n';
    return 1;
  }
```

(continues on next page)

(continued from previous page)

```
}
```

Output

```
lymph online seen_samples=1660
lymph online metrics [accuracy]: 0.9875
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:2436

OnlineLinearRegressor::trained

Signature

```
[[nodiscard]] bool OnlineLinearRegressor::trained(self) const noexcept;
```

Notes

Boolean state check for object readiness.

Parameters

None.

Returns

- Returns bool.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
OnlineLinearRegressor onlineLinearRegressor{};
auto out = onlineLinearRegressor.trained(/* args */);
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:2435

prep::SimpleImputer::learn

Signature

```
inline prep::SimpleImputer& prep::SimpleImputer::learn(self, const␣
↪pluss::table::DataFrame& frame, const TrainOptions& options);
```

Notes

Fits model/transform state from the provided training data. Call this before inference-oriented methods.

Parameters

- `frame` : `const pluss::table::DataFrame&` Input feature data used by this call.
- `options` : `const TrainOptions&` Configuration object controlling algorithm behavior and safety defaults.

Returns

- Returns `prep::SimpleImputer&`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1

Source: `PlussAI/examples/NodeJ/core/nodej_cv_tune_demo.cpp`

```
  std::cout << "regression quantile sweep objectives:";
  for (const auto& trial : reg_sweep_or.value().trials) std::cout << " " << trial.
↪objective;
  std::cout << "\n";
  std::cout << "regression quantile sweep best index: " << reg_sweep_or.value().best_
↪index << "\n";

  auto bounded_or = make_classifier(linear_logistic_spec());
  if (!require_ok(bounded_or, "make_classifier(linear bounded)")) return 1;
  auto bounded = bounded_or.move_value();

  TrainOptions bounded_train = train;
  bounded_train.parameter_lower_bounds = pluss::make_series(std::vector<double>{0.0}, {1}
↪, f64());
  bounded_train.parameter_upper_bounds = pluss::make_series(std::vector<double>{0.0}, {1}
↪, f64());
  if (!bounded->learn(batch, bounded_train).ok()) {
    std::cerr << "bounded linear learn failed\n";
    return 1;
  }

  auto bounded_scores_or = bounded->infer_scores(batch.features);
  if (!require_ok(bounded_scores_or, "infer_scores(bounded)")) return 1;
  std::cout << "max |bounded score|: " << bounded_scores_or.value().abs().max().item
↪<double>() << "\n";

  std::cout << "NodeJ CV + tune demo completed.\n";
  return 0;
}
```

Output

```
NodeJ CV + tune demo
linear mean metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0.25 0.25 0.
↪208333 0.25 nan 0.0625 0.960532
linear std metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0.176777 0.
↪176777 0.216506 0.25 nan 0.108253 0.192821
kernel mean metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 1 1 1 1 1 1␣
↪0.358895
kernel std metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0 0 0 0 0 0␣
↪0.0358183
sweep trial objectives: 0.25 1
sweep best index: 1
probe best index: 1
regression quantile sweep objectives: 0.0356052 0.736518
regression quantile sweep best index: 0
max |bounded score|: 0
NodeJ CV + tune demo completed.
```

Example 2

Source: PlussAI/examples/NodeJ/core/nodej_dataframe_bridge_demo.cpp

```
      .family = ModelFamily::Linear,
      .options = LinearFamilyOptions{
          .objective = LinearObjective::LeastSquares,
          .regularization = RegularizationKind::L2,
          .regularization_strength = 1e-8,
          .fit_intercept = true,
      },
  };

  auto reg_or = make_regressor(spec);
  if (!require_ok(reg_or, "make_regressor")) return 1;
  auto reg = reg_or.move_value();
  Status st = reg->learn(batch);
  if (!st.ok()) {
    std::cerr << "learn failed: " << st.message << "\n";
    return 1;
  }

  auto metrics_or = reg->assess(batch, AssessOptions{.metrics = {MetricKind::Mse,␣
↪MetricKind::R2}});
  if (!require_ok(metrics_or, "assess")) return 1;
  print_metrics("linear metrics [mse, r2]", metrics_or.value());

  std::cout << "NodeJ DataFrame bridge demo completed.\n";
  return 0;
}
```

Output

```
NodeJ DataFrame bridge demo
adapted batch: samples=6 features=4 weighted=yes
linear metrics [mse, r2]: 9.43684e-08 1
```

(continues on next page)

(continued from previous page)

```
NodeJ DataFrame bridge demo completed.
```

Example 3

Source: PlussAI/examples/NodeJ/core/nodej_flow_bundle_demo.cpp

```
  auto flow_or = make_flow(spec);
  if (!flow_or.ok()) {
    std::cerr << "make_flow failed: " << flow_or.status().message << "\n";
    return 1;
  }
  Flow flow = flow_or.move_value();
  if (!flow.add_standard_scaler().ok()) {
    std::cerr << "add_standard_scaler failed\n";
    return 1;
  }
  TrainOptions train{};
  train.max_iterations = 2500;
  train.tolerance = 1e-8;
  train.random.seed = 77;
  train.random.deterministic = true;
  if (!flow.learn(batch, train).ok()) {
    std::cerr << "flow learn failed\n";
    return 1;
  }
```

Output

```
saved flow bundle to: <repo_root>/PlussAI/examples/NodeJ/trained_models/nodej_flow_
↪bundle_demo.bin
flow bundle roundtrip exact prediction parity: true
nodej_flow_bundle_demo complete
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:1864

prep::SimpleImputer::learn_map

Signature

```
inline pluss::table::DataFrame prep::SimpleImputer::learn_map(self, const␣
↪pluss::table::DataFrame& frame, const TrainOptions& learn_options, const InferOptions&␣
↪infer_options);
```

Notes

Fits model/transform state from the provided training data. Call this before inference-oriented methods.

Parameters

- `frame` : `const pluss::table::DataFrame&` Input feature data used by this call.
- `infer_options` : `const InferOptions&` Configuration object controlling algorithm behavior and safety defaults.
- `learn_options` : `const TrainOptions&` Configuration object controlling algorithm behavior and safety defaults.

Returns

- Returns `pluss::table::DataFrame`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1

Source: `PlussAI/examples/NodeJ/core/nodej_prep_split_demo.cpp`

```
            << " test idx count=" << strat_or.value().test_indices.numel() << "\n";

  split.folds = 3;
  auto fold_or = pluss::ai::nodej::fold_plan(dense.sample_count(), split);
  if (!require_ok(fold_or, "fold_plan")) return 1;
  std::cout << "kfold folds=" << fold_or.value().folds.size() << "\n";

  SimpleImputer imputer(SimpleImputerOptions{
      .strategy = SimpleImputerStrategy::Median,
      .constant_value = 0.0,
      .empty_feature_fallback = 0.0,
  });
  auto imp_or = imputer.learn_map(dense);
  if (!require_ok(imp_or, "SimpleImputer.learn_map")) return 1;
  const Series x_imputed = imp_or.move_value();
  print_shape("imputed", x_imputed);

  StandardScaler zscore{};
  auto z_or = zscore.learn_map(FeatureBatch{.x = x_imputed, .layout =␣
↪ObservationLayout::Rows});
  if (!require_ok(z_or, "StandardScaler.learn_map")) return 1;
  const Series x_z = z_or.move_value();
  print_shape("standardized", x_z);

  MinMaxScaler minmax{};
  auto mm_or = minmax.learn_map(FeatureBatch{.x = x_imputed, .layout =␣
↪ObservationLayout::Rows});
  if (!require_ok(mm_or, "MinMaxScaler.learn_map")) return 1;
  const Series x_mm = mm_or.move_value();
  print_shape("minmax", x_mm);
```

(continues on next page)

(continued from previous page)

```
  RobustScaler robust{};
  auto rb_or = robust.learn_map(FeatureBatch{.x = x_imputed, .layout =␣
↪ObservationLayout::Rows});
  if (!require_ok(rb_or, "RobustScaler.learn_map")) return 1;
  const Series x_rb = rb_or.move_value();
```

Output

```
NodeJ split + prep demo
holdout train idx count=4 test idx count=2
strat holdout train idx count=4 test idx count=2
kfold folds=3
imputed shape=[6, 3]
standardized shape=[6, 3]
minmax shape=[6, 3]
robust shape=[6, 3]
neighbor-imputed shape=[6, 3]
iterative-imputed shape=[6, 3]
vector-normalized shape=[6, 3]
maxabs shape=[6, 3]
...
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:1887

prep::SimpleImputer::map

Signature

```
inline pluss::table::DataFrame prep::SimpleImputer::map(self, const␣
↪pluss::table::DataFrame& frame, const InferOptions& options) const;
```

Notes

Applies preprocessing/transformation behavior configured by the current options.

Parameters

- `frame` : `const pluss::table::DataFrame&` Input feature data used by this call.
- `options` : `const InferOptions&` Configuration object controlling algorithm behavior and safety defaults.

Returns

- Returns `pluss::table::DataFrame`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1

Source: PlussAI/examples/NodeJ/core/nodej_prep_split_demo.cpp

```
      .interactions_only = false,
      .include_bias = true,
  });
  auto poly_or = poly.learn_map(FeatureBatch{.x = x_imputed, .layout =␣
↪ObservationLayout::Rows});
  if (!require_ok(poly_or, "PolynomialLift.learn_map")) return 1;
  print_shape("polynomial-lift", poly_or.value());

  const Series x_query = pluss::make_series(std::vector<double>{
                                              7.0, 5.0, 1.0,
                                              8.0, 5.5, 0.0},
                                            {2, 3},
                                            f64());
  auto mapped_or = zscore.map(FeatureBatch{.x = x_query, .layout =␣
↪ObservationLayout::Rows});
  if (!require_ok(mapped_or, "StandardScaler.map")) return 1;
  print_shape("standardized query", mapped_or.value());

  const Series categories = pluss::make_series(std::vector<double>{
                                                 0.0, 1.0,
                                                 1.0, 1.0,
                                                 2.0, 0.0,
                                                 1.0, 2.0,
                                                 0.0, 2.0,
                                                 2.0, 1.0},
                                               {6, 2},
                                               f64());
  const FeatureBatch cat_batch{.x = categories, .layout = ObservationLayout::Rows};

  OneHotEncoder one_hot(OneHotEncoderOptions{
      .max_categories = 8,
      .overflow = CategoryOverflowPolicy::KeepMostFrequent,
      .include_unknown_bucket = true,
  });
  auto one_hot_or = one_hot.learn_map(cat_batch);
```

Output

```
NodeJ split + prep demo
holdout train idx count=4 test idx count=2
strat holdout train idx count=4 test idx count=2
kfold folds=3
imputed shape=[6, 3]
standardized shape=[6, 3]
minmax shape=[6, 3]
robust shape=[6, 3]
neighbor-imputed shape=[6, 3]
iterative-imputed shape=[6, 3]
```

(continues on next page)

(continued from previous page)

```
vector-normalized shape=[6, 3]
maxabs shape=[6, 3]
...
```

Example 2

Source: PlussAI/examples/NodeJ/core/nodej_unsupervised_demo.cpp

```
  ModelSpec kmeans_spec{
      .task = TaskKind::Transform,
      .family = ModelFamily::Cluster,
      .options = ClusterFamilyOptions{
          .method = ClusterMethod::KMeans,
          .clusters = 2,
          .max_iterations = 200,
          .restarts = 5,
          .tolerance = 1e-5,
      },
  };
  auto kmeans_or = make_transformer(kmeans_spec);
  if (!kmeans_or.ok()) {
    std::cerr << "make_transformer(kmeans) failed: " << kmeans_or.status().message << "\n
↪";
    return 1;
  }
  auto kmeans = kmeans_or.move_value();
  if (!kmeans->learn(features).ok()) {
    std::cerr << "kmeans learn failed\n";
    return 1;
  }
  auto labels_or = kmeans->map(features);
  if (!labels_or.ok()) {
    std::cerr << "kmeans map failed: " << labels_or.status().message << "\n";
    return 1;
  }
  const auto labels = labels_or.value().reshape({60}).to(pluss::pCPU,␣
↪pluss::ScalarType::Float64).to_vec<double>();
  std::cout << "kmeans labels (first 10): ";
  for (int i = 0; i < 10; ++i) std::cout << labels[static_cast<std::size_t>(i)] << " ";
  std::cout << "\n";
```

Output

```
kmeans labels (first 10): 0 0 0 0 0 0 0 0 0 0
pca output shape: [80, 2]
nodej_unsupervised_demo complete
```

Example 3

Source: PlussAI/examples/NodeJ/lymph/lymph_unsupervised_demo.cpp

```
  auto kmeans_or = pluss::ai::lymph::quick::kmeans_clusterer(batch, 2);
  if (!kmeans_or.ok()) {
    std::cerr << "lymph::kmeans_clusterer failed: " << kmeans_or.status().message << "\n
↪";
    return 1;
  }
  auto labels_or = kmeans_or.value()->map(batch);
  if (!labels_or.ok()) {
    std::cerr << "kmeans map failed: " << labels_or.status().message << "\n";
    return 1;
  }
  std::cout << "lymph kmeans labels: ";
  for (double v : labels_or.value().reshape({6}).to(pluss::pCPU,␣
↪pluss::ScalarType::Float64).to_vec<double>()) {
    std::cout << v << " ";
  }
  std::cout << "\n";
```

Output

```
lymph kmeans labels: 1 1 1 0 0 0
lymph pca output shape: [40, 2]
lymph_unsupervised_demo complete
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:1876

prep::StandardScaler::learn

Signature

```
inline prep::StandardScaler& prep::StandardScaler::learn(self, const␣
↪pluss::table::DataFrame& frame, const TrainOptions& options);
```

Notes

Fits model/transform state from the provided training data. Call this before inference-oriented methods.

Parameters

- `frame` : `const pluss::table::DataFrame&` Input feature data used by this call.
- `options` : `const TrainOptions&` Configuration object controlling algorithm behavior and safety defaults.

Returns

- Returns `prep::StandardScaler&`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1

Source: PlussAI/examples/NodeJ/core/nodej_cv_tune_demo.cpp

```
  std::cout << "regression quantile sweep objectives:";
  for (const auto& trial : reg_sweep_or.value().trials) std::cout << " " << trial.
↪objective;
  std::cout << "\n";
  std::cout << "regression quantile sweep best index: " << reg_sweep_or.value().best_
↪index << "\n";

  auto bounded_or = make_classifier(linear_logistic_spec());
  if (!require_ok(bounded_or, "make_classifier(linear bounded)")) return 1;
  auto bounded = bounded_or.move_value();

  TrainOptions bounded_train = train;
  bounded_train.parameter_lower_bounds = pluss::make_series(std::vector<double>{0.0}, {1}
↪, f64());
  bounded_train.parameter_upper_bounds = pluss::make_series(std::vector<double>{0.0}, {1}
↪, f64());
  if (!bounded->learn(batch, bounded_train).ok()) {
    std::cerr << "bounded linear learn failed\n";
    return 1;
  }

  auto bounded_scores_or = bounded->infer_scores(batch.features);
  if (!require_ok(bounded_scores_or, "infer_scores(bounded)")) return 1;
  std::cout << "max |bounded score|: " << bounded_scores_or.value().abs().max().item
↪<double>() << "\n";

  std::cout << "NodeJ CV + tune demo completed.\n";
  return 0;
}
```

Output

```
NodeJ CV + tune demo
linear mean metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0.25 0.25 0.
↪208333 0.25 nan 0.0625 0.960532
linear std metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0.176777 0.
↪176777 0.216506 0.25 nan 0.108253 0.192821
kernel mean metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 1 1 1 1 1 1␣
↪0.358895
kernel std metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0 0 0 0 0 0␣
↪0.0358183
sweep trial objectives: 0.25 1
sweep best index: 1
probe best index: 1
```

(continues on next page)

(continued from previous page)

```
regression quantile sweep objectives: 0.0356052 0.736518
regression quantile sweep best index: 0
max |bounded score|: 0
NodeJ CV + tune demo completed.
```

Example 2

Source: PlussAI/examples/NodeJ/core/nodej_dataframe_bridge_demo.cpp

```
      .family = ModelFamily::Linear,
      .options = LinearFamilyOptions{
          .objective = LinearObjective::LeastSquares,
          .regularization = RegularizationKind::L2,
          .regularization_strength = 1e-8,
          .fit_intercept = true,
      },
  };

  auto reg_or = make_regressor(spec);
  if (!require_ok(reg_or, "make_regressor")) return 1;
  auto reg = reg_or.move_value();
  Status st = reg->learn(batch);
  if (!st.ok()) {
    std::cerr << "learn failed: " << st.message << "\n";
    return 1;
  }

  auto metrics_or = reg->assess(batch, AssessOptions{.metrics = {MetricKind::Mse,␣
↪MetricKind::R2}});
  if (!require_ok(metrics_or, "assess")) return 1;
  print_metrics("linear metrics [mse, r2]", metrics_or.value());

  std::cout << "NodeJ DataFrame bridge demo completed.\n";
  return 0;
}
```

Output

```
NodeJ DataFrame bridge demo
adapted batch: samples=6 features=4 weighted=yes
linear metrics [mse, r2]: 9.43684e-08 1
NodeJ DataFrame bridge demo completed.
```

Example 3

Source: PlussAI/examples/NodeJ/core/nodej_flow_bundle_demo.cpp

```
  auto flow_or = make_flow(spec);
  if (!flow_or.ok()) {
    std::cerr << "make_flow failed: " << flow_or.status().message << "\n";
    return 1;
  }
```

(continues on next page)

(continued from previous page)

```
  Flow flow = flow_or.move_value();
  if (!flow.add_standard_scaler().ok()) {
    std::cerr << "add_standard_scaler failed\n";
    return 1;
  }
  TrainOptions train{};
  train.max_iterations = 2500;
  train.tolerance = 1e-8;
  train.random.seed = 77;
  train.random.deterministic = true;
  if (!flow.learn(batch, train).ok()) {
    std::cerr << "flow learn failed\n";
    return 1;
  }
```

Output

```
saved flow bundle to: <repo_root>/PlussAI/examples/NodeJ/trained_models/nodej_flow_
↪bundle_demo.bin
flow bundle roundtrip exact prediction parity: true
nodej_flow_bundle_demo complete
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:1901

prep::StandardScaler::learn_map

Signature

```
inline pluss::table::DataFrame prep::StandardScaler::learn_map(self, const␣
↪pluss::table::DataFrame& frame, const TrainOptions& learn_options, const InferOptions&␣
↪infer_options);
```

Notes

Fits model/transform state from the provided training data. Call this before inference-oriented methods.

Parameters

- `frame` : `const pluss::table::DataFrame&` Input feature data used by this call.
- `infer_options` : `const InferOptions&` Configuration object controlling algorithm behavior and safety defaults.
- `learn_options` : `const TrainOptions&` Configuration object controlling algorithm behavior and safety defaults.

Returns

- Returns `pluss::table::DataFrame`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1

Source: PlussAI/examples/NodeJ/core/nodej_prep_split_demo.cpp

```
            << " test idx count=" << strat_or.value().test_indices.numel() << "\n";

  split.folds = 3;
  auto fold_or = pluss::ai::nodej::fold_plan(dense.sample_count(), split);
  if (!require_ok(fold_or, "fold_plan")) return 1;
  std::cout << "kfold folds=" << fold_or.value().folds.size() << "\n";

  SimpleImputer imputer(SimpleImputerOptions{
      .strategy = SimpleImputerStrategy::Median,
      .constant_value = 0.0,
      .empty_feature_fallback = 0.0,
  });
  auto imp_or = imputer.learn_map(dense);
  if (!require_ok(imp_or, "SimpleImputer.learn_map")) return 1;
  const Series x_imputed = imp_or.move_value();
  print_shape("imputed", x_imputed);

  StandardScaler zscore{};
  auto z_or = zscore.learn_map(FeatureBatch{.x = x_imputed, .layout =␣
↪ObservationLayout::Rows});
  if (!require_ok(z_or, "StandardScaler.learn_map")) return 1;
  const Series x_z = z_or.move_value();
  print_shape("standardized", x_z);

  MinMaxScaler minmax{};
  auto mm_or = minmax.learn_map(FeatureBatch{.x = x_imputed, .layout =␣
↪ObservationLayout::Rows});
  if (!require_ok(mm_or, "MinMaxScaler.learn_map")) return 1;
  const Series x_mm = mm_or.move_value();
  print_shape("minmax", x_mm);

  RobustScaler robust{};
  auto rb_or = robust.learn_map(FeatureBatch{.x = x_imputed, .layout =␣
↪ObservationLayout::Rows});
  if (!require_ok(rb_or, "RobustScaler.learn_map")) return 1;
  const Series x_rb = rb_or.move_value();
```

Output

```
NodeJ split + prep demo
holdout train idx count=4 test idx count=2
strat holdout train idx count=4 test idx count=2
kfold folds=3
imputed shape=[6, 3]
```

(continues on next page)

(continued from previous page)

```
standardized shape=[6, 3]
minmax shape=[6, 3]
robust shape=[6, 3]
neighbor-imputed shape=[6, 3]
iterative-imputed shape=[6, 3]
vector-normalized shape=[6, 3]
maxabs shape=[6, 3]
...
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:1924

prep::StandardScaler::map

Signature

```
inline pluss::table::DataFrame prep::StandardScaler::map(self, const␣
↪pluss::table::DataFrame& frame, const InferOptions& options) const;
```

Notes

Applies preprocessing/transformation behavior configured by the current options.

Parameters

- `frame` : `const pluss::table::DataFrame&` Input feature data used by this call.
- `options` : `const InferOptions&` Configuration object controlling algorithm behavior and safety defaults.

Returns

- Returns `pluss::table::DataFrame`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1

Source: `PlussAI/examples/NodeJ/core/nodej_prep_split_demo.cpp`

```
      .interactions_only = false,
      .include_bias = true,
  });
  auto poly_or = poly.learn_map(FeatureBatch{.x = x_imputed, .layout =␣
↪ObservationLayout::Rows});
  if (!require_ok(poly_or, "PolynomialLift.learn_map")) return 1;
  print_shape("polynomial-lift", poly_or.value());
```

(continues on next page)

(continued from previous page)

```
  const Series x_query = pluss::make_series(std::vector<double>{
                                                7.0, 5.0, 1.0,
                                                8.0, 5.5, 0.0},
                                            {2, 3},
                                            f64());
  auto mapped_or = zscore.map(FeatureBatch{.x = x_query, .layout =␣
↪ObservationLayout::Rows});
  if (!require_ok(mapped_or, "StandardScaler.map")) return 1;
  print_shape("standardized query", mapped_or.value());

  const Series categories = pluss::make_series(std::vector<double>{
                                                    0.0, 1.0,
                                                    1.0, 1.0,
                                                    2.0, 0.0,
                                                    1.0, 2.0,
                                                    0.0, 2.0,
                                                    2.0, 1.0},
                                                {6, 2},
                                                f64());
  const FeatureBatch cat_batch{.x = categories, .layout = ObservationLayout::Rows};

  OneHotEncoder one_hot(OneHotEncoderOptions{
      .max_categories = 8,
      .overflow = CategoryOverflowPolicy::KeepMostFrequent,
      .include_unknown_bucket = true,
  });
  auto one_hot_or = one_hot.learn_map(cat_batch);
```

Output

```
NodeJ split + prep demo
holdout train idx count=4 test idx count=2
strat holdout train idx count=4 test idx count=2
kfold folds=3
imputed shape=[6, 3]
standardized shape=[6, 3]
minmax shape=[6, 3]
robust shape=[6, 3]
neighbor-imputed shape=[6, 3]
iterative-imputed shape=[6, 3]
vector-normalized shape=[6, 3]
maxabs shape=[6, 3]
...
```

Example 2

Source: PlussAI/examples/NodeJ/core/nodej_unsupervised_demo.cpp

```
  ModelSpec kmeans_spec{
      .task = TaskKind::Transform,
      .family = ModelFamily::Cluster,
      .options = ClusterFamilyOptions{
```

(continues on next page)

(continued from previous page)

```
          .method = ClusterMethod::KMeans,
          .clusters = 2,
          .max_iterations = 200,
          .restarts = 5,
          .tolerance = 1e-5,
      },
  };
  auto kmeans_or = make_transformer(kmeans_spec);
  if (!kmeans_or.ok()) {
    std::cerr << "make_transformer(kmeans) failed: " << kmeans_or.status().message << "\n
↪";
    return 1;
  }
  auto kmeans = kmeans_or.move_value();
  if (!kmeans->learn(features).ok()) {
    std::cerr << "kmeans learn failed\n";
    return 1;
  }
  auto labels_or = kmeans->map(features);
  if (!labels_or.ok()) {
    std::cerr << "kmeans map failed: " << labels_or.status().message << "\n";
    return 1;
  }
  const auto labels = labels_or.value().reshape({60}).to(pluss::pCPU,␣
↪pluss::ScalarType::Float64).to_vec<double>();
  std::cout << "kmeans labels (first 10): ";
  for (int i = 0; i < 10; ++i) std::cout << labels[static_cast<std::size_t>(i)] << " ";
  std::cout << "\n";
```

Output

```
kmeans labels (first 10): 0 0 0 0 0 0 0 0 0 0
pca output shape: [80, 2]
nodej_unsupervised_demo complete
```

Example 3

Source: PlussAI/examples/NodeJ/lymph/lymph_unsupervised_demo.cpp

```
  auto kmeans_or = pluss::ai::lymph::quick::kmeans_clusterer(batch, 2);
  if (!kmeans_or.ok()) {
    std::cerr << "lymph::kmeans_clusterer failed: " << kmeans_or.status().message << "\n
↪";
    return 1;
  }
  auto labels_or = kmeans_or.value()->map(batch);
  if (!labels_or.ok()) {
    std::cerr << "kmeans map failed: " << labels_or.status().message << "\n";
    return 1;
  }
  std::cout << "lymph kmeans labels: ";
  for (double v : labels_or.value().reshape({6}).to(pluss::pCPU,␣
```

(continues on next page)

(continued from previous page)

```
↪pluss::ScalarType::Float64).to_vec<double>()) {
    std::cout << v << " ";
  }
  std::cout << "\n";
```

Output

```
lymph kmeans labels: 1 1 1 0 0 0
lymph pca output shape: [40, 2]
lymph_unsupervised_demo complete
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:1913

SimpleImputer::features

Signature

```
SimpleImputer& SimpleImputer::features(self, std::initializer_list<std::string_view>␣
↪names);
```

Notes

Instance method on `self`.

Parameters

- `names` : `std::initializer_list<std::string_view>` List-like argument; pass values in the order expected by the signature.

Returns

- Returns `SimpleImputer&`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1

Source: `PlussAI/examples/NodeJ/core/bike_sharing_end_to_end.cpp`

```
    const auto split = lymph::holdout_split(frame, lymph::HoldoutSplitOptions{
                                                       .test_size = 0.2,
                                                       .random_state = 42,
                                                       .deterministic = true,
                                                       .shuffle = true,
                                                   });
```

(continues on next page)

(continued from previous page)

```
    const DataFrame train_x = split.train_set.features(
        {"season", "year", "month", "hour", "is_holiday", "weekday", "is_workingday",
↪"weather_condition", "temp", "atemp", "humidity", "windspeed"});
    const DataFrame test_x = split.test_set.features(
        {"season", "year", "month", "hour", "is_holiday", "weekday", "is_workingday",
↪"weather_condition", "temp", "atemp", "humidity", "windspeed"});
    const Series train_y = split.train_set.features("total_count").as_series();
    const Series test_y = split.test_set.features("total_count").as_series();
```

Example 2

Source: PlussAI/examples/NodeJ/core/nodej_end_to_end_hour_workflow_showcase.cpp

```
  frame = frame.cast_timeline({"date"});
  frame = frame.cast_series(
      {
          "season", "year", "month", "hour", "is_holiday", "weekday", "is_workingday",
          "weather_condition", "temp", "atemp", "humidity", "windspeed", "casual",
↪"registered",
          "total_count",
      },
      ScalarType::Float64);

  //========== feature engineering ========
  // This section creates interaction and nonlinear terms directly on DataFrame columns.
  const Series s_temp = frame.features("temp").as_series();
  const Series s_atemp = frame.features("atemp").as_series();
  const Series s_humidity = frame.features("humidity").as_series();
  const Series s_windspeed = frame.features("windspeed").as_series();
  const Series s_hour = frame.features("hour").as_series();
  const Series s_month = frame.features("month").as_series();
  const Series s_weather = frame.features("weather_condition").as_series();

  frame = frame.set_features("temp_sq", Thread(s_temp.square()));
  frame = frame.set_features("atemp_sq", Thread(s_atemp.square()));
  frame = frame.set_features("humidity_sq", Thread(s_humidity.square()));
  frame = frame.set_features("windspeed_sq", Thread(s_windspeed.square()));
  frame = frame.set_features("temp_humidity_interaction", Thread(s_temp.mul(s_
↪humidity)));
  frame = frame.set_features("temp_windspeed_interaction", Thread(s_temp.mul(s_
↪windspeed)));
  frame = frame.set_features("atemp_humidity_interaction", Thread(s_atemp.mul(s_
↪humidity)));
  frame = frame.set_features("atemp_windspeed_interaction", Thread(s_atemp.mul(s_
↪windspeed)));
  frame = frame.set_features("temp_minus_atemp", Thread(s_temp.sub(s_atemp)));
  frame = frame.set_features("hour_centered", Thread(s_hour.sub(12.0)));
  frame = frame.set_features("hour_centered_sq", Thread(s_hour.sub(12.0).square()));
  frame = frame.set_features("month_centered", Thread(s_month.sub(6.5)));
  frame = frame.set_features("month_centered_sq", Thread(s_month.sub(6.5).square()));
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:82

SimpleImputer::learned

Signature

```
[[nodiscard]] bool SimpleImputer::learned(self) const noexcept;
```

Notes

Fits model/transform state from the provided training data. Call this before inference-oriented methods.

Parameters

None.

Returns

- Returns bool.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
SimpleImputer simpleImputer{};
auto out = simpleImputer.learned(/* args */);
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:106

SimpleImputer::summary

Signature

```
[[nodiscard]] SimpleImputerSummary SimpleImputer::summary(self) const;
```

Notes

Instance method on self.

Parameters

None.

Returns

- Returns `SimpleImputerSummary`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1

Source: `PlussAI/examples/NodeJ/lymph/lymph_end_to_end_hour_workflow_showcase.cpp`

```
});
const Series train_y = holdout.train_set.features("total_count").as_series();
const Series test_y = holdout.test_set.features("total_count").as_series();

//========== standalone lymph preprocessing API ========
// This section demonstrates direct prep estimators (learn/map/learn_map)
// without going through Workflow composition.
auto simple_imp = lymph::prep::simple_imputer(lymph::prep::SimpleImputerOptions{
    .strategy = lymph::prep::SimpleImputerStrategy::Median,
});
const DataFrame train_simple_imputed = simple_imp.learn_map(train_x);
const DataFrame test_simple_imputed = simple_imp.map(test_x);
const auto simple_imp_summary = simple_imp.summary();

auto knn_imp = lymph::prep::neighbor_imputer(lymph::prep::NeighborImputerOptions{
    .neighbors = 7,
    .weighting = lymph::prep::NeighborImputerWeighting::Distance,
    .distance_floor = 1e-12,
    .empty_feature_fallback = 0.0,
});
const DataFrame train_knn_imputed = knn_imp.learn_map(train_x);
const DataFrame test_knn_imputed = knn_imp.map(test_x);
const auto knn_imp_summary = knn_imp.summary();

auto iter_imp = lymph::prep::iterative_imputer(lymph::prep::IterativeImputerOptions{
    .rounds = 6,
    .ridge = 1e-5,
    .empty_feature_fallback = 0.0,
});
const DataFrame train_iter_imputed = iter_imp.learn_map(train_x);
const DataFrame test_iter_imputed = iter_imp.map(test_x);
const auto iter_imp_summary = iter_imp.summary();
```

Source

- `PlussAI/include/pluss/ai/lymph/api.h:99`

StandardScaler::features

Signature

```
StandardScaler& StandardScaler::features(self, std::initializer_list<std::string_view>␣
↪names);
```

Notes

Instance method on `self`.

Parameters

- `names` : `std::initializer_list<std::string_view>` List-like argument; pass values in the order expected by the signature.

Returns

- Returns `StandardScaler&`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1

Source: `PlussAI/examples/NodeJ/core/bike_sharing_end_to_end.cpp`

```
    const auto split = lymph::holdout_split(frame, lymph::HoldoutSplitOptions{
                                                       .test_size = 0.2,
                                                       .random_state = 42,
                                                       .deterministic = true,
                                                       .shuffle = true,
                                                   });
    const DataFrame train_x = split.train_set.features(
        {"season", "year", "month", "hour", "is_holiday", "weekday", "is_workingday",
↪"weather_condition", "temp", "atemp", "humidity", "windspeed"});
    const DataFrame test_x = split.test_set.features(
        {"season", "year", "month", "hour", "is_holiday", "weekday", "is_workingday",
↪"weather_condition", "temp", "atemp", "humidity", "windspeed"});
    const Series train_y = split.train_set.features("total_count").as_series();
    const Series test_y = split.test_set.features("total_count").as_series();
```

Example 2

Source: `PlussAI/examples/NodeJ/core/nodej_end_to_end_hour_workflow_showcase.cpp`

```
  frame = frame.cast_timeline({"date"});
  frame = frame.cast_series(
      {
          "season", "year", "month", "hour", "is_holiday", "weekday", "is_workingday",
```

(continues on next page)

(continued from previous page)

```
          "weather_condition", "temp", "atemp", "humidity", "windspeed", "casual",
↪"registered",
          "total_count",
      },
      ScalarType::Float64);

  //========== feature engineering ========
  // This section creates interaction and nonlinear terms directly on DataFrame columns.
  const Series s_temp = frame.features("temp").as_series();
  const Series s_atemp = frame.features("atemp").as_series();
  const Series s_humidity = frame.features("humidity").as_series();
  const Series s_windspeed = frame.features("windspeed").as_series();
  const Series s_hour = frame.features("hour").as_series();
  const Series s_month = frame.features("month").as_series();
  const Series s_weather = frame.features("weather_condition").as_series();

  frame = frame.set_features("temp_sq", Thread(s_temp.square()));
  frame = frame.set_features("atemp_sq", Thread(s_atemp.square()));
  frame = frame.set_features("humidity_sq", Thread(s_humidity.square()));
  frame = frame.set_features("windspeed_sq", Thread(s_windspeed.square()));
  frame = frame.set_features("temp_humidity_interaction", Thread(s_temp.mul(s_
↪humidity)));
  frame = frame.set_features("temp_windspeed_interaction", Thread(s_temp.mul(s_
↪windspeed)));
  frame = frame.set_features("atemp_humidity_interaction", Thread(s_atemp.mul(s_
↪humidity)));
  frame = frame.set_features("atemp_windspeed_interaction", Thread(s_atemp.mul(s_
↪windspeed)));
  frame = frame.set_features("temp_minus_atemp", Thread(s_temp.sub(s_atemp)));
  frame = frame.set_features("hour_centered", Thread(s_hour.sub(12.0)));
  frame = frame.set_features("hour_centered_sq", Thread(s_hour.sub(12.0).square()));
  frame = frame.set_features("month_centered", Thread(s_month.sub(6.5)));
  frame = frame.set_features("month_centered_sq", Thread(s_month.sub(6.5).square()));
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:129

StandardScaler::learned

Signature

```
[[nodiscard]] bool StandardScaler::learned(self) const noexcept;
```

Notes

Fits model/transform state from the provided training data. Call this before inference-oriented methods.

Parameters

None.

Returns

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
StandardScaler standardScaler{};
auto out = standardScaler.learned(/* args */);
```

Source

- `PlussAI/include/pluss/ai/lymph/api.h:154`

`StandardScaler::summary`

Signature

```
[[nodiscard]] StandardScalerSummary StandardScaler::summary(self) const;
```

Notes

Instance method on `self`.

Parameters

None.

Returns

- Returns `StandardScalerSummary`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1

Source: `PlussAI/examples/NodeJ/lymph/lymph_end_to_end_hour_workflow_showcase.cpp`

```
    });
    const Series train_y = holdout.train_set.features("total_count").as_series();
    const Series test_y = holdout.test_set.features("total_count").as_series();

    //========== standalone lymph preprocessing API ========
    // This section demonstrates direct prep estimators (learn/map/learn_map)
    // without going through Workflow composition.
    auto simple_imp = lymph::prep::simple_imputer(lymph::prep::SimpleImputerOptions{
        .strategy = lymph::prep::SimpleImputerStrategy::Median,
    });
    const DataFrame train_simple_imputed = simple_imp.learn_map(train_x);
    const DataFrame test_simple_imputed = simple_imp.map(test_x);
    const auto simple_imp_summary = simple_imp.summary();

    auto knn_imp = lymph::prep::neighbor_imputer(lymph::prep::NeighborImputerOptions{
        .neighbors = 7,
        .weighting = lymph::prep::NeighborImputerWeighting::Distance,
        .distance_floor = 1e-12,
        .empty_feature_fallback = 0.0,
    });
    const DataFrame train_knn_imputed = knn_imp.learn_map(train_x);
    const DataFrame test_knn_imputed = knn_imp.map(test_x);
    const auto knn_imp_summary = knn_imp.summary();

    auto iter_imp = lymph::prep::iterative_imputer(lymph::prep::IterativeImputerOptions{
        .rounds = 6,
        .ridge = 1e-5,
        .empty_feature_fallback = 0.0,
    });
    const DataFrame train_iter_imputed = iter_imp.learn_map(train_x);
    const DataFrame test_iter_imputed = iter_imp.map(test_x);
    const auto iter_imp_summary = iter_imp.summary();
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:146

Workflow::coeff

Signature

```
[[nodiscard]] Series Workflow::coeff(self) const;
```

Notes

Instance method on `self`.

Parameters

None.

Returns

- Returns `Series`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1

Source: `PlussAI/examples/NodeJ/core/bike_sharing_end_to_end.cpp`

```
    auto linear_workflow = lymph::Workflow(lymph::linear_factory::linear_
↪regression(LinearFamilyOptions{
        .objective = LinearObjective::LeastSquares,
        .regularization = RegularizationKind::None,
        .regularization_strength = 0.0,
        .fit_intercept = true,
    }));
    linear_workflow
        .simple_imputer(lymph::prep::SimpleImputerOptions{.strategy =␣
↪lymph::prep::SimpleImputerStrategy::Median})
        .standard_scaler();
    linear_workflow.learn_(train_x, train_y, train_options);
    const auto linear_metrics = linear_workflow.assess(test_x, test_y, assess_options);
    std::cout << "linear test metrics [rmse, r2, mae]: " << linear_metrics.rmse << " " <
↪< linear_metrics.r2 << " "
              << linear_metrics.mae << "\n";
    std::cout << "linear params: intercept=" << linear_workflow.intercept() << " coeff="
↪<< linear_workflow.coeff() << "\n";
```

Source

- `PlussAI/include/pluss/ai/lymph/api.h:1271`

`Workflow::dump_model_state`

Signature

```
[[nodiscard]] std::vector<Series> Workflow::dump_model_state(self) const;
```

Notes

Instance method on `self`.

Parameters

None.

Returns

- Returns `std::vector<Series>`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
Workflow workflow{};
auto out = workflow.dump_model_state(/* args */);
```

Source

- `PlussAI/include/pluss/ai/lymph/api.h:1579`

`Workflow::dump_transform_states`

Signature

```
[[nodiscard]] std::vector<std::vector<Series>> Workflow::dump_transform_states(self)␣
↪const;
```

Notes

Instance method on `self`.

Parameters

None.

Returns

- Returns `std::vector<std::vector<Series>>`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
Workflow workflow{};
auto out = workflow.dump_transform_states(/* args */);
```

Source

- `PlussAI/include/pluss/ai/lymph/api.h:1574`

Workflow::has_model

Signature

```
[[nodiscard]] bool Workflow::has_model(self) const noexcept;
```

Notes

Boolean introspection helper that reports whether a capability/state is available.

Parameters

None.

Returns

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
Workflow workflow{};
auto out = workflow.has_model(/* args */);
```

Source

- `PlussAI/include/pluss/ai/lymph/api.h:1234`

Workflow::has_model_spec

Signature

```
[[nodiscard]] bool Workflow::has_model_spec(self) const;
```

Notes

Boolean introspection helper that reports whether a capability/state is available.

Parameters

None.

Returns

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
Workflow workflow{};
auto out = workflow.has_model_spec(/* args */);
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:1238

Workflow::has_opaque_transforms

Signature

```
[[nodiscard]] bool Workflow::has_opaque_transforms(self) const;
```

Notes

Boolean introspection helper that reports whether a capability/state is available.

Parameters

None.

Returns

- Returns bool.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
Workflow workflow{};
auto out = workflow.has_opaque_transforms(/* args */);
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:1239

Workflow::intercept

Signature

```
[[nodiscard]] Series Workflow::intercept(self) const;
```

Notes

Instance method on `self`.

Parameters

None.

Returns

- Returns `Series`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1

Source: `PlussAI/examples/NodeJ/core/bike_sharing_end_to_end.cpp`

```
    auto linear_workflow = lymph::Workflow(lymph::linear_factory::linear_
↪regression(LinearFamilyOptions{
        .objective = LinearObjective::LeastSquares,
        .regularization = RegularizationKind::None,
        .regularization_strength = 0.0,
        .fit_intercept = true,
    }));
    linear_workflow
        .simple_imputer(lymph::prep::SimpleImputerOptions{.strategy =␣
↪lymph::prep::SimpleImputerStrategy::Median})
        .standard_scaler();
    linear_workflow.learn_(train_x, train_y, train_options);
    const auto linear_metrics = linear_workflow.assess(test_x, test_y, assess_options);
    std::cout << "linear test metrics [rmse, r2, mae]: " << linear_metrics.rmse << " " <
↪< linear_metrics.r2 << " "
              << linear_metrics.mae << "\n";
    std::cout << "linear params: intercept=" << linear_workflow.intercept() << " coeff="
↪<< linear_workflow.coeff() << "\n";
```

Source

- `PlussAI/include/pluss/ai/lymph/api.h:1276`

`Workflow::load`

Signature

```
[[nodiscard]] static Workflow Workflow::load(self, const std::string& path);
```

Notes

Loads a previously persisted artifact and restores its inference-ready state.

Parameters

- `path` : `const std::string&` Path to persisted model/data artifact.

Returns

- Returns `Workflow`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1

Source: `PlussAI/examples/NodeJ/core/bike_sharing_end_to_end.cpp`

```
    linear_workflow
        .simple_imputer(lymph::prep::SimpleImputerOptions{.strategy =␣
↪lymph::prep::SimpleImputerStrategy::Median})
        .standard_scaler();
    linear_workflow.learn_(train_x, train_y, train_options);
    const auto linear_metrics = linear_workflow.assess(test_x, test_y, assess_options);
    std::cout << "linear test metrics [rmse, r2, mae]: " << linear_metrics.rmse << " " <
↪< linear_metrics.r2 << " "
              << linear_metrics.mae << "\n";
    std::cout << "linear params: intercept=" << linear_workflow.intercept() << " coeff="
↪<< linear_workflow.coeff() << "\n";

    const Series linear_pred_series = linear_workflow.infer(test_x);
    const fs::path linear_model_path = model_dir / "bike_sharing_linear.bin";
    linear_workflow.save(linear_model_path.string(), train_x, train_y, train_options);
    const auto linear_restored = lymph::Workflow::load(linear_model_path.string());
    const Series linear_pred_restored_series = linear_restored.infer(test_x);
    const Series y_true = test_y.to(pluss::pCPU, ScalarType::Float64).reshape({test_y.
↪numel()});

    const Series linear_pred = linear_pred_series.to(pluss::pCPU, ScalarType::Float64).
↪reshape({linear_pred_series.numel()});
    const double linear_max_abs_diff = lymph::max_abs_diff(linear_pred_series, linear_
↪pred_restored_series);
    std::cout << "linear restored prediction max_abs_diff: " << linear_max_abs_diff << "\
↪n";

    DataFrame linear_frame({
        {"y_true", Thread(y_true)},
        {"y_pred", Thread(linear_pred)},
        {"residual", Thread(y_true.sub(linear_pred))},
    });
```

(continues on next page)

(continued from previous page)

```
    linear_frame.save_csv((out_dir / "bike_sharing_test_predictions_linear.csv").
↪string());
    linear_frame.plot()
        .scatter("y_true", "y_pred")
        .headline("Observed vs predicted bike demand (linear)")
        .save_pdf((out_dir / "bike_pred_vs_true_linear.pdf").string());
    linear_frame.plot()
        .hist("residual", 80)
        .headline("Residual distribution (linear)")
```

Example 2

Source: PlussAI/examples/NodeJ/core/nodej_workflow_tuning_demo.cpp

```
                << " cv_mae=" << trial.cv.mean.mae << "\n";
    }
    std::cout << "best candidate: " << tuned.report.best_name << "\n";

    auto best = std::move(tuned.best_workflow);
    best.learn_(train_x, train_y, train_options);
    const auto test_metrics = best.assess(test_x, test_y, assess_options);
    std::cout << "test metrics [rmse, r2, mae]: " << test_metrics.rmse << " " << test_
↪metrics.r2 << " " << test_metrics.mae << "\n";

    const Series pred = best.infer(test_x);
    const fs::path best_model_path = model_dir / "hour_best_workflow.bin";
    best.save(best_model_path.string(), train_x, train_y, train_options);
    auto restored = nja::Workflow::load(best_model_path.string());
    const Series pred_restored = restored.infer(test_x);
    const double restored_max_abs_diff = nja::max_abs_diff(pred, pred_restored);
    std::cout << "restored prediction max_abs_diff: " << restored_max_abs_diff << "\n";

    const Series y_true = test_y.to(pluss::pCPU, ScalarType::Float64).reshape({test_y.
↪numel()});
    const Series y_pred = pred.to(pluss::pCPU, ScalarType::Float64).reshape({pred.
↪numel()});
    DataFrame pred_frame({
        {"y_true", Thread(y_true)},
        {"y_pred", Thread(y_pred)},
        {"residual", Thread(y_true.sub(y_pred))},
    });
    pred_frame.save_csv((out_dir / "hour_workflow_tuning_predictions.csv").string());
    pred_frame.plot()
        .scatter("y_true", "y_pred")
        .headline("Observed vs predicted bike demand (best workflow)")
        .save_pdf((out_dir / "hour_workflow_tuning_pred_vs_true.pdf").string());

    std::cout << "saved best model: " << best_model_path << "\n";
    std::cout << "saved outputs under: " << out_dir << "\n";
    return 0;
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:1628

Workflow::model

Signature

```
[[nodiscard]] const WorkflowModel& Workflow::model(self) const;
```

```
[[nodiscard]] WorkflowModel& Workflow::model(self);
```

Notes

Instance method on `self`.

Parameters

None.

Returns

- Returns `WorkflowModel&`.
- Returns `const WorkflowModel&`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1

Source: `PlussAI/examples/NodeJ/core/bike_sharing_end_to_end.cpp`

```
    DataFrame linear_frame({
        {"y_true", Thread(y_true)},
        {"y_pred", Thread(linear_pred)},
        {"residual", Thread(y_true.sub(linear_pred))},
    });
    linear_frame.save_csv((out_dir / "bike_sharing_test_predictions_linear.csv").
↪string());
    linear_frame.plot()
        .scatter("y_true", "y_pred")
        .headline("Observed vs predicted bike demand (linear)")
        .save_pdf((out_dir / "bike_pred_vs_true_linear.pdf").string());
    linear_frame.plot()
        .hist("residual", 80)
        .headline("Residual distribution (linear)")
        .save_pdf((out_dir / "bike_residual_hist_linear.pdf").string());
    std::cout << "saved model (linear): " << linear_model_path << "\n";
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:1261
- PlussAI/include/pluss/ai/lymph/api.h:1266

Workflow::model_spec

Signature

```
[[nodiscard]] const std::optional<ModelSpec>& Workflow::model_spec(self) const;
```

Notes

Instance method on `self`.

Parameters

None.

Returns

- Returns `const std::optional<ModelSpec>&`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
Workflow workflow{};
auto out = workflow.model_spec(/* args */);
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:1240

Workflow::restore

Signature

```
[[nodiscard]] static Workflow Workflow::restore(self, const io::FlowBundle& bundle);
```

Notes

Instance method on `self`.

Parameters

- `bundle` : `const io::FlowBundle&` Method argument; see signature type for accepted form.

Returns

- Returns `Workflow`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```cpp
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
auto out = Workflow::restore(/* args */);
```

Source

- `PlussAI/include/pluss/ai/lymph/api.h:1632`

`Workflow::set_model`

Signature

```cpp
Workflow& Workflow::set_model(self, const ModelSpec& spec);
```

```cpp
Workflow& Workflow::set_model(self, WorkflowModel model);
```

Notes

Sets or updates internal configuration/state used by later train/infer/evaluate calls.

Parameters

- `model` : `WorkflowModel` Method argument; see signature type for accepted form.
- `spec` : `const ModelSpec&` Method argument; see signature type for accepted form.

Returns

- Returns `Workflow&`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```cpp
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
Workflow workflow{};
auto out = workflow.set_model(/* args */);
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:1249
- PlussAI/include/pluss/ai/lymph/api.h:1255

Workflow::stage_count

Signature

```
[[nodiscard]] std::int64_t Workflow::stage_count(self) const;
```

Notes

Reports a cardinality/count statistic from the current object state.

Parameters

None.

Returns

- Returns std::int64_t.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
Workflow workflow{};
auto out = workflow.stage_count(/* args */);
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:1236

Workflow::task_kind

Signature

```
[[nodiscard]] TaskKind Workflow::task_kind(self) const;
```

Notes

Instance method on self.

Parameters

None.

Returns

- Returns `TaskKind`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
Workflow workflow{};
auto out = workflow.task_kind(/* args */);
```

Source

- `PlussAI/include/pluss/ai/lymph/api.h:1237`

`Workflow::trained`

Signature

```
[[nodiscard]] bool Workflow::trained(self) const;
```

Notes

Boolean state check for object readiness.

Parameters

None.

Returns

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
Workflow workflow{};
auto out = workflow.trained(/* args */);
```

Source

- `PlussAI/include/pluss/ai/lymph/api.h:1235`

Workflow::transform_specs

Signature

```
[[nodiscard]] const std::vector<TransformSpec>& Workflow::transform_specs(self) const;
```

Notes

Instance method on `self`.

Parameters

None.

Returns

- Returns `const std::vector<TransformSpec>&`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
Workflow workflow{};
auto out = workflow.transform_specs(/* args */);
```

Source

- `PlussAI/include/pluss/ai/lymph/api.h:1244`

Workflow::Workflow

Signature

```
explicit Workflow(const ModelSpec& spec) : model_(WorkflowModel(spec));
```

```
explicit Workflow(WorkflowModel model) : model_(std::move(model));
```

Notes

Instance method on `self`.

Parameters

- `arg1 : WorkflowModel model) : model_(std::move(model) | const ModelSpec& spec) : model_(WorkflowModel(spec)` Method argument; see signature type for accepted form.

Returns

- Returns method result for the selected overload.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1

Source: PlussAI/examples/NodeJ/core/bike_sharing_end_to_end.cpp

```
    auto linear_workflow = lymph::Workflow(lymph::linear_factory::linear_
↪regression(LinearFamilyOptions{
        .objective = LinearObjective::LeastSquares,
        .regularization = RegularizationKind::None,
        .regularization_strength = 0.0,
        .fit_intercept = true,
    }));
    linear_workflow
        .simple_imputer(lymph::prep::SimpleImputerOptions{.strategy =␣
↪lymph::prep::SimpleImputerStrategy::Median})
        .standard_scaler();
    linear_workflow.learn_(train_x, train_y, train_options);
    const auto linear_metrics = linear_workflow.assess(test_x, test_y, assess_options);
    std::cout << "linear test metrics [rmse, r2, mae]: " << linear_metrics.rmse << " " <
↪< linear_metrics.r2 << " "
              << linear_metrics.mae << "\n";
    std::cout << "linear params: intercept=" << linear_workflow.intercept() << " coeff="
↪<< linear_workflow.coeff() << "\n";
```

Example 2

Source: PlussAI/examples/NodeJ/core/nodej_workflow_tuning_demo.cpp

```
    std::vector<nja::selection::Candidate> candidates{};
    {
      auto wf = nja::Workflow(nja::linear_factory::linear_regression(LinearFamilyOptions{
          .objective = LinearObjective::LeastSquares,
          .regularization = RegularizationKind::L2,
          .regularization_strength = 1e-6,
          .fit_intercept = true,
      }));
      wf.simple_imputer(nja::prep::SimpleImputerOptions{.strategy =␣
↪nja::prep::SimpleImputerStrategy::Median})
          .standard_scaler();
      candidates.emplace_back("linear_ridge", std::move(wf));
    }
    {
      auto wf = nja::Workflow(nja::tree_factory::decision_tree_
↪regressor(TreeFamilyOptions{
          .criterion = TreeCriterion::Mse,
```

(continues on next page)

(continued from previous page)

```
        .max_depth = 18,
        .min_samples_leaf = 8,
        .min_samples_split = 24,
    }));
    wf.simple_imputer(nja::prep::SimpleImputerOptions{.strategy =␣
↪nja::prep::SimpleImputerStrategy::Median});
    candidates.emplace_back("decision_tree", std::move(wf));
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:1229
- PlussAI/include/pluss/ai/lymph/api.h:1231

WorkflowModel::coefficients

Signature

```
[[nodiscard]] std::vector<double> WorkflowModel::coefficients(self) const;
```

Notes

Instance method on `self`.

Parameters

None.

Returns

- Returns `std::vector<double>`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
WorkflowModel workflowModel{};
auto out = workflowModel.coefficients(/* args */);
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:854

WorkflowModel::dump_model_state

Signature

```
[[nodiscard]] std::vector<Series> WorkflowModel::dump_model_state(self) const;
```

Notes

Instance method on `self`.

Parameters

None.

Returns

- Returns `std::vector<Series>`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
WorkflowModel workflowModel{};
auto out = workflowModel.dump_model_state(/* args */);
```

Source

- `PlussAI/include/pluss/ai/lymph/api.h:901`

`WorkflowModel::dump_transform_states`

Signature

```
[[nodiscard]] std::vector<std::vector<Series>> WorkflowModel::dump_transform_
↪states(self) const;
```

Notes

Instance method on `self`.

Parameters

None.

Returns

- Returns `std::vector<std::vector<Series>>`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
WorkflowModel workflowModel{};
auto out = workflowModel.dump_transform_states(/* args */);
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:897

WorkflowModel::has_model

Signature

```
[[nodiscard]] bool WorkflowModel::has_model(self) const noexcept;
```

Notes

Boolean introspection helper that reports whether a capability/state is available.

Parameters

None.

Returns

- Returns bool.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
WorkflowModel workflowModel{};
auto out = workflowModel.has_model(/* args */);
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:558

WorkflowModel::has_model_spec

Signature

```
[[nodiscard]] bool WorkflowModel::has_model_spec(self) const noexcept;
```

Notes

Boolean introspection helper that reports whether a capability/state is available.

Parameters

None.

Returns

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
WorkflowModel workflowModel{};
auto out = workflowModel.has_model_spec(/* args */);
```

Source

- `PlussAI/include/pluss/ai/lymph/api.h:559`

`WorkflowModel::has_opaque_transforms`

Signature

```
[[nodiscard]] bool WorkflowModel::has_opaque_transforms(self) const noexcept;
```

Notes

Boolean introspection helper that reports whether a capability/state is available.

Parameters

None.

Returns

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
WorkflowModel workflowModel{};
auto out = workflowModel.has_opaque_transforms(/* args */);
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:560

WorkflowModel::intercept_term

Signature

```
[[nodiscard]] double WorkflowModel::intercept_term(self) const;
```

Notes

Instance method on `self`.

Parameters

None.

Returns

- Returns `double`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
WorkflowModel workflowModel{};
auto out = workflowModel.intercept_term(/* args */);
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:859

WorkflowModel::linear_parameters

Signature

```
[[nodiscard]] LinearParameters WorkflowModel::linear_parameters(self) const;
```

Notes

Instance method on `self`.

Parameters

None.

Returns

- Returns `LinearParameters`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
WorkflowModel workflowModel{};
auto out = workflowModel.linear_parameters(/* args */);
```

Source

- `PlussAI/include/pluss/ai/lymph/api.h:816`

`WorkflowModel::load`

Signature

```
[[nodiscard]] static WorkflowModel WorkflowModel::load(self, const std::string& path);
```

Notes

Loads a previously persisted artifact and restores its inference-ready state.

Parameters

- `path` : `const std::string&` Path to persisted model/data artifact.

Returns

- Returns `WorkflowModel`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1

Source: `PlussAI/examples/NodeJ/core/bike_sharing_end_to_end.cpp`

```
    linear_workflow
        .simple_imputer(lymph::prep::SimpleImputerOptions{.strategy =␣
↪lymph::prep::SimpleImputerStrategy::Median})
        .standard_scaler();
    linear_workflow.learn_(train_x, train_y, train_options);
    const auto linear_metrics = linear_workflow.assess(test_x, test_y, assess_options);
```

(continues on next page)

(continued from previous page)

```
    std::cout << "linear test metrics [rmse, r2, mae]: " << linear_metrics.rmse << " " <
↪< linear_metrics.r2 << " "
              << linear_metrics.mae << "\n";
    std::cout << "linear params: intercept=" << linear_workflow.intercept() << " coeff="
↪<< linear_workflow.coeff() << "\n";

    const Series linear_pred_series = linear_workflow.infer(test_x);
    const fs::path linear_model_path = model_dir / "bike_sharing_linear.bin";
    linear_workflow.save(linear_model_path.string(), train_x, train_y, train_options);
    const auto linear_restored = lymph::Workflow::load(linear_model_path.string());
    const Series linear_pred_restored_series = linear_restored.infer(test_x);
    const Series y_true = test_y.to(pluss::pCPU, ScalarType::Float64).reshape({test_y.
↪numel()});

    const Series linear_pred = linear_pred_series.to(pluss::pCPU, ScalarType::Float64).
↪reshape({linear_pred_series.numel()});
    const double linear_max_abs_diff = lymph::max_abs_diff(linear_pred_series, linear_
↪pred_restored_series);
    std::cout << "linear restored prediction max_abs_diff: " << linear_max_abs_diff << "\
↪n";

    DataFrame linear_frame({
        {"y_true", Thread(y_true)},
        {"y_pred", Thread(linear_pred)},
        {"residual", Thread(y_true.sub(linear_pred))},
    });
    linear_frame.save_csv((out_dir / "bike_sharing_test_predictions_linear.csv").
↪string());
    linear_frame.plot()
        .scatter("y_true", "y_pred")
        .headline("Observed vs predicted bike demand (linear)")
        .save_pdf((out_dir / "bike_pred_vs_true_linear.pdf").string());
    linear_frame.plot()
        .hist("residual", 80)
        .headline("Residual distribution (linear)")
```

Example 2

Source: PlussAI/examples/NodeJ/core/nodej_workflow_tuning_demo.cpp

```
                << " cv_mae=" << trial.cv.mean.mae << "\n";
    }
    std::cout << "best candidate: " << tuned.report.best_name << "\n";

    auto best = std::move(tuned.best_workflow);
    best.learn_(train_x, train_y, train_options);
    const auto test_metrics = best.assess(test_x, test_y, assess_options);
    std::cout << "test metrics [rmse, r2, mae]: " << test_metrics.rmse << " " << test_
↪metrics.r2 << " " << test_metrics.mae << "\n";

    const Series pred = best.infer(test_x);
    const fs::path best_model_path = model_dir / "hour_best_workflow.bin";
```

(continues on next page)

(continued from previous page)

```
    best.save(best_model_path.string(), train_x, train_y, train_options);
    auto restored = nja::Workflow::load(best_model_path.string());
    const Series pred_restored = restored.infer(test_x);
    const double restored_max_abs_diff = nja::max_abs_diff(pred, pred_restored);
    std::cout << "restored prediction max_abs_diff: " << restored_max_abs_diff << "\n";

    const Series y_true = test_y.to(pluss::pCPU, ScalarType::Float64).reshape({test_y.
↪numel()});
    const Series y_pred = pred.to(pluss::pCPU, ScalarType::Float64).reshape({pred.
↪numel()});
    DataFrame pred_frame({
        {"y_true", Thread(y_true)},
        {"y_pred", Thread(y_pred)},
        {"residual", Thread(y_true.sub(y_pred))},
    });
    pred_frame.save_csv((out_dir / "hour_workflow_tuning_predictions.csv").string());
    pred_frame.plot()
        .scatter("y_true", "y_pred")
        .headline("Observed vs predicted bike demand (best workflow)")
        .save_pdf((out_dir / "hour_workflow_tuning_pred_vs_true.pdf").string());

    std::cout << "saved best model: " << best_model_path << "\n";
    std::cout << "saved outputs under: " << out_dir << "\n";
    return 0;
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:893

WorkflowModel::model_spec

Signature

```
[[nodiscard]] const std::optional<ModelSpec>& WorkflowModel::model_spec(self) const␣
↪noexcept;
```

Notes

Instance method on `self`.

Parameters

None.

Returns

- Returns `const std::optional<ModelSpec>&`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
WorkflowModel workflowModel{};
auto out = workflowModel.model_spec(/* args */);
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:563

WorkflowModel::restore

Signature

```
[[nodiscard]] static WorkflowModel WorkflowModel::restore(self, const io::FlowBundle&␣
↪bundle);
```

Notes

Instance method on `self`.

Parameters

- `bundle` : `const io::FlowBundle&` Method argument; see signature type for accepted form.

Returns

- Returns `WorkflowModel`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
auto out = WorkflowModel::restore(/* args */);
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:889

WorkflowModel::stage_count

Signature

```
[[nodiscard]] std::int64_t WorkflowModel::stage_count(self) const noexcept;
```

Notes

Reports a cardinality/count statistic from the current object state.

Parameters

None.

Returns

- Returns `std::int64_t`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
WorkflowModel workflowModel{};
auto out = workflowModel.stage_count(/* args */);
```

Source

- `PlussAI/include/pluss/ai/lymph/api.h:561`

`WorkflowModel::task_kind`

Signature

```
[[nodiscard]] TaskKind WorkflowModel::task_kind(self) const noexcept;
```

Notes

Instance method on `self`.

Parameters

None.

Returns

- Returns `TaskKind`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
WorkflowModel workflowModel{};
auto out = workflowModel.task_kind(/* args */);
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:562

WorkflowModel::trained

Signature

```
[[nodiscard]] bool WorkflowModel::trained(self) const noexcept;
```

Notes

Boolean state check for object readiness.

Parameters

None.

Returns

- Returns bool.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
WorkflowModel workflowModel{};
auto out = workflowModel.trained(/* args */);
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:557

WorkflowModel::transform_specs

Signature

```
[[nodiscard]] const std::vector<TransformSpec>& WorkflowModel::transform_specs(self)␣
↪const noexcept;
```

Notes

Instance method on self.

Parameters

None.

Returns

- Returns `const std::vector<TransformSpec>&`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
WorkflowModel workflowModel{};
auto out = workflowModel.transform_specs(/* args */);
```

Source

- `PlussAI/include/pluss/ai/lymph/api.h:564`

`WorkflowModel::workflow`

Signature

```
[[nodiscard]] const Workflow& WorkflowModel::workflow(self) const noexcept;
```

```
[[nodiscard]] Workflow& WorkflowModel::workflow(self) noexcept;
```

Notes

Instance method on `self`.

Parameters

None.

Returns

- Returns `Workflow&`.
- Returns `const Workflow&`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1

Source: `PlussAI/examples/NodeJ/core/nodej_workflow_tuning_demo.cpp`

```
    const Series y_true = test_y.to(pluss::pCPU, ScalarType::Float64).reshape({test_y.
↪numel()});
    const Series y_pred = pred.to(pluss::pCPU, ScalarType::Float64).reshape({pred.
```

(continues on next page)

(continued from previous page)

```
↪numel()});
    DataFrame pred_frame({
        {"y_true", Thread(y_true)},
        {"y_pred", Thread(y_pred)},
        {"residual", Thread(y_true.sub(y_pred))},
    });
    pred_frame.save_csv((out_dir / "hour_workflow_tuning_predictions.csv").string());
    pred_frame.plot()
        .scatter("y_true", "y_pred")
        .headline("Observed vs predicted bike demand (best workflow)")
        .save_pdf((out_dir / "hour_workflow_tuning_pred_vs_true.pdf").string());
```

Example 2

Source: PlussAI/examples/NodeJ/lymph/lymph_end_to_end_hour_workflow_showcase.cpp

```
    //========== schema normalization ========
    // This section makes feature names explicit and consistent for the ML workflow.
    auto frame = raw.rename_features({
        {"dteday", "date"},
        {"holiday", "is_holiday"},
        {"workingday", "is_workingday"},
        {"weathersit", "weather_condition"},
        {"hum", "humidity"},
        {"mnth", "month"},
        {"cnt", "total_count"},
        {"hr", "hour"},
        {"yr", "year"},
    });
```

Source

- PlussAI/include/pluss/ai/lymph/api.h:928
- PlussAI/include/pluss/ai/lymph/api.h:929

WorkflowModel::WorkflowModel

Signature

```
explicit WorkflowModel(const ModelSpec& spec) : workflow_(resolve(make_flow(spec), "make_
↪flow"));
```

```
explicit WorkflowModel(Workflow workflow) : workflow_(std::move(workflow));
```

Notes

Instance method on `self`.

Parameters

- `arg1 : Workflow workflow) :  workflow_(std::move(workflow) | const ModelSpec& spec) : workflow_(resolve(make_flow(spec), "make_flow")` Method argument; see signature type for accepted form.

Returns

- Returns method result for the selected overload.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

```
// Minimal call skeleton. Replace placeholders with concrete data/model objects.
WorkflowModel workflowModel{};
auto out = workflowModel.WorkflowModel(/* args */);
```

Source

- `PlussAI/include/pluss/ai/lymph/api.h:543`
- `PlussAI/include/pluss/ai/lymph/api.h:548`

CHAPTER

SIX

BUILD OUTPUTS

- HTML root: `PlussAI/docs/sphinx/_build/html/index.html`
- PDF output: `PlussAI/docs/sphinx/_build/latex/plussai_nodej_lymph_api_docs.pdf`

Build commands:

```
sphinx-build -b html PlussAI/docs/sphinx PlussAI/docs/sphinx/_build/html
sphinx-build -M latexpdf PlussAI/docs/sphinx PlussAI/docs/sphinx/_build
```